

GitSky

Un template industriel pour la startup factory

Nabil Mabrouk

2026

Table des matières

Introduction : GitSky, un Template Industriel pour la Startup Factory

1 Architecture Conteneurisée Professionnelle

- 1.1 Philosophie de l'Architecture
- 1.2 Vue d'Ensemble des Services
- 1.3 Orchestration avec Docker Compose
- 1.4 Gestion des Variables d'Environnement
- 1.5 Reverse Proxy et SSL avec Traefik
- 1.6 Hébergement Multi-Projets sur un Seul VPS
- 1.7 Commandes de Référence

2 Les Trois Tiers de GitSky : To, T1, T2

- 2.1 Introduction
- 2.2 1. Le Paradoxe de la Startup Factory
- 2.3 2. Les Trois Tiers en Synthèse
- 2.4 3. Ce qui est Activé à Chaque Tier
- 2.5 4. Critères de Promotion entre Tiers
- 2.6 5. Comment Monter un Projet de Tier
- 2.7 6. Le Kill Mechanism : Arrêter à Temps
- 2.8 7. Anti-Patterns à Éviter
- 2.9 Checklist du Chapitre

3 Initialisation du Backend avec FastAPI

- 3.1 Pourquoi FastAPI ?
- 3.2 Structure d'un Projet Professionnel
- 3.3 Le Système de Modules
- 3.4 Configuration Centralisée avec Pydantic Settings
- 3.5 Connexion à la Base de Données (SQLAlchemy)
- 3.6 Le Point d'Entrée : `main.py`
- 3.7 Lancer le Backend

4 Modélisation des Données avec SQLAlchemy

- 4.1 Introduction à la Modélisation
- 4.2 Les Modèles du Core
- 4.3 Les Modèles des Modules
- 4.4 Table Récapitulative des Modèles par Couche
- 4.5 Gestion des Migrations avec Alembic

5 L'API Core et les Contrats de Modules

- 5.1 Introduction
- 5.2 Validation des Données avec Pydantic

- 5.3 Contrat d'Interface d'un Module
- 5.4 Chargement Dynamique des Routers
- 5.5 Services Transverses : Clients Partagés
- 5.6 Aperçu du Framework Agentic IA

6 Le Frontend Moderne avec React et Vite

- 6.1 Pourquoi cette Stack ?
- 6.2 Structure du Projet Frontend
- 6.3 Tailwind CSS 4 et le Design System
- 6.4 Gestion des Alias de Chemin
- 6.5 L'importance de Vite en Développement

7 Authentification et Sécurité de Session

- 7.1 Introduction
- 7.2 Gestion Globale de l'État : AuthContext
- 7.3 Stratégie de Sécurité : JWT et Cookies
- 7.4 Fin de Session et Révocation
- 7.5 Protection des Routes (Guards)
- 7.6 Rôles et Progression Utilisateur
- 7.7 Authentification et Système à Trois Tiers

8 Internationalisation (Module i18n)

- 8.1 Introduction
- 8.2 Architecture i18n
- 8.3 Configuration
- 8.4 Utilisation dans un Composant
- 8.5 Contenu Multilingue en Base
- 8.6 Bonnes Pratiques
- 8.7 Impact sur le SEO

9 Dashboard Admin : Shell Extensible

- 9.1 Introduction
- 9.2 Architecture du Shell
- 9.3 Onglets du Shell (Toujours Présents)
- 9.4 Onglets Apportés par les Modules
- 9.5 Bibliothèque de Médias
- 9.6 Sécurité du Dashboard

10 SEO et Optimisation pour la Visibilité

- 10.1 Introduction
- 10.2 Gestion Dynamique des Meta Tags
- 10.3 Sitemap et Robots.txt Dynamiques
- 10.4 Données Structurées (JSON-LD)

10.5 Intégration avec le Module i18n

10.6 Indexation Sélective (noindex)

10.7 SEO dès le Tier To

11 Créer un Module Métier : L'Exemple de l'Université Virtuelle

11.1 Introduction

11.2 Anatomie du Module

11.3 Le Catalogue Public

11.4 Le Moteur de Rendu Markdown

11.5 Contrôle d'Accès au Contenu

11.6 L'Onglet Admin du Module

11.7 Le Contrat d'Interface, Point par Point

11.8 Reproduire ce Pattern pour son Propre Métier

12 Module Onboarding : Profilage Dynamique

12.1 Introduction

12.2 Le Flow JSON : Règles Métier sans Code

12.3 L'Expérience Utilisateur (Frontend)

12.4 Modes d'Utilisation

12.5 L'Endpoint d'Évaluation

12.6 Persistance du Profil

12.7 Créer un Flow d'Onboarding pour son Projet

13 Module Analytics : GeoIP et Visualisation

13.1 Introduction

13.2 Le Middleware de Tracking

13.3 Le Service GeoIP Partagé

13.4 Visualisations Admin

13.5 Endpoints Analytics

13.6 Conformité RGPD

13.7 Purge et Rétention

14 Module Security : Détection d'Intrusion

14.1 Introduction

14.2 Philosophie : Journaliser, ne pas Bloquer

14.3 Types d'Événements Détectés

14.4 Le Modèle SecurityEvent

14.5 Interface Admin

14.6 Complémentarité avec Traefik

14.7 Purge et Rétention

14.8 Ce que le Middleware ne Fait Pas

15 Framework Agentic IA et Services Intelligents

- 15.1 Introduction au Framework Multi-Agents
- 15.2 Architecture du Système Multi-Agents
- 15.3 Le LLM Proxy Partagé
- 15.4 Configuration des Services via YAML
- 15.5 Modèles de Données pour le Tracking des Exécutions
- 15.6 Le Moteur d'Orchestration
- 15.7 Tâches Longues : Job Asynchrone, sans Broker
- 15.8 API Agent-Services
- 15.9 Dashboard Agentic Frontend
- 15.10 Création d'un Nouveau Service Agentic
- 15.11 Intégration avec les Autres Composants de GitSky
- 15.12 Scénarios d'Utilisation Avancés
- 15.13 Dépannage et Maintenance

16 Monétisation avec Stripe : Boutique et Abonnements

- 16.1 Introduction
- 16.2 Architecture Webhook-First
- 16.3 Stripe Multi-Projets : un Compte, N Produits
- 16.4 Sécurité du Webhook Router
- 16.5 Boutique de Produits Numériques
- 16.6 Abonnements Stripe
- 16.7 Webhook — Le Cœur du Système
- 16.8 Pages Frontend
- 16.9 Checklist de Mise en Production

17 Le Générateur `create-gitsky-project`

- 17.1 Introduction
- 17.2 Choix Technique : Copier
- 17.3 Le Fichier `config.yaml`
- 17.4 Génération d'un Nouveau Projet
- 17.5 Logique Complexe : Context Hooks, Tasks et Migrations
- 17.6 Mise à Jour d'un Projet Existant
- 17.7 Bootstrapping d'une Flotte : Zero-to-N Projets
- 17.8 Anti-Patterns à Éviter

18 Services Partagés sur un Seul VPS

- 18.1 Introduction
- 18.2 Vue d'Ensemble
- 18.3 1. Traefik : Routage HTTPS Mutualisé
- 18.4 2. PostgreSQL : un conteneur par projet
- 18.5 3. Landing Collector
- 18.6 4. LLM Proxy Partagé

18.7 5. Relais SMTP

18.8 6. Service GeoIP

18.9 7. Router de Webhooks Stripe

18.10 Sécurité des Services Partagés

18.11 Coût Total des Services Partagés

19 Fleet Dashboard : la Vue Unifiée de la Flotte

19.1 Introduction

19.2 Architecture

19.3 Vue Principale : la Grille de Projets

19.4 Onglets Détaillés par Projet

19.5 Sources de Données du Dashboard

19.6 Les Alertes Automatiques

19.7 Métriques Agrégées de Flotte

19.8 Le Fleet Dashboard comme Contrat

19.9 Un Kill est un Événement du Dashboard, pas un Script Manuel

20 Cycle de Vie d'un Projet dans la Flotte

20.1 Introduction

20.2 Vue d'Ensemble : Cinq Stages

20.3 Stage 0 — Harvest

20.4 Stage 1 — Tier To (Landing)

20.5 Stage 2 — Tier T1 (MVP Lite)

20.6 Stage 3 — Tier T2 (SaaS Complet)

20.7 Migration Sous-Domaine → Domaine Premier

20.8 Stage 4 — Émancipation ou Consolidation

20.9 Logique d'Évaluation du `kill_check`

20.10 Kill Mechanism : le Détail Opérationnel

20.11 Reconstitution d'un Projet Killed

20.12 Le Journal de la Flotte

21 Dockerisation Avancée pour la Production

21.1 Introduction

21.2 Le Pattern Multi-Stage

21.3 Sécurité des Conteneurs

21.4 Serveur de Production : Gunicorn + Uvicorn

21.5 Surveillance de l'État (Healthchecks)

21.6 Un Seul Dockerfile pour Trois Tiers

21.7 Empreinte Mémoire par Tier (Mesurée)

22 Configuration du Serveur de Production

22.1 Ubuntu Server 24.04 — Sécurisation, Docker et Accès SSH

- 22.2 Prérequis et Contexte
- 22.3 Première Connexion avec PuTTY
- 22.4 Sécurisation SSH par Clé Publique
- 22.5 Mise à Jour du Système
- 22.6 Installation des Paquets de Base
- 22.7 Configuration du Pare-feu UFW
- 22.8 Configuration de Fail2ban
- 22.9 Installation de Docker
- 22.10 Structure des Répertoires
- 22.11 Déploiement du Traefik Partagé
- 22.12 Récapitulatif de l'État du Serveur
- 22.13 FAQ
- 22.14 Bootstrap des Services Partagés pour une Flotte

23 Maintenance en Production : Sécurité, Sauvegardes et Surveillance

- 23.1 Pourquoi la Maintenance est Non-Négociable
- 23.2 Partie 1 : Sauvegardes PostgreSQL
- 23.3 Partie 2 : Durcissement de la Sécurité
- 23.4 Partie 3 : Santé de la Base de Données PostgreSQL
- 23.5 Partie 4 : Monitoring et Surveillance
- 23.6 Partie 5 : Mises à Jour
- 23.7 Partie 6 : Calendrier de Maintenance
- 23.8 Partie 7 : Procédures d'Urgence
- 23.9 Maintenance à l'Échelle de la Flotte
- 23.10 Conclusion

24 GitSky Studio : Direction Artistique de Flotte

- 24.1 Introduction
- 24.2 Le Châssis et la Vitrine
- 24.3 Le Pipeline d'Agents
- 24.4 Le Creative Manifest : Geler la Sortie de l'IA
- 24.5 Le Modèle de Publication
- 24.6 Itération : des Diffs sur le Manifest
- 24.7 Les Guardrails : Cadrer, pas Brider
- 24.8 La Boucle de Feedback : la Vraie Douve
- 24.9 Anti-Patterns à Éviter

25 Guide de l'Opérateur : Utiliser, Déployer, Maintenir GitSky

- 25.1 1. Utiliser : de l'idée au projet
- 25.2 2. Déployer : du projet à la mise en ligne
- 25.3 3. Maintenir : garder la flotte en bonne santé
- 25.4 Le cycle complet, en une image

26 Conclusion : GitSky, un Template pour Industrialiser le SaaS

26.1 Ce que nous avons construit

26.2 Les Huit Principes Fondamentaux

26.3 Ce que GitSky rend Possible

26.4 Prochaines Étapes

26.5 Un Mot sur la Philosophie GitSky

Introduction : GitSky, un Template Industriel pour la Startup Factory

Cet ouvrage ne décrit pas la construction d'une seule webapp. Il pose les fondations d'un **template industriel** — GitSky — capable de porter simultanément un grand nombre de projets à des stades de maturité très différents, du test rapide d'une idée non validée jusqu'à la webapp en production avec revenus récurrents.

GitSky, un Template et une Discipline

GitSky n'est pas un produit fini, c'est un système :

- **Un template paramétrable** basé sur FastAPI, React et PostgreSQL, capable de générer un projet prêt à démarrer via un générateur automatisé (`create-gitsky-project`).
- **Un système à trois tiers** — T0 (Landing), T1 (MVP lite), T2 (SaaS complet) — pour ajuster la complexité et le coût de chaque projet à son stade de validation.
- **Une flotte industrielle** hébergée sur un unique VPS mutualisé, avec services partagés (Traefik, PostgreSQL, LLM proxy, landing-collector) et un fleet dashboard central pour piloter l'ensemble.
- **Une discipline de portefeuille** avec critères de promotion numériques et mécanisme de kill automatique, pour concentrer les ressources sur les projets qui prouvent leur valeur.

L'objectif est de compresser le temps entre l'idée et la webapp déployée, de plusieurs semaines à quelques heures, tout en préservant la qualité d'une architecture professionnelle.

Architecture Cible : un Core Léger, des Modules Optionnels

Au cœur de GitSky se trouve une architecture stricte en couches :

- **GitSky-core** : Docker infra, FastAPI shell, gestion des utilisateurs et rôles, authentification JWT, admin shell, SEO de base. Présent à tous les tiers.
- **GitSky-modules** : onboarding dynamique, système de contenu (tutoriaux/leçons), analytics GeoIP, security middleware, framework agentic IA, i18n, monétisation Stripe. Chacun activable indépendamment via variable d'environnement.
- **GitSky-app** : la couche applicative métier propre à chaque projet (modèles de données, endpoints, UI spécifiques).

Le stack technique est unifié à tous les tiers :

- **Backend FastAPI** avec SQLAlchemy asynchrone et Pydantic Settings.
- **Frontend React 19 + Vite + Tailwind 4** pour la vitesse et la réactivité.
- **PostgreSQL 16** comme socle de persistance, avec une base par projet pour une isolation nette.

- **Docker Compose** pour l'orchestration, **Traefik partagé** pour le routage HTTPS multi-projets.
- **Framework agentic** activable pour les projets nécessitant des services IA.

Chaque projet ne paie que le coût des modules qu'il active — un T0 en RAM tient dans quelques dizaines de mégaoctets, un T2 monte à quelques centaines.

Feuille de Route de l'Ouvrage

Ce livre est structuré en cinq parties :

1. **Fondations et Philosophie** — Architecture conteneurisée multi-projets, système à trois tiers.
2. **GitSky-core** — Ce qui est présent à tous les tiers : initialisation FastAPI, modèles core, authentification, admin shell, frontend, SEO.
3. **Modules Optionnels** — i18n, onboarding, contenu pédagogique, analytics, sécurité, framework agentic, monétisation Stripe.
4. **Industrialisation** — Générateur `create-gitsky-project`, services partagés sur un VPS, fleet dashboard, cycle de vie complet d'un projet.
5. **Production et Maintenance** — Docker de production (un seul artefact, calibré par tier), configuration serveur Ubuntu 24.04, sauvegardes et surveillance de flotte, et un guide opérateur de bout en bout : utiliser, déployer, maintenir.

À l'issue de ce parcours, vous disposerez non seulement d'un template exploitable pour lancer un portefeuille de projets, mais surtout d'une discipline opérationnelle pour maintenir la flotte en bonne santé.

Dr. Nabil MABROUK

1 Architecture Conteneurisée Professionnelle

1.1 Philosophie de l'Architecture

Une architecture conteneurisée professionnelle repose sur quatre principes fondamentaux pour garantir la fiabilité, la scalabilité et la multi-tenance du template **GitSky**.

Parité des environnements : Le code qui tourne en développement sur votre machine doit être structurellement identique à celui qui tourne en production sur le serveur. Seules les configurations (secrets, URLs, niveaux de logs) changent.

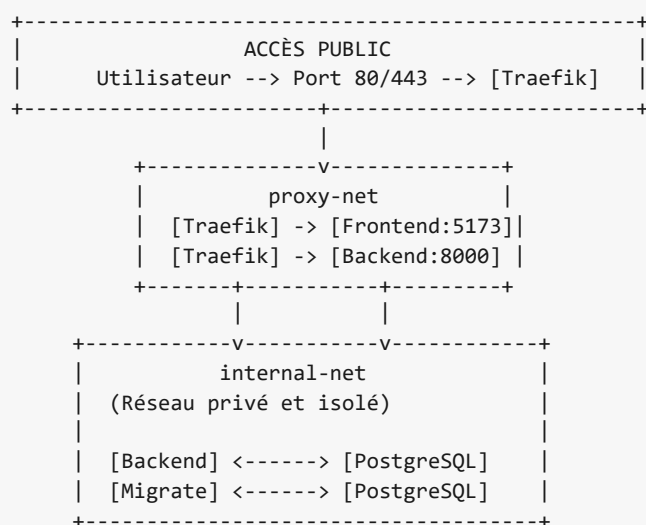
Isolation et Segmentation : Chaque service est confiné dans son propre conteneur. Le frontend ne communique jamais directement avec la base de données ; tout passe par l'API Backend. Cette segmentation limite les risques de sécurité.

Infrastructure-as-Code : Toute notre infrastructure est décrite dans des fichiers YAML versionnés. Recréer l'intégralité d'un projet GitSky ne nécessite qu'une seule commande.

Multi-tenance native : Un même VPS doit pouvoir héberger simultanément des dizaines de projets isolés. Cela impose que le nom du projet (`PROJECT_NAME`) paramètre l'ensemble des ressources (conteneurs, réseaux, base de données), et que les composants partagés — Traefik en premier lieu — le soient explicitement.

1.2 Vue d'Ensemble des Services

Le projet GitSky est composé de trois briques majeures orchestrées par Docker :



Les services clés :

- **Frontend (React/Vite) :** L'interface utilisateur moderne et réactive.
- **Backend (FastAPI) :** Le cerveau traitant la logique métier et l'accès aux données.
- **Database (PostgreSQL) :** Le stockage persistant et robuste.
- **Migrate (Alembic) :** Un service éphémère qui met à jour le schéma de la base de données au démarrage.

1.3 Orchestration avec Docker Compose

L'orchestration est gérée par trois fichiers distincts pour séparer la structure de la configuration d'environnement.

1.3.1 1. La Base Commune : `docker-compose.yml`

Ce fichier définit la “squelette” de notre application sans valeurs en dur.

```

services:
  frontend:
    container_name: ${PROJECT_NAME}_frontend
    environment:
      - VITE_API_URL=${VITE_API_URL}
    networks:
      - proxy-net
      - internal-net
    depends_on:
      - backend

  backend:
    container_name: ${PROJECT_NAME}_backend
    environment:
      - DATABASE_URL=postgresql+asyncpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      - ENVIRONMENT=${ENVIRONMENT}
      - SECRET_KEY=${SECRET_KEY}
    networks:
      - proxy-net
      - internal-net
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16.3-alpine
    container_name: ${PROJECT_NAME}_db
    restart: unless-stopped
    environment:
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
      - POSTGRES_DB=${POSTGRES_DB}
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - internal-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  migrate:
    container_name: ${PROJECT_NAME}_migrate
    command: python -m scripts.migrate # applique core + chaînes des modules activés
    environment:
      - DATABASE_URL=postgresql+asyncpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
    networks:
      - internal-net
    depends_on:
      db:
        condition: service_healthy
    restart: "no"

volumes:
  db_data:

networks:
  proxy-net:
    external: true
    name: proxy-net
  internal-net:
    driver: bridge
    internal: true

```

1.3.2 2. L'environnement de Développement : `docker-compose.dev.yml`

En développement, nous activons le rechargement automatique (hot-reload) et exposons les ports pour faciliter le débogage.

```
services:
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile.dev
    volumes:
      - ./frontend:/app
      - /app/node_modules
    ports:
      - "5173:5173"

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile.dev
    volumes:
      - ./backend:/app
      - ./backend/uploads:/app/uploads
    ports:
      - "8000:8000"

  db:
    ports:
      - "5432:5432" # Permet d'utiliser DBeaver ou TablePlus localement
```

1.4 Gestion des Variables d'Environnement

Un projet professionnel ne doit jamais contenir de secrets (mots de passe, clés API) dans le code source. Nous utilisons un fichier `.env` (ignoré par Git) basé sur un modèle `.env.example`.

Le fichier `.env.example` type pour GitSky :

```
PROJECT_NAME=gitsky
ENVIRONMENT=development
VITE_API_URL=http://localhost:8000
SECRET_KEY=votre_cle_secrete_tres_longue

# PostgreSQL
POSTGRES_USER=hitl_user
POSTGRES_PASSWORD=hitl_password
POSTGRES_DB=hitl_db
```

1.5 Reverse Proxy et SSL avec Traefik

Pour la production, nous utilisons **Traefik**. C'est un reverse proxy moderne qui gère automatiquement les certificats SSL (HTTPS) via Let's Encrypt.

L'architecture GitSky prévoit que Traefik tourne dans un conteneur séparé, partageant le réseau `proxy-net`. Chaque service (Frontend et Backend) s'annonce à Traefik via des **labels** Docker :

```
# Exemple de labels pour le Backend en production
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.backend-gitsky.rule=Host(`api.votre-domaine.com`)"
  - "traefik.http.routers.backend-gitsky.entrypoints=websecure"
  - "traefik.http.routers.backend-gitsky.tls.certresolver=letsencrypt"
```

1.6 Hébergement Multi-Projets sur un Seul VPS

Le template GitSky est conçu pour cohabiter à plusieurs dizaines d'instances sur un unique VPS. Trois mécanismes rendent cela viable.

1.6.1 Un Seul Trafik pour Toute la Flotte

Un unique conteneur Traefik tourne à l'échelle du VPS et route le trafic vers l'ensemble des projets hébergés. Chaque projet expose ses labels Traefik et rejoint le réseau `proxy-net` **partagé entre tous les projets** — Traefik les découvre automatiquement à leur démarrage.

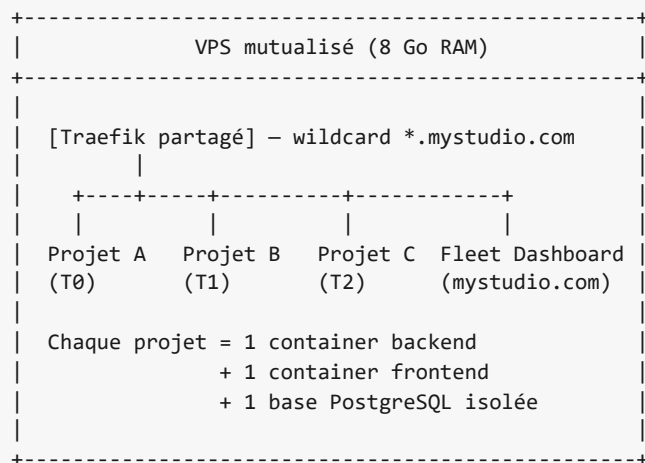
1.6.2 Certificats SSL Wildcard

Traefik gère un certificat wildcard `*.mystudio.com` via Let's Encrypt DNS-01. Ajouter un nouveau sous-domaine à un projet ne déclenche donc aucune renégociation de certificat. Le domaine premier (`mystudio.com`) est réservé au fleet dashboard qui pilote la flotte (voir Chap 19).

1.6.3 Isolation Stricte par Projet

Chaque projet dispose :

- de son propre réseau `internal-net` (jamais partagé avec un autre projet)
- de sa propre base de données PostgreSQL (isolation nette, restauration ou kill facile)
- de ses propres conteneurs backend et frontend
- d'un `PROJECT_NAME` unique qui préfixe l'ensemble des ressources Docker



Sur un VPS à 20 €/mois, ce modèle permet d’héberger simultanément **une centaine de projets To**, **une vingtaine de T1**, et **quelques T2** sans saturation (voir Chap 2 pour le détail des tiers).

1.7 Commandes de Référence

Voici les commandes essentielles pour piloter votre infrastructure :

Action	Commande
Lancer en développement	<code>docker compose -f docker-compose.yml -f docker-compose.dev.yml up</code>
Reconstruire les images	<code>docker compose build --no-cache</code>
Voir les logs en temps réel	<code>docker compose logs -f</code>
Arrêter et supprimer les conteneurs	<code>docker compose down</code>

Ce chapitre pose les fondations techniques mutualisées du template. Dans le prochain chapitre, nous détaillons le système à trois tiers qui permet à un même code base de porter aussi bien un test de landing éphémère qu’une webapp en production avec revenus récurrents.

2 Les Trois Tiers de GitSky : To, T1, T2

2.1 Introduction

GitSky n'est pas destiné à un projet unique. Il est conçu comme un **template industriel** capable de porter un grand nombre de projets à des stades de maturité très différents : du test de demande à la webapp en production avec revenus récurrents. Un template mono-taille échoue toujours d'un côté ou de l'autre — surdimensionné pour tester une idée, sous-dimensionné pour porter un produit validé.

La solution retenue est un système à **trois tiers**, activables via un système de feature flags. Un même code base, trois profils : **To** (Landing), **T1** (MVP lite), **T2** (SaaS complet). Chaque projet naît en To, monte de tier uniquement s'il fournit un signal mesurable, et est arrêté sinon.

2.2 1. Le Paradoxe de la Startup Factory

Générer des idées coûte peu. Les valider coûte cher. Les développer coûte encore plus. Cette asymétrie impose une discipline :

- **Beaucoup de tests légers** en amont, pour laisser émerger le signal.
- **Un investissement croissant** à mesure que le signal se confirme.
- **Un mécanisme de kill** systématique pour libérer les ressources des projets sans signal.

Un template GitSky déployé en pleine version pour tester chaque idée serait un gaspillage de ressources (RAM, disque, temps humain) et un frein à la vitesse — chaque instance prendrait des heures à mettre en place. Inversement, une simple page HTML statique ne suffirait plus une fois qu'un projet commence à avoir de vrais utilisateurs.

Les trois tiers réconcilient ces deux exigences dans un seul template.

2.3 2. Les Trois Tiers en Synthèse

Tier	Cas d'usage	Temps de déploiement	Empreinte RAM	Coût mensuel indicatif
To — Landing	Test de demande sur une idée non validée	< 5 min	~50 Mo	~0,20 €/mois
T1 — MVP lite	Produit minimal pour valider un usage réel	< 2 h	~200 Mo	~2 €/mois
T2 — SaaS complet	Produit avec traction, monétisation active	1 à 3 jours	~700 Mo	~7 €/mois + coûts LLM/Stripe à l'usage

Ces coûts sont calibrés sur un **VPS mutualisé de 8 Go de RAM à 20 €/mois**, hébergeant plusieurs projets simultanément derrière un unique Traefik. Sur cette base :

- Le VPS peut porter **100+ instances To** sans saturation.
- Il peut porter **10 à 20 instances T1** avec confort.
- Il peut porter **3 à 5 instances T2** en production.

Cette économie d'échelle est la raison d'être du choix d'un VPS partagé plutôt que d'un hébergement scale-to-zero externe. Le prix marginal d'un To supplémentaire est proche de zéro tant que le VPS n'est pas saturé.

2.4 3. Ce qui est Activé à Chaque Tier

Chaque module de GitSky peut être activé ou désactivé indépendamment via une variable d'environnement `MODULE_*`. Les trois tiers ne sont que des **profils par défaut** de ces flags — un projet peut toujours surcharger un flag pour un besoin spécifique.

Module	To	T1	T2
Authentication (JWT + refresh)	✗	✓	✓
Admin shell	✗	✗	✓
Internationalisation (i18n FR/EN)	✗	✗	✓
Analytics GeoIP + world map	via collector partagé	✓	✓
Onboarding dynamique	✗	✗	✓
Content system (tutorials/lessons)	✗	✗	selon projet
Framework agentic IA	✗	optionnel	✓
Monétisation boutique (Stripe)	✗	✗	✓
Monétisation abonnements	✗	✗	✓
SecurityMiddleware	via proxy Traefik	✓	✓
SEO dynamique	✓	✓	✓

Le principe : **un module désactivé ne charge aucune route, aucun modèle SQL, aucune migration Alembic**. L'empreinte mémoire d'un To est donc effectivement minimale, non pas parce que le code manque, mais parce que le code inutile est court-circuité au démarrage.

Le fichier `.env` d'un To typique :

```
GITSKY_TIER=t0
PROJECT_NAME=pain-scraper
VITE_API_URL=https://pain-scraper.mystudio.com
# Tous les MODULE_* sont désactivés par défaut du profil T0
```

Un T2 avec agentic activé :

```
GITSKY_TIER=t2
PROJECT_NAME=code-reviewer-pro
MODULE_AGENTIC=true # surcharge du profil T2 si besoin
MODULE_MONETIZATION_SUBSCRIPTION=true
STRIPE_SECRET_KEY=sk_live_xxx
ANTHROPIC_API_KEY=sk-ant-xxx
```

2.5 4. Critères de Promotion entre Tiers

Un projet ne monte pas de tier par décision arbitraire. Chaque promotion est **conditionnée à un signal numérique mesurable**, collecté par le landing-collector partagé ou le tracking analytics du projet.

2.5.1 To → T1 : Le Signal de Demande

Un projet To est promu en T1 lorsqu'il satisfait **au moins une** des conditions suivantes, mesurées sur les 21 premiers jours d'exposition :

Signal	Seuil
Conversion visiteurs → captures email	≥ 3 % sur trafic qualifié (≥ 500 visites)
Nombre absolu d'inscriptions	≥ 30 emails collectés
Retours qualitatifs actionables	≥ 3 réponses à un email de suivi décrivant un besoin précis

Le seuil de 3 % est calibré sur des benchmarks courants de landings B2B — en dessous, la traction est probablement fortuite. Le seuil de 500 visites élimine les projets qui n'ont pas encore été testés en conditions réelles.

2.5.2 T1 → T2 : Le Signal d'Usage et de Volonté de Payer

Un projet T1 est promu en T2 lorsqu'il satisfait **toutes** les conditions suivantes après 30 jours d'exposition :

Signal	Seuil
Utilisateurs actifs (usage ≥ 3 fois par semaine)	≥ 10
Rétention D7 (utilisateurs revenant à J+7)	≥ 30 %
Signal de volonté de payer	≥ 1 paiement réel OU ≥ 3 déclarations explicites

La rétention D7 est le meilleur prédicteur simple de la survie d'un produit. En dessous de 30 %, ajouter de la monétisation ne convertira personne — le problème n'est pas le paiement, c'est l'usage.

2.5.3 La Règle Inverse : Ne Jamais Naître en T2

L'anti-pattern le plus coûteux est de démarrer directement en T2, en pariant qu'une idée « évidente » réussira. Le coût :

- **Semaines de développement** avant le moindre test utilisateur.
- **Opportunité perdue** sur les autres idées non testées pendant ce temps.
- **Attachement émotionnel** au produit fini, qui rend le kill quasi impossible.

Règle absolue : chaque projet naît en T0. Aucune exception.

2.6 5. Comment Monter un Projet de Tier

La montée d'un tier n'est jamais une réécriture — c'est une **activation de modules** et une migration de données incrémentale.

2.6.1 T0 → T1

1. **Mise à jour du .env** : `GITSKY_TIER=t1`, activation de `MODULE_AUTH` et des modules métier propres au projet.

2. **Migration des leads en users** : les emails collectés en To dans le landing-collector partagé sont importés dans la table `users` du projet avec `role=waitlist`.
3. **Alembic upgrade** : les tables nécessaires aux modules activés (users, sessions, tables métier) sont créées.
4. **Rebuild et déploiement** : `docker compose build && docker compose up -d`. Le sous-domaine reste identique (`pain-scrapers.mystudio.com`).
5. **Notification** : un email est envoyé aux leads migrés pour les inviter à créer leur compte.

2.6.2 T1 → T2

1. **Mise à jour du `.env`** : `GITSKY_TIER=t2`, activation de `MODULE_ADMIN`, `MODULE_I18N`, `MODULE_MONETIZATION_*` selon la stratégie de revenus.
2. **Configuration Stripe** : produits ou abonnements créés dans le dashboard Stripe partagé du studio.
3. **Domaine dédié (optionnel)** : passage de `pain-scrapers.mystudio.com` à un domaine premier (`pain-scrapers.com`), avec ajustement des labels Traefik.
4. **Alembic upgrade** : tables `products`, `purchases`, `subscriptions`, `security_events`, etc.
5. **Contenu i18n** : rédaction des traductions EN si le marché cible n'est pas exclusivement francophone.
6. **Communication** : email d'annonce aux utilisateurs T1 existants, éventuellement avec une offre early adopters.

Chaque étape est **réversible** tant qu'on ne détruit pas de données — un simple retour au `.env` précédent suivi d'un Alembic downgrade suffit à revenir au tier antérieur.

2.7 6. Le Kill Mechanism : Arrêter à Temps

Un projet qui ne remplit pas ses critères de promotion doit être arrêté. Sans ce mécanisme, la flotte se remplit de zombies qui consomment ressources et attention sans jamais rembourser leur coût.

2.7.1 Règles de Kill Automatique

Un cron quotidien (`kill_check`) évalue chaque projet et marque les candidats au shutdown :

Tier	Condition de kill	Coût cumulé maximal
To	Aucun des signaux du §4 atteint à J+21	100 € en trafic + infra
T1	Rétention D7 < 15 % à J+30 OU aucun signal WTP à J+45	500 € en cumul
T2	Churn mensuel > 20 % sur 3 mois consécutifs OU MRR < 100 € à J+90	évaluation manuelle

Les seuils T0 et T1 déclenchent un shutdown automatique. Le seuil T2 déclenche une **alerte au fleet dashboard** — la décision reste humaine à ce niveau, car un produit T2 peut avoir des raisons contextuelles de sous-performer (saisonnalité, changement d'algorithme SEO, panne prolongée).

2.7.2 Procédure de Shutdown

Le shutdown n'est jamais brutal :

1. **J-2 avant kill** : email au propriétaire du projet + entrée `pending_kill` dans le fleet dashboard.
2. **J-0 kill** : `docker compose down`, retrait des labels Traefik, désactivation des enregistrements DNS pointant vers le sous-domaine.
3. **Sauvegarde froide** : dump PostgreSQL compressé archivé sur S3 ou Backblaze pendant 90 jours (permet une réactivation si signal tardif).
4. **Libération des ressources** : le sous-domaine retourne dans le pool disponible, la DB est marquée `archived`.
5. **Journalisation** : le projet est marqué `killed` dans le fleet dashboard avec date, tier atteint, raison, coût total.

2.7.3 Ce que le Kill n'Est Pas

Un kill n'est pas un échec du projet — c'est le **succès du protocole**. Sur 30 idées testées, on s'attend statistiquement à en tuer 25. Les 5 restantes financent les 25 par leur montée en T1/T2. Refuser de tuer, c'est refuser de valider le portefeuille de projets.

2.8 7. Anti-Patterns à Éviter

Trois erreurs récurrentes que le système de tiers rend particulièrement coûteuses :

Démarrer en T2 « parce que je connais mon marché » Les cimetières de startups sont pleins d'idées « évidentes ». Le T0 coûte quelques euros — la conviction subjective ne coûte rien de moins qu'à celui qui l'a réellement testée.

Monter en T2 sur un signal T1 flou Si les seuils T1 → T2 ne sont pas franchement atteints, ajouter de l'admin, de l'i18n et de la monétisation ne créera pas la demande manquante — cela consommera du temps qui aurait servi à tester d'autres idées.

Refuser de tuer un projet attaché Un projet dans lequel on a investi du temps devient émotionnellement coûteux à arrêter. Le kill mechanism automatique protège l'opérateur de sa propre inertie.

2.9 Checklist du Chapitre

- ☐ Je démarre systématiquement chaque nouveau projet en T0
- ☐ J'ai des critères numériques explicites pour la promotion T0 → T1 → T2

- Je sais quels modules s'activent à chaque tier et pourquoi
 - Le mécanisme de kill est armé et testé sur mon environnement
 - Je préserve les données d'un projet tué pendant 90 jours avant destruction définitive
 - Je considère un kill comme un succès du protocole, pas comme un échec
-

Ce système de tiers structure tout le reste de l'ouvrage : la Partie II décrit le core présent à tous les tiers, la Partie III les modules optionnels que chaque tier active, et la Partie IV le générateur et la flotte qui rendent l'ensemble opérationnel à grande échelle. Dans le prochain chapitre, nous détaillons l'initialisation du backend FastAPI, socle commun aux trois tiers.

3 Initialisation du Backend avec FastAPI

3.1 Pourquoi FastAPI ?

Pour le template **GitSky**, nous avons besoin d'un backend capable de gérer des tâches asynchrones, une validation de données stricte, une documentation automatique, et surtout **d'activer ou désactiver des modules entiers de fonctionnalités selon le tier du projet**. FastAPI s'est imposé comme le choix naturel grâce à :

1. **Performance** : Basé sur Starlette et Pydantic, il est l'un des frameworks Python les plus rapides.
2. **Asynchronisme natif** : Crucial pour les appels à l'IA ou les traitements de données lourds sans bloquer l'API.
3. **Type Safety** : Utilisation intensive des hints Python pour réduire les bugs et améliorer l'auto-complétion.
4. **Modularité par inclusion de routers** : Permet le chargement dynamique de modules selon la configuration, sans surcoût mémoire pour les modules inactifs.

3.2 Structure d'un Projet Professionnel

L'architecture GitSky repose sur trois couches distinctes : `core`, `modules`, `domain`. Cette séparation est ce qui permet à un même code base de porter les trois tiers présentés au chapitre précédent.

```

backend/
├── app/
│   ├── core/
│   │   ├── auth/
│   │   ├── admin_shell/
│   │   ├── database.py
│   │   ├── config.py
│   │   ├── main.py
│   │   ├── models.py
│   │   └── seo.py
│   │   # Toujours présent – socle de tous les tiers
│   │   # JWT + gestion des rôles utilisateur
│   │   # Coquille admin extensible
│   │   # Session et moteur SQLAlchemy
│   │   # Paramètres Pydantic Settings + flags module
│   │   # Initialisation FastAPI + chargement des modules
│   │   # Modèles core (User, Role)
│   │   # Composants SEO de base
│   ├── modules/
│   │   ├── analytics/
│   │   ├── onboarding/
│   │   ├── tutorials/
│   │   ├── security/
│   │   ├── i18n/
│   │   ├── agentic/
│   │   └── monetization/
│   │   # Modules optionnels activables par MODULE_*
│   │   # GeoIP tracking + world map
│   │   # Profilage dynamique
│   │   # Système de contenu pédagogique
│   │   # SecurityMiddleware
│   │   # Chargement des locales serveur
│   │   # Framework multi-agents IA
│   │   # Stripe boutique + abonnements
│   ├── domain/
│   │   ├── models.py
│   │   ├── routers.py
│   │   └── schemas.py
│   │   # Métier spécifique au projet
│   │   # Tables métier
│   │   # Endpoints métier
│   │   # Validation Pydantic métier
│   ├── shared/
│   │   └── clients/
│   │   # Utilitaires transverses
│   │   # Clients HTTP, LLM proxy, SMTP...
│   ├── alembic/
│   │   ├── core/
│   │   └── modules/
│   │   # Migrations
│   │   # Chaîne de migrations du core
│   │   # Chaîne par module (activée conditionnellement)
│   ├── uploads/
│   ├── requirements.txt
│   └── alembic.ini
    # Stockage des médias
    # Dépendances Python
    # Configuration des migrations

```

3.2.1 Trois Couches, Trois Responsabilités

- **core/** ne dépend que de la stack (FastAPI, SQLAlchemy, Pydantic). Il ne connaît ni les modules ni le domaine — mais il expose les hooks nécessaires pour les charger.
- **modules/** dépend uniquement du core. Chaque module s’active ou se désactive via un flag `MODULE_*` sans que ni le core ni les autres modules n’aient besoin d’être modifiés.
- **domain/** dépend du core et éventuellement de certains modules. C’est la couche unique par projet, produite par le générateur `create-gitsky-project` (voir Chap 17).

3.3 Le Système de Modules

Un module est un dossier isolé qui contient tout ce qu’il apporte : modèles SQLAlchemy, routeurs FastAPI, schémas Pydantic, migrations Alembic. À l’initialisation de l’API, `main.py` charge dynamiquement les modules activés :

```
# app/core/main.py – extrait
from fastapi import FastAPI
from app.core.config import get_settings
from app.core import auth, admin_shell

settings = get_settings()
app = FastAPI(title=f"GitSky – {settings.project_name}")

# Chargement systématique du core
app.include_router(auth.router, prefix="/api/auth")
app.include_router(admin_shell.router, prefix="/api/admin")

# Chargement conditionnel des modules
if settings.module_analytics:
    from app.modules.analytics import router as analytics_router
    app.include_router(analytics_router, prefix="/api/analytics")

if settings.module_onboarding:
    from app.modules.onboarding import router as onboarding_router
    app.include_router(onboarding_router, prefix="/api/onboarding")

if settings.module_agentic:
    from app.modules.agentic import router as agentic_router
    app.include_router(agentic_router, prefix="/api/agent-services")

# ... idem pour les autres modules
```

Trois propriétés découlent de ce système :

1. **Aucun surcoût mémoire pour un module inactif** — son code n’est jamais importé.
2. **Aucune migration Alembic inutile** — la chaîne de migrations d’un module désactivé n’est simplement pas incluse dans l’environnement Alembic.
3. **Extensibilité sans modification du core** — ajouter un nouveau module se fait en respectant un contrat d’interface simple (`router` , `models` , `migrations/`).

3.4 Configuration Centralisée avec Pydantic Settings

Nous utilisons `pydantic-settings` pour charger et valider nos variables d’environnement. Le `Settings` centralise à la fois les paramètres de projet et les flags de modules :

```
# app/core/config.py
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    project_name: str
    secret_key: str
    frontend_url: str = "http://localhost:5173"
    environment: str = "development"
    database_url: str

    # Tier – profil par défaut des flags module (t0 | t1 | t2)
    gitsky_tier: str = "t2"

    # Modules – chacun surcharge le profil de tier si présent dans .env
    module_auth: bool = True
    module_admin: bool = True
    module_analytics: bool = False
    module_onboarding: bool = False
    module_tutorials: bool = False
    module_security_middleware: bool = True
    module_i18n: bool = False
    module_agentic: bool = False
    module_monetization_shop: bool = False
    module_monetization_subscription: bool = False

    class Config:
        env_file = ".env"

@lru_cache()
def get_settings() -> Settings:
    return Settings()
```

Cela garantit qu’une variable manquante (`SECRET_KEY` , `DATABASE_URL` , `PROJECT_NAME`) empêche l’application de démarrer, plutôt que de la laisser tourner dans un état incohérent.

3.5 Connexion à la Base de Données (SQLAlchemy)

Chaque projet GitSky possède sa **propre base de données PostgreSQL** — cette isolation est ce qui permet à un projet de vivre, migrer de tier, ou être arrêté sans impacter les autres. La connexion utilise SQLAlchemy en mode **asynchrone** (moteur `asyncpg` en production) et fournit une session fraîche à chaque requête via le pattern « Dependency Injection ».

```
# app/core/database.py
from collections.abc import AsyncGenerator
from sqlalchemy.ext.asyncio import (
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)
from sqlalchemy.orm import declarative_base
from app.core.config import get_settings

settings = get_settings()

engine = create_async_engine(settings.database_url, future=True)
SessionLocal = async_sessionmaker(
    engine, class_=AsyncSession, expire_on_commit=False, autoflush=False
)
Base = declarative_base()

async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with SessionLocal() as session:
        yield session
```

L'URL de connexion prend la forme `postgresql+asyncpg://user:pass@db:5432/dbname` en production. Les endpoints qui consomment cette session sont eux-mêmes `async` et utilisent `await` pour chaque opération de base (`await db.execute(...)` , `await db.commit()`).

3.6 Le Point d'Entrée : `main.py`

Le fichier `main.py` orchestre l'application, applique les middlewares (CORS, éventuellement `SecurityMiddleware`), et inclut les routeurs core puis les modules activés :

```
# app/core/main.py – version complète
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import get_settings
from app.core import auth, admin_shell

settings = get_settings()
app = FastAPI(title=f"GitSky – {settings.project_name}")

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=[settings.frontend_url],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# SecurityMiddleware conditionnel (module)
if settings.module_security_middleware:
    from app.modules.security import SecurityMiddleware
    app.add_middleware(SecurityMiddleware)

# Routers core (toujours actifs)
app.include_router(auth.router, prefix="/api/auth")
app.include_router(admin_shell.router, prefix="/api/admin")

# Routers modules (chargés selon les flags)
if settings.module_analytics:
    from app.modules.analytics import router as analytics_router
    app.include_router(analytics_router, prefix="/api/analytics")

# ... idem pour les autres modules

@app.get("/health")
async def health():
    return {"status": "ok", "project": settings.project_name, "tier": settings.gitisky_tier}
```

3.7 Lancer le Backend

Grâce à Docker Compose, le lancement est automatique. Le point d'entrée Uvicorn reste identique quel que soit le tier :

```
uvicorn app.core.main:app --host 0.0.0.0 --port 8000 --reload
```

Le flag `--reload` est utilisé en développement uniquement. En production, Gunicorn orchestre plusieurs workers Uvicorn (voir Chap 21).

Le squelette du backend est en place. Dans le prochain chapitre, nous détaillons la modélisation des données — en distinguant explicitement les modèles du core (toujours présents) et les modèles apportés par chaque module optionnel.

4 Modélisation des Données avec SQLAlchemy

4.1 Introduction à la Modélisation

Le schéma de données de GitSky est réparti entre le **core** (présent à tous les tiers) et les **modules** (chacun apportant ses propres tables). Cette organisation reflète directement l'architecture décrite au chapitre précédent : chaque table appartient à une couche identifiée, et un module désactivé n'introduit ni ses modèles ni ses migrations dans la base.

Nous utilisons **SQLAlchemy** comme ORM, ce qui nous permet de :

1. Bénéficier d'une validation de type forte via Python.
2. Manipuler des objets Python plutôt que d'écrire du SQL brut.
3. Garantir l'intégrité référentielle entre entités.

4.2 Les Modèles du Core

Deux entités sont toujours présentes lorsque `MODULE_AUTH=true` — soit dès le tier T1.

4.2.1 User — L'Identité et les Rôles

Le modèle `User` centralise l'identité et les permissions. Une énumération `UserRole` hiérarchise les accès :

```
# app/core/models.py
class UserRole(str, enum.Enum):
    anonymous = "anonymous"
    waitlist = "waitlist"
    user = "user"
    premium = "premium"
    admin = "admin"

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, nullable=False, index=True)
    hashed_password = Column(String, nullable=False)
    role = Column(Enum(UserRole), default=UserRole.user)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

En To (Landing pur), même la table `users` n'est pas nécessaire — `MODULE_AUTH=false` désactive à la fois le routeur d'authentification et la migration qui crée la table. Les emails sont alors collectés par le landing-collector partagé (voir Chap 18).

4.3 Les Modèles des Modules

Chaque module apporte ses propres modèles, activés uniquement si le flag `MODULE_*` correspondant est à `true`.

4.3.1 Module `tutorials` — Contenu Pédagogique

Ce module implémente une université virtuelle. Il repose sur une relation **One-to-Many** entre `Tutorial` et `Lesson` :

```
# app/modules/tutorials/models.py
class Tutorial(Base):
    __tablename__ = "tutorials"
    id = Column(Integer, primary_key=True)
    title = Column(String, nullable=False)
    slug = Column(String, unique=True, index=True)
    lang = Column(String(5), default="fr", index=True)
    access_role = Column(Enum(UserRole), default=UserRole.user)

    lessons = relationship("Lesson", back_populates="tutorial", order_by="Lesson.order")

class Lesson(Base):
    __tablename__ = "lessons"
    id = Column(Integer, primary_key=True)
    tutorial_id = Column(Integer, ForeignKey("tutorials.id", ondelete="CASCADE"))
    title = Column(String, nullable=False)
    content = Column(Text) # Markdown
    order = Column(Integer, default=0)
```

Activation : `MODULE_TUTORIALS=true`.

4.3.2 Module `onboarding` — Profilage Dynamique

Le modèle `UserProfile` stocke les résultats d'un questionnaire d'onboarding, en relation 1:1 avec `User` :

```
# app/modules/onboarding/models.py
class UserProfile(Base):
    __tablename__ = "user_profiles"
    user_id = Column(Integer, ForeignKey("users.id"), unique=True)
    flow_id = Column(String, nullable=False)
    answers = Column(Text) # JSON sérialisé
    profile = Column(String) # Label calculé (ex: 'power_user')
    score = Column(Integer)
```

Activation : `MODULE_ONBOARDING=true`.

4.3.3 Module `analytics` — Tracking Anonymisé RGPD

Le modèle `Visit` enregistre l'activité sans stocker l'IP en clair :


```
# app/modules/analytics/models.py
class Visit(Base):
    __tablename__ = "visits"
    id = Column(Integer, primary_key=True)
    ip_hash = Column(String, index=True)
    country_code = Column(String(2), index=True)
    city = Column(String)
    path = Column(String)
    user_role = Column(String)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

Activation : `MODULE_ANALYTICS=true` . En To, la collecte se fait plutôt via le landing-collector partagé pour mutualiser l'infrastructure GeoIP (Chap 18).

4.3.4 Module `security` — Journal des Événements de Sécurité

Le modèle `SecurityEvent` enregistre chaque tentative d'intrusion détectée par le `SecurityMiddleware` :

```
# app/modules/security/models.py
class SecurityEvent(Base):
    __tablename__ = "security_events"
    id = Column(Integer, primary_key=True)
    event_type = Column(String, index=True) # path_scan, injection_attempt...
    severity = Column(String, index=True) # low, medium, high, critical
    ip_address = Column(String, index=True)
    path = Column(String)
    user_agent = Column(String)
    details = Column(JSON)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

Activation : `MODULE_SECURITY_MIDDLEWARE=true` (activé par défaut à T1 et T2).

4.3.5 Module `monetization` — Boutique et Abonnements

Ce module apporte trois tables, activables via deux flags indépendants (`MODULE_MONETIZATION_SHOP` , `MODULE_MONETIZATION_SUBSCRIPTION`).

4.3.5.1 Produits et Achats

```
# app/modules/monetization/models.py – extrait shop
class Product(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    slug = Column(String, unique=True, index=True)
    price_cents = Column(Integer, nullable=False) # 2900 = 29 €
    stripe_price_id = Column(String)
    file_path = Column(String)
    is_active = Column(Boolean, default=True)

class Purchase(Base):
    __tablename__ = "purchases"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id")) # nullable = achat invité
    product_id = Column(Integer, ForeignKey("products.id"))
    email = Column(String, nullable=False)
    stripe_session_id = Column(String, unique=True)
    download_token = Column(String, unique=True)
    download_count = Column(Integer, default=0)
    max_downloads = Column(Integer, default=5)
    token_expires_at = Column(DateTime(timezone=True))
    fulfilled_at = Column(DateTime(timezone=True))
```

4.3.5.2 Abonnements

```
# app/modules/monetization/models.py – extrait subscription
class Subscription(Base):
    __tablename__ = "subscriptions"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"), unique=True)
    stripe_subscription_id = Column(String, unique=True)
    stripe_customer_id = Column(String)
    status = Column(Enum(SubscriptionStatus))
    current_period_end = Column(DateTime(timezone=True))
    trial_end = Column(DateTime(timezone=True))
    cancelled_at = Column(DateTime(timezone=True))
```

Règle métier : si `status ∈ {active, trialing}`, le rôle utilisateur devient `premium`. Dès un changement d'état (échec de paiement, annulation), le rôle est rétrogradé vers `user`. Cette logique est pilotée intégralement par les webhooks Stripe, sans intervention humaine.

4.4 Table Récapitulative des Modèles par Couche

Modèle	Couche	Flag d'activation	To	T1	T2
User , UserRole	core	MODULE_AUTH	✗	✓	✓
Tutorial , Lesson	module tutorials	MODULE_TUTORIALS	✗	✗	selon projet
UserProfile	module onboarding	MODULE_ONBOARDING	✗	✗	✓
Visit	module analytics	MODULE_ANALYTICS	via collector partagé	✓	✓
SecurityEvent	module security	MODULE_SECURITY_MIDDLEWARE	via proxy Traefik	✓	✓
Product , Purchase	module monetization	MODULE_MONETIZATION_SHOP	✗	✗	selon projet
Subscription	module monetization	MODULE_MONETIZATION_SUBSCRIPTION	✗	optionnel	✓

4.5 Gestion des Migrations avec Alembic

Le schéma évolue par ajouts versionnés. Chaque couche a **sa propre chaîne de migrations** :

```
backend/alembic/
├── core/                # Migrations du core (User, Role)
│   ├── env.py
│   └── versions/
├── modules/
│   ├── tutorials/      # Migrations du module tutorials
│   ├── onboarding/
│   ├── analytics/
│   ├── security/
│   └── monetization/
└── alembic.ini         # Configuration multi-chaînes
```

Au démarrage du service `migrate` (voir Chap 1), un runner Python parcourt les flags `MODULE_*` activés et applique la chaîne de chaque module. Point crucial : **chaque chaîne possède sa propre table de version** (`alembic_version_core` , `alembic_version_analytics` , ...). C'est ce qui leur permet de cohabiter dans la base unique du projet sans écraser mutuellement leur état — une seule table `alembic_version` partagée rendrait le multi-chaînes impossible.

```
# scripts/migrate.py – extrait
# _config(section, dossier, version_table, url) construit un alembic.config.Config.
from alembic import command
from app.core.config import get_settings

def run_migrations() -> None:
    settings = get_settings()
    url = settings.database_url

    # Core : toujours appliqué.
    command.upgrade(_config("alembic", "alembic/core", "alembic_version_core", url), "head")

    # Modules : une chaîne (et une table de version) par flag MODULE_* activé.
    if settings.module_tutorials:
        command.upgrade(_config("tutorials", "alembic/modules/tutorials",
                                "alembic_version_tutorials", url), "head")
    if settings.module_onboarding:
        command.upgrade(_config("onboarding", "alembic/modules/onboarding",
                                "alembic_version_onboarding", url), "head")
    # ... idem pour les autres modules
```

Un runner Python (plutôt qu'un script shell) reste portable entre environnements et s'appelle directement dans les tests. La CLI Alembic classique (`alembic --name onboarding upgrade head`) demeure disponible grâce aux sections déclarées dans `alembic.ini`.

Cette approche implique un coût de complexité initial, mais permet trois propriétés critiques :

1. **Une base de données minimale à chaque tier** — pas de tables inutiles.
2. **Migrer de tier = ajouter des chaînes de migrations** sans risque sur celles déjà appliquées.
3. **Désactiver un module** n'a pas d'impact sur les autres, ni sur le core.

4.5.1 Workflow Alembic pour un Nouveau Module

Lors de la création d'un nouveau module :

1. **Créer le dossier** : `alembic/modules/mon_module/`.
2. **Générer une migration** : `alembic --name mon_module revision --autogenerate -m "initial"`.
3. **Vérifier le script généré** avant application.
4. **Ajouter le flag** correspondant à `Settings`, une section dans `alembic.ini`, et l'appliquer dans le runner `scripts/migrate.py`.

Le schéma étant posé et découpé par couche, nous allons maintenant voir comment exposer ces données via une API performante dans le chapitre suivant, en respectant la même séparation core/modules.

5 L'API Core et les Contrats de Modules

5.1 Introduction

L'API du template GitSky suit le même découpage en couches que le backend : le **core** définit les patterns et les contrats communs, tandis que chaque **module** apporte ses propres endpoints. Ce chapitre couvre les patterns partagés — validation Pydantic, chargement dynamique des routers, contrats d'interface — ainsi qu'une vue d'ensemble du framework agentic IA (dont le détail est en Chap 15).

5.2 Validation des Données avec Pydantic

FastAPI utilise **Pydantic** pour valider les données entrantes et sortantes. Chaque routeur — core ou module — définit ses schémas dans un fichier `schemas.py` :

```
# app/modules/onboarding/schemas.py - exemple
from pydantic import BaseModel

class OnboardingAnswer(BaseModel):
    flow_id: str
    answers: dict[str, str]

class OnboardingResult(BaseModel):
    title: str
    description: str
    score: int
    profile: str
    label: str
```

Grâce à ces schémas, FastAPI génère automatiquement la documentation Swagger et rejette toute requête mal formée avant même qu'elle n'atteigne la logique métier.

5.3 Contrat d'Interface d'un Module

Un module qui apporte des endpoints à l'API doit exposer une interface minimale que le core sait détecter :

```
# app/modules/mon_module/__init__.py
from fastapi import APIRouter

router = APIRouter()

@router.get("/status")
async def status():
    return {"module": "mon_module", "ok": True}
```

Le core, dans `main.py`, importe conditionnellement ce `router` selon le flag `MODULE_MON_MODULE` (voir Chap 3). Aucun couplage direct n'existe entre le core et le module au-delà de ce contrat.

5.4 Chargement Dynamique des Routers

Le pattern d'inclusion conditionnelle est central :

```
# app/core/main.py – pattern répété pour chaque module
if settings.module_analytics:
    from app.modules.analytics import router as analytics_router
    app.include_router(analytics_router, prefix="/api/analytics")
```

Trois règles à respecter :

1. **Import à l'intérieur du `if`** — sinon le module est chargé même s'il est désactivé.
2. **Préfixe cohérent** — toujours `/api/<nom-module>` pour la lisibilité.
3. **Un seul router par module** — les sous-routers internes du module sont agrégés dans son `__init__.py`.

5.5 Services Transverses : Clients Partagés

Certains services ne sont pas des routers exposés par un module — ce sont des **clients partagés** utilisés par plusieurs modules ou par la couche domaine :

Client	Rôle	Utilisé par
<code>landing_collector_client</code>	Poste un formulaire de landing dans la base centrale de la flotte	Tier To sans DB propre
<code>llm_proxy_client</code>	Appelle un LLM via le proxy partagé (quota et logs par projet)	Framework agentic, tout module IA
<code>geoip_client</code>	Résout une IP en pays/ville depuis le service GeoIP mutualisé	Module analytics, module security
<code>smtp_client</code>	Envoie un email transactionnel via le relais SMTP partagé	Onboarding, monetization, notifications kill

Ces clients vivent dans `app/shared/clients/` et sont partagés à travers le core. Leur implémentation et leur configuration côté flotte sont détaillées au Chap 18 (Services partagés).

5.6 Aperçu du Framework Agentic IA

Le module `agentic` transforme un projet GitSky en écosystème d'automatisation multi-agents. Son activation reste optionnelle (`MODULE_AGENTIC=true`) car il apporte une empreinte non négligeable — modèles, orchestrator, tool registry, memory system, guardrails.

Endpoints principaux exposés par le module :

```
GET  /api/agent-services/services
POST /api/agent-services/services/{service_slug}/execute
GET  /api/agent-services/executions/{execution_id}
```

Le détail de l'architecture agentic est présenté au Chap 15, et celui du LLM proxy partagé (mutualisation des clés API, quotas par projet, logs) au Chap 18.

Le socle API est en place, avec ses patterns communs et son contrat d'interface pour les modules. Dans la partie suivante, nous construisons l'interface utilisateur pour interagir avec ce backend modulaire.

6 Le Frontend Moderne avec React et Vite

6.1 Pourquoi cette Stack ?

Pour l'interface utilisateur de **GitSky**, nous avons privilégié la réactivité, la vitesse de développement et une esthétique "Dark Mode" soignée. Le choix s'est porté sur :

1. **React 19** : La dernière version du framework de Meta, offrant une gestion d'état simplifiée et des performances accrues.
2. **Vite** : Le remplaçant moderne de Webpack. Il offre un démarrage instantané et un rechargement à chaud ultra-rapide pendant le développement.
3. **Tailwind CSS 4** : Une bibliothèque utilitaire de CSS qui permet de construire des interfaces complexes sans quitter le code HTML/JSX.
4. **Radix UI & shadcn/ui** : Une base de composants accessibles et stylisables qui nous évite de réinventer la roue pour les boutons, les boîtes de dialogue ou les menus.

6.2 Structure du Projet Frontend

Nous suivons une architecture claire pour séparer les responsabilités :

```
frontend/
├── src/
│   ├── assets/           # Images, icônes, logos
│   ├── components/      # Composants réutilisables (Navbar, WaitlistForm)
│   │   ├── ui/          # Composants de base (Button, Input, Card)
│   │   └── analytics/   # Visualisation de données (WorldMap, Timeline)
│   ├── context/         # Gestion globale de l'état (AuthContext)
│   ├── lib/             # Utilitaires (cn, apiFetch)
│   ├── pages/           # Pages principales de l'application
│   │   └── admin/       # Interface d'administration
│   ├── App.tsx          # Routage principal (React Router)
│   ├── main.tsx         # Point d'entrée de l'application
│   └── i18n.ts          # Configuration multi-langue
├── public/              # Fichiers statiques et locales JSON
└── vite.config.ts       # Configuration de Vite
```

6.3 Tailwind CSS 4 et le Design System

Tailwind 4 apporte une simplification majeure dans la configuration. Le template GitSky l'utilise pour imposer une identité visuelle forte mais **paramétrable par projet** — le générateur `create-gitsky-project` (Chap 17) injecte la couleur primaire, la police et le logo depuis le `config.yaml` du projet :

- **Dark Mode natif** : Utilisation systématique des couleurs `zinc` et `black`, avec un accent primaire configurable via la variable CSS `--color-primary`.
- **Typographie** : `Geist` par défaut, surchargeable via `branding.font_family` dans le config.

- **Transitions** : Utilisation de `tw-animate-css` pour des micro-interactions fluides.

Exemple d'un bouton primaire dont la couleur est paramétrée au niveau du projet :

```
<button className="bg-primary text-primary-foreground font-semibold px-6 py-2.5
                  rounded-lg hover:bg-primary/90 transition shadow-lg">
  Rejoindre la plateforme
</button>
```

Grâce à cette paramétrisation, chaque instance GitSky affiche sa propre identité visuelle sans modification du code des composants.

6.4 Gestion des Alias de Chemin

Pour éviter les imports complexes du type `../../components`, nous utilisons des alias. `@/` pointe directement vers le dossier `src/`.

```
// Au lieu de :
import { Button } from "../../components/ui/button";

// Nous écrivons :
import { Button } from "@components/ui/button";
```

6.5 L'importance de Vite en Développement

En local, Vite ne compile pas l'intégralité de l'application au démarrage. Il sert le code source via des modules ES natifs du navigateur. Résultat : peu importe la taille du projet, l'application est prête en moins de 300ms.

L'interface étant configurée, nous allons maintenant voir comment gérer l'authentification et sécuriser les accès à notre plateforme dans le chapitre suivant.

7 Authentification et Sécurité de Session

7.1 Introduction

Le module d'authentification est **activé dès le tier T1** (`MODULE_AUTH=true`). Il fournit à la fois l'API backend (JWT + refresh) et le contexte frontend qui suit l'état de connexion à travers l'application. En To (Landing pur), il n'est pas activé — la collecte des emails passe par le landing-collector partagé de la flotte (voir Chap 18).

7.2 Gestion Globale de l'État : AuthContext

Côté React, nous utilisons la **Context API** pour centraliser l'identité de l'utilisateur, son rôle et ses jetons :

```
// src/context/AuthContext.tsx
export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [accessToken, setAccessToken] = useState<string | null>(
    localStorage.getItem("access_token")
  );

  // Vérification de la session au chargement
  useEffect(() => {
    const token = localStorage.getItem("access_token");
    if (token) fetchProfile(token);
  }, []);

  // ... login, logout, refresh
}
```

7.3 Stratégie de Sécurité : JWT et Cookies

GitSky utilise une stratégie hybride pour sécuriser les sessions :

1. **Access Token** : un JWT de courte durée (15 min) stocké en `localStorage` pour les appels API.
2. **Refresh Token** : un jeton stocké dans un cookie **HttpOnly** côté backend, ce qui permet de renouveler la session sans exposer le jeton aux scripts (protection XSS).

Le refresh est déclenché automatiquement par un intercepteur `apiFetch` lorsque l'API renvoie un `401` — l'utilisateur ne perçoit rien.

7.4 Fin de Session et Révocation

Un JWT est *stateless* : une fois émis, le serveur lui fait confiance jusqu'à expiration. Trois mécanismes complètent donc la stratégie hybride :

1. **POST /api/auth/logout** — expire le cookie refresh HttpOnly côté serveur. Vider le `localStorage` ne suffit pas : sans cet appel, le refresh resterait valable 7 jours sur la machine. `AuthContext.logout()` l'appelle systématiquement. L'endpoint est volontairement sans auth : il doit fonctionner même avec un access token expiré.
2. **POST /api/auth/logout-all** — révoque *tous* les refresh émis pour le compte (« déconnexion partout »). Chaque `User` porte un compteur `token_version`, embarqué dans le refresh (claim `tv`) ; l'endpoint incrémente le compteur, et `/refresh` refuse tout jeton dont le `tv` est périmé — même signé, même non expiré. C'est le seul levier de révocation d'un JWT stateless (cookie exfiltré, machine compromise).
3. **Politique de mot de passe** — 8 caractères minimum, appliquée **au register uniquement** (`RegisterRequest`). Le login reste non contraint : un compte créé avant la règle doit toujours pouvoir se connecter.

❌ `logout()` frontend qui ne vide que le `localStorage` — le cookie survit. ✅ `logout()` qui appelle l'endpoint, et `logout-all` en cas de doute sur un poste.

Le **rate limiting du login** (5 req/min) n'est pas dans l'application : conformément à la doctrine du Chap 14, il est porté par un routeur Traefik dédié généré dans le `docker-compose.yml` de production (voir Chap 21) — chaque essai coûtant un hachage argon2, un login illimité serait à la fois du credential stuffing et un DoS CPU à bas coût.

7.5 Protection des Routes (Guards)

Toutes les pages ne sont pas accessibles à tout le monde. Trois composants “Guards” encapsulent la logique d'accès :

- **PrivateRoute** : redirige vers `/login` si l'utilisateur n'est pas connecté.
- **AdminRoute** : vérifie le rôle `admin` — utilisé uniquement quand `MODULE_ADMIN=true`.
- **GuestRoute** : empêche un utilisateur connecté d'accéder aux pages de login/register.

Exemple d'utilisation :

```
<Route path="/admin" element={
  <AdminRoute><AdminDashboard /></AdminRoute>
} />
```

7.6 Rôles et Progression Utilisateur

Le module `auth` gère cinq rôles (voir Chap 4 pour la définition SQLAlchemy) :

Rôle	Origine	Droits
anonymous	Non connecté	Consultation publique uniquement
waitlist	Inscrit via To ou onboarding sans activation	Accès très limité
user	Utilisateur activé	Fonctionnalités standard
premium	Attribué par les webhooks Stripe (Chap 16)	Fonctionnalités payantes
admin	Attribué manuellement en base	Dashboard admin

Les transitions de rôle (`waitlist` → `user` via activation email, `user` → `premium` via abonnement Stripe) sont **automatiques** et pilotées soit par des events internes, soit par des webhooks externes.

7.7 Authentification et Système à Trois Tiers

Le module auth n'est pas actif en To — un projet en tier landing collecte les emails via le landing-collector partagé sans créer de comptes. Lors de la promotion To → T1, les leads sont importés dans la table `users` avec le rôle `waitlist`, et invités à créer leur mot de passe via un email transactionnel.

Cette progression est décrite en détail au Chap 20 (cycle de vie d'un projet).

L'infrastructure d'authentification étant en place, nous verrons dans le prochain chapitre comment le module `i18n` rend l'application accessible à un public international, à partir du tier T2.

8 Internationalisation (Module i18n)

8.1 Introduction

Le module `i18n` de GitSky rend une application multilingue **à partir du tier T2**. Il n'est jamais activé en To ou T1 — la traduction est un investissement dont le retour n'apparaît qu'à un stade avancé du produit, quand le marché domestique est validé.

Activation : `MODULE_I18N=true`.

Sur les projets où l'internationalisation n'est pas pertinente (marché exclusivement francophone, produits techniques anglophones dès le départ), le flag reste à `false` et l'application tourne en une seule langue sans surcoût.

8.2 Architecture i18n

L'implémentation repose sur deux composants coordonnés :

- **Côté frontend** : `react-i18next` charge les traductions à la demande et met à jour l'interface sans rechargement.
- **Côté backend** : un endpoint de type `GET /api/content/tutorials?lang=fr` filtre le contenu selon la langue demandée.

Les traductions sont découpées par **namespace** pour rester maintenables :

```
frontend/public/locales/  
├── fr/  
│   ├── common.json    # Navigation, footer, actions génériques  
│   ├── auth.json      # Login, register, mot de passe oublié  
│   ├── learn.json     # Contenu pédagogique  
│   └── admin.json     # Dashboard admin  
└── en/  
    ├── common.json  
    ├── auth.json  
    ├── learn.json  
    └── admin.json
```

8.3 Configuration

Le fichier `src/i18n.ts` initialise `react-i18next` :

```
// src/i18n.ts
import i18n from "i18next";
import LanguageDetector from "i18next-browser-languagedetector";
import Backend from "i18next-http-backend";
import { initReactI18next } from "react-i18next";

i18n
  .use(Backend)
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    fallbackLng: "fr",
    supportedLngs: ["fr", "en"],
    ns: ["common", "auth", "learn", "admin"],
    backend: {
      loadPath: "/locales/{{lng}}/{{ns}}.json",
    },
  });
```

8.4 Utilisation dans un Composant

```
import { useTranslation } from "react-i18next";

function Navbar() {
  const { t, i18n } = useTranslation("common");
  return (
    <nav>
      <a href="/learn">{t("nav.learn")}</a>
      <a href="/pricing">{t("nav.pricing")}</a>
      <LangSelector current={i18n.language} />
    </nav>
  );
}
```

Le `LangSelector` permet à l'utilisateur de basculer de langue à chaud sans rechargement — le choix est persisté dans le `localStorage`.

8.5 Contenu Multilingue en Base

Pour les modules qui gèrent du contenu (comme `tutorials`), le modèle SQLAlchemy inclut une colonne `lang` (voir Chap 4). Le filtrage se fait côté backend :

```
@router.get("/tutorials")
async def list_tutorials(lang: str = "fr", db: Session = Depends(get_db)):
    return db.query(Tutorial).filter(Tutorial.lang == lang).all()
```

8.6 Bonnes Pratiques

- **Une clé de traduction par intention**, pas par écran. `auth.login.submit` est plus réutilisable que `login_page_submit_button`.
- **Fallback vers le français** en cas de clé manquante en anglais — mieux qu'un `[missing key]`.

- **Extraction automatique** avec `i18next-parser` en pré-commit hook pour éviter les traductions manquantes.
- **Pluralisation gérée par i18next**, jamais en concaténation :

```
{  
  "notifications.count": "{{count}} notification",  
  "notifications.count_plural": "{{count}} notifications"  
}
```

8.7 Impact sur le SEO

Un site multilingue impose des balises `hreflang` et un `sitemap.xml` par langue. Le module SEO de GitSky (Chap 10) gère ces balises automatiquement quand `MODULE_I18N` est activé, en générant une entrée par (URL, langue) dans le sitemap.

Avec l'authentification et l'i18n en place côté core, le prochain chapitre décrit le shell admin — surface d'administration extensible que chaque module optionnel viendra peupler.

9 Dashboard Admin : Shell Extensible

9.1 Introduction

Le dashboard admin de GitSky suit le même principe modulaire que le backend : le **core** fournit un **shell** — l'authentification admin, la mise en page, la navigation, le squelette d'onglets — que **chaque module activé** contribue à peupler par ses propres composants et endpoints.

Le shell est disponible dès `MODULE_ADMIN=true` (soit à partir du tier T2 par défaut). En T0 et T1, il n'existe simplement pas — pas de route `/admin`, pas de rôle `admin` à attribuer.

9.2 Architecture du Shell

Le dashboard est structuré autour d'un `AdminLayout` qui compose une barre latérale et un espace principal. La barre latérale est **peuplée dynamiquement** selon les modules activés :

```
// frontend/src/pages/admin/AdminLayout.tsx - extrait
const tabs = [
  { key: "users",    label: "Utilisateurs", path: "/admin/users",    enabled: true }, // shell
  { key: "waitlist", label: "Waitlist",    path: "/admin/waitlist",  enabled: true }, // shell
  { key: "analytics", label: "Analytics",  path: "/admin/analytics", enabled: modules.analytics },
  { key: "content",  label: "Contenu",      path: "/admin/content",  enabled: modules.tutorials },
  { key: "security", label: "Sécurité",      path: "/admin/security", enabled: modules.security },
  { key: "shop",     label: "Boutique",     path: "/admin/shop",     enabled: modules.monetization
},
];
```

Le flag `modules.<nom>` est fourni au frontend via un endpoint `/api/admin/modules` qui expose la configuration côté backend. Un onglet dont le module n'est pas activé n'apparaît ni dans la barre latérale ni comme route valide.

9.3 Onglets du Shell (Toujours Présents)

Deux onglets sont apportés par le shell lui-même et ne dépendent d'aucun module optionnel.

9.3.1 Utilisateurs

Gestion des comptes, rôles et suspensions. L'interface permet de modifier dynamiquement le rôle d'un utilisateur (par exemple `user` → `premium` en cas d'attribution manuelle), de suspendre un accès en cas de comportement suspect, et de consulter le profil complet.

9.3.2 Waitlist

Pour gérer une croissance progressive, un projet GitSky peut utiliser une liste d'attente. Les administrateurs envoient ou renvoient des invitations d'un clic, générant automatiquement des jetons d'invitation stockés en base et transmis par email transactionnel.

9.4 Onglets Apportés par les Modules

Chaque module optionnel qui expose une surface admin contribue à son propre onglet :

Onglet	Module	Détails
Analytics	<code>analytics</code>	Cartes mondiales, timelines, filtres par rôle (voir Chap 13)
Contenu	<code>tutorials</code>	Gestion des tutoriaux et leçons Markdown (voir Chap 11)
Sécurité	<code>security</code>	Journal des événements du SecurityMiddleware (voir Chap 14)
Boutique	<code>monetization</code>	Produits, ventes, abonnements (voir Chap 16)

Le pattern est le suivant. Chaque module fournit :

1. **Un routeur backend** exposé sous `/api/admin/<module>` (protégé par vérification de rôle `admin` côté backend).
2. **Une page React** rendue à l'URL `/admin/<module>`.
3. **Une entrée de menu** dans la configuration du `AdminLayout`.

9.5 Bibliothèque de Médias

Un composant de médias peut être proposé par le module `tutorials` pour fournir un gestionnaire de fichiers depuis le dashboard : upload d'images, vidéos ou PDF ensuite insérables dans le contenu via des snippets Markdown (`[video:URL]` , `[audio:URL]` — voir Chap 11 pour le rendu). Ce composant n'a pas de flag propre — il est disponible dès que `MODULE_TUTORIALS=true`.

9.6 Sécurité du Dashboard

L'accès au dashboard est doublement protégé :

1. **Côté frontend** : `AdminRoute` (Chap 7) redirige tout utilisateur non-admin vers `/`.
2. **Côté backend** : chaque endpoint `/api/admin/*` vérifie le rôle via une dépendance `require_admin` avant tout traitement.

Ne jamais se fier uniquement à la protection frontend — un attaquant peut appeler l'API directement, la vérification serveur est la seule qui compte.

Le shell admin étant posé, le prochain chapitre décrit le SEO — dernier composant présent à tous les tiers — avant d'aborder les modules optionnels qui viennent enrichir le shell.

10 SEO et Optimisation pour la Visibilité

10.1 Introduction

Le SEO fait partie du **core** de GitSky — les composants qu'il fournit sont présents à tous les tiers. Une landing invisible aux moteurs de recherche perd 80 % de son trafic potentiel, donc l'optimisation est active dès To.

Ce chapitre couvre les patterns SEO à intégrer dans une application Single Page React : balises meta dynamiques, sitemap généré côté backend, données structurées Schema.org, et intégration avec le module `i18n` (Chap 8) lorsqu'il est activé.

10.2 Gestion Dynamique des Meta Tags

Contrairement aux frameworks comme Next.js qui gèrent les meta tags côté serveur, une application React nécessite une bibliothèque tierce pour modifier les balises `<head>`. GitSky utilise `react-helmet-async`.

10.2.1 Le Composant SEO Réutilisable

Le core centralise la gestion des balises dans un composant unique. Il gère :

- Le titre de la page (formaté avec le nom du projet).
- La description pour Google.
- Les balises **Open Graph** pour un partage élégant sur Facebook et LinkedIn.
- Les **Twitter Cards** pour les aperçus sur X.
- La balise `canonical` pour éviter le contenu dupliqué.

```
<SEO
  title="Apprendre l'automatisation"
  description="Découvrez nos tutoriaux HITL"
  url="/learn"
/>
```

10.3 Sitemap et Robots.txt Dynamiques

Pour que les robots d'indexation découvrent tout le contenu, un fichier `sitemap.xml` est indispensable. Comme le contenu peut provenir de modules variables selon le projet (tutoriaux, produits, articles...), ce fichier est **généré dynamiquement par le backend**.

10.3.1 Génération Côté Backend (FastAPI)

L'endpoint `/sitemap.xml` parcourt les tables publiques exposées par les modules activés — par exemple `Tutorial` si `MODULE_TUTORIALS=true`, `Product` si `MODULE_MONETIZATION_SHOP=true` — et génère un flux XML listant les URLs **indexables** (les pages publiques uniquement). Chaque module importe ses modèles **à l'intérieur du `if`** : c'est le même chargement conditionnel que pour les routeurs (Chap 5 §3), et il garantit qu'un module désactivé n'introduit ni ses URLs ni ses dépendances dans le sitemap.

Le critère « indexable » s'appuie sur les champs réels de chaque modèle : un tutorial l'est quand son `access_role` vaut `anonymous` (page publique, cf. Chap 11), un produit quand il est `is_active`. La stack étant asynchrone (Chap 3), la requête passe par `await db.execute(select(...))`.

```
# app/core/seo.py - extrait
@router.get("/sitemap.xml")
async def sitemap(db: AsyncSession = Depends(get_db)) -> Response:
    urls = _static_urls()
    if settings.module_tutorials:
        from app.core.models import UserRole
        from app.modules.tutorials.models import Tutorial
        rows = (await db.execute(
            select(Tutorial).where(Tutorial.access_role == UserRole.anonymous)
        )).scalars().all()
        urls += [SeoUrl(path=f"/learn/{t.slug}") for t in rows]
    if settings.module_monetization_shop:
        from app.modules.monetization.models import Product
        rows = (await db.execute(
            select(Product).where(Product.is_active.is_(True))
        )).scalars().all()
        urls += [SeoUrl(path=f"/shop/{p.slug}") for p in rows]
    return Response(content=_render_sitemap(urls), media_type="application/xml")
```

10.4 Données Structurées (JSON-LD)

Pour apparaître sous forme de “Rich Snippets” dans les résultats de recherche, GitSky intègre des données structurées au format **Schema.org**.

Pour la page d'un tutorial exposée par le module `tutorials`, le type `Course` permet à Google d'afficher le nombre de leçons et la durée directement dans ses résultats. Pour un produit du module `monetization`, le type `Product` avec les prix et la disponibilité s'affiche dans Google Shopping.

10.5 Intégration avec le Module i18n

Quand `MODULE_I18N=true`, le composant SEO génère automatiquement les balises `hreflang` :

```
<link rel="alternate" hreflang="fr" href="https://mon-projet.com/learn" />
<link rel="alternate" hreflang="en" href="https://mon-projet.com/en/learn" />
<link rel="alternate" hreflang="x-default" href="https://mon-projet.com/learn" />
```

Le sitemap est également généré **par langue** — chaque URL apparaît une fois par langue supportée, avec la balise `xhtml:link` correspondante. Cette conformité aux directives Google est indispensable pour ne pas être pénalisé sur le contenu dupliqué entre les versions linguistiques.

10.6 Indexation Sélective (`noindex`)

Toutes les pages ne doivent pas être indexées. Les pages de profil utilisateur, le dashboard admin, ou les pages de succès de paiement contiennent des informations privées ou peu utiles au référencement. La propriété `noindex={true}` du composant `SEO` envoie l'instruction aux moteurs de recherche.

10.7 SEO dès le Tier To

Un projet To (une simple landing) bénéficie de tout le socle SEO ci-dessus. Les métriques à surveiller à ce stade sont différentes :

- **Google Search Console** : la landing est-elle indexée ?
- **Position moyenne** sur les mots-clés cibles (extraits du harvest de la phase idée).
- **Taux de clic** dans les SERPs sur les impressions générées.

Un To mal indexé après 21 jours est un signal négatif fort pour le kill mechanism (voir Chap 2).

Notre socle est désormais complet côté frontend et backend, à tous les tiers. La partie suivante détaille les modules optionnels qui viennent enrichir un projet selon ses besoins.

11 Créer un Module Métier : L'Exemple de l'Université Virtuelle

11.1 Introduction

Ce chapitre illustre la création d'un **module métier** de bout en bout, en prenant pour exemple l'université virtuelle — un catalogue de tutoriaux et leçons Markdown qui a servi de projet applicatif de référence à GitSky.

Le module `tutorials` que nous décrivons ici suit exactement le même contrat que les modules du core décrits aux chapitres précédents : un dossier `app/modules/tutorials/` qui apporte ses modèles, ses routeurs, ses schémas, ses migrations, et une entrée dans le shell admin.

Activation : `MODULE_TUTORIALS=true` (T2 par défaut, ou selon le projet).

11.2 Anatomie du Module

```
app/modules/tutorials/  
├── __init__.py      # Exporte `router` – contrat d'interface  
├── models.py        # Tutorial + Lesson (voir Chap 4)  
├── schemas.py       # Schémas Pydantic  
├── routers.py       # Endpoints publics + admin  
├── content_renderer.py # Moteur de rendu Markdown  
└── migrations/     # Chaîne Alembic dédiée
```

Chaque fichier a une responsabilité précise. Ce découpage est **la convention** pour tous les modules GitSky : le générateur `create-gitsky-project` (Chap 17) scaffold de cette structure lorsqu'un nouveau module est déclaré.

11.3 Le Catalogue Public

Le catalogue est la porte d'entrée pour l'apprenant. Il doit être clair et intelligent — ne proposer que le contenu pertinent au visiteur.

11.3.1 Filtrage par Langue et Rôle

Grâce à l'intégration du module `i18n` (Chap 8) côté frontend et d'un paramètre `lang` côté API, l'utilisateur ne voit que les cours disponibles dans sa langue. Un système de badges indique si un cours est **Gratuit** ou **Premium** :

```
// Learn.tsx – extrait
useEffect(() => {
  const lang = i18n.language.split("-")[0];
  fetch(`${API}/api/content/tutorials?lang=${lang}`)
    .then(data => setTutorials(data));
}, [i18n.language]);
```

11.4 Le Moteur de Rendu Markdown

Pour un contenu technique, le format **Markdown** est idéal — les auteurs écrivent rapidement, avec support natif du code, des tableaux et des liens. Le composant `MarkdownRenderer` du module est basé sur `react-markdown`.

11.4.1 Fonctionnalités Avancées

1. **GFM (GitHub Flavored Markdown)** : support des tableaux, listes de tâches et liens automatiques.
2. **Syntax Highlighting** : coloration syntaxique du code via `rehype-highlight` et le thème GitHub Dark.
3. **Embeds Multimédia** : balises personnalisées `[video:URL]`, `[audio:URL]`.
4. **Intégration YouTube** : détection automatique des liens YouTube transformés en `iframe`.

```
// Exemple de pré-traitement du contenu
const processed = content.replace(
  /\[video:(.*?)\]/g,
  (_, url) => `[
](${url})
```

11.5 Contrôle d'Accès au Contenu

La sécurité est gérée à deux niveaux :

- **Frontend** : les liens vers le contenu Premium sont masqués pour les utilisateurs non autorisés.
- **Backend** : les permissions sont systématiquement vérifiées via une dépendance FastAPI avant de délivrer le contenu d'une leçon.

Ne jamais se fier uniquement au filtrage frontend — un attaquant peut appeler l'API directement.

11.6 L'Onglet Admin du Module

Le module contribue un onglet **Contenu** au dashboard admin (voir Chap 9). Il permet à un administrateur de :

- Créer et modifier des tutoriaux.
- Ajouter des leçons Markdown avec preview en direct.

- Uploader des médias via la bibliothèque intégrée.
- Publier ou dépublier du contenu.

L'onglet est déclaré dans `AdminLayout` avec `enabled: modules.tutorials` — il n'apparaît que si le module est activé.

11.7 Le Contrat d'Interface, Point par Point

Pour créer un module métier propre, quatre engagements sont à respecter :

Engagement	Fichier concerné	Vérifié par
Exposer un unique <code>router</code> FastAPI	<code>__init__.py</code>	Le core lors du chargement
Ne jamais importer d'autres modules	tout le dossier	Convention (revue de code)
Fournir sa propre chaîne de migrations	<code>alembic/modules/<module>/</code>	Runner <code>scripts/migrate.py</code>
Rester désactivable via <code>MODULE_*</code>	<code>config.py</code>	Test : lancer sans le flag doit fonctionner

Un module qui viole ce contrat introduit du couplage et casse la promesse d'isolation qui rend le template industrialisable.

11.8 Reproduire ce Pattern pour son Propre Métier

Pour ajouter un module métier à un projet GitSky :

1. Créer `app/modules/mon_module/` avec la structure décrite plus haut.
2. Déclarer les modèles SQLAlchemy dans `models.py`.
3. Générer la migration initiale : `alembic --name mon_module revision --autogenerate -m "initial"`.
4. Ajouter le flag `module_mon_module: bool = False` dans `Settings`.
5. Ajouter le chargement conditionnel dans `main.py`.
6. Écrire les schémas Pydantic dans `schemas.py`.
7. Implémenter les endpoints dans `routers.py`.
8. (Optionnel) Ajouter un onglet dans `AdminLayout` avec `enabled: modules.mon_module`.

Le générateur `create-gitsky-project` (Chap 17) automatise entièrement ces étapes lorsqu'un nouveau module est déclaré dans le `config.yaml` du projet.

Après cet exemple concret, le chapitre suivant présente un autre module fourni en standard : le moteur d'onboarding dynamique — utile pour tout projet qui souhaite profiler ses utilisateurs à l'inscription.

12 Module Onboarding : Profilage Dynamique

12.1 Introduction

Le module `onboarding` est un moteur d'évaluation par flux JSON qui profile un utilisateur à son inscription (ou lors d'une mise à jour de profil), et lui attribue un label technique et un score exploitables par le reste de l'application.

Activation : `MODULE_ONBOARDING=true` (par défaut en T2, mais peut être activé dès T1 pour les projets où le profilage est un enjeu de conversion).

Contrairement à une inscription classique, l'onboarding transforme un formulaire administratif en **question sur le besoin de l'utilisateur** — ce qui améliore à la fois l'engagement initial et la qualité de la segmentation ensuite.

12.2 Le Flow JSON : Règles Métier sans Code

Le cœur du module est un **moteur de scoring** qui évalue des flows au format JSON. Cette approche permet de modifier les règles sans toucher au code Python — un chargé de produit peut ajuster le questionnaire directement dans le fichier.

```
// backend/app/modules/onboarding/flows/user_profiling.json = extrait
{
  "questions": [
    { "id": "role",      "label": "Quel est votre rôle ?",      "options": ["dev", "pm", "designer", "other"] },
    { "id": "team_size", "label": "Taille de votre équipe ?", "options": ["solo", "small", "medium", "large"] },
    { "id": "goal",      "label": "Objectif principal ?",      "options": ["speed", "quality", "learning"] }
  ],
  "scoring": {
    "rules": [
      { "conditions": { "role": "dev", "team_size": "solo" }, "result": { "profile": "solo_builder", "score": 80 } },
      { "conditions": { "role": "pm", "goal": "quality" }, "result": { "profile": "quality_pm", "score": 70 } }
    ],
    "default": { "profile": "explorer", "score": 30 }
  }
}
```

Le moteur évalue les règles dans l'ordre, retourne la première correspondance, ou le `default` sinon :

```
# app/modules/onboarding/engine.py
def evaluate_scoring(flow: dict, answers: dict[str, str]) -> dict:
    for rule in flow["scoring"]["rules"]:
        if all(answers.get(k) == v for k, v in rule["conditions"].items()):
            return rule["result"]
    return flow["scoring"]["default"]
```

12.3 L'Expérience Utilisateur (Frontend)

Le composant `Onboarding.tsx` du module gère un automate à trois phases :

1. **Phase Questions** : affichage séquentiel des questions chargées dynamiquement depuis l'API.
2. **Phase Résultat** : affichage du score et du profil calculé (ex : *Solo Builder*).
3. **Phase Inscription** : formulaire de création de compte pré-rempli avec le contexte du profil.

Un indicateur de progression encourage l'utilisateur à terminer. Chaque réponse est stockée localement avant d'être envoyée pour évaluation.

12.4 Modes d'Utilisation

Le composant est conçu pour deux cas d'usage majeurs :

- **Mode Inscription** : nouveau visiteur arrivant sur la plateforme.
- **Mode Mise à jour** : utilisateur connecté qui redéfinit son profil.

Le mode est déterminé automatiquement :

```
const isUpdate = (searchParams.get("update") === "true") || (user !== null);
```

12.5 L'Endpoint d'Évaluation

Lorsque l'utilisateur répond à la dernière question, le frontend appelle `POST /api/onboarding/evaluate` :

```
# app/modules/onboarding/routers.py
@router.post("/evaluate")
async def evaluate(answer: OnboardingAnswer) -> OnboardingResult:
    flow = load_flow(answer.flow_id)
    result = evaluate_scoring(flow, answer.answers)
    result_screen = load_result_screen(result["profile"])
    return OnboardingResult(**result, **result_screen)
```

Le backend retourne :

1. **Scoring Result** : le profil technique calculé et le score.
2. **Result Screen** : configuration visuelle personnalisée (titre, description, CTA) selon le profil.

12.6 Persistance du Profil

Une fois le compte créé ou mis à jour, les réponses brutes et le profil calculé sont sauvegardés dans la table `UserProfile` (voir Chap 4). Cela permet de :

- Personnaliser l'interface de la page profil.
- Adapter dynamiquement le catalogue ou les recommandations selon le profil.
- Fournir des analytics précis sur la typologie de l'audience (module `analytics`, Chap 13).

12.7 Créer un Flow d'Onboarding pour son Projet

Pour utiliser le module dans un contexte différent (par exemple profiler des acheteurs sur un e-shop) :

1. Créer un nouveau fichier `app/modules/onboarding/flows/mon_flow.json` avec ses questions et règles.
2. Créer les écrans de résultat associés dans `app/modules/onboarding/screens/`.
3. Déclencher le flow depuis le frontend en passant `flow_id="mon_flow"` à l'endpoint.

Aucun code Python à écrire — le moteur générique traite n'importe quel flow respectant le schéma.

L'utilisateur étant profilé, nous passons aux visualisations que le module `analytics` construit à partir de ces données, dans le chapitre suivant.

13 Module Analytics : GeoIP et Visualisation

13.1 Introduction

Le module `analytics` de GitSky enregistre l'activité des visiteurs de manière **respectueuse du RGPD** et alimente le dashboard admin en visualisations de flux et d'audience. Il est activable à partir du tier T1 (`MODULE_ANALYTICS=true`) — en To, la collecte est déportée sur le landing-collector partagé de la flotte (voir Chap 18).

13.2 Le Middleware de Tracking

Un `TrackingMiddleware` intercepte chaque requête entrante, résout l'IP en pays/ville via **MaxMind GeoLite2** (mutualisé sur la flotte), et enregistre une entrée `Visit` en base :

```
# app/modules/analytics/middleware.py - extrait
class TrackingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        response = await call_next(request)
        # Enregistrement asynchrone pour ne pas ralentir la réponse
        asyncio.get_running_loop().run_in_executor(
            None, _store_visit,
            request.client.host, request.url.path, get_user_id(request)
        )
        return response
```

13.2.1 Un Tracking Non-Bloquant

Il est crucial que l'enregistrement d'une visite ne ralentisse pas la réponse. L'appel à `run_in_executor` délègue l'écriture en base à un thread séparé, la réponse HTTP part immédiatement.

13.2.2 Géolocalisation Anonymisée

L'IP réelle n'est jamais stockée ; seul un **hash salé** (`ip_hash`) est conservé pour identifier les visiteurs uniques sans les tracer individuellement.

```
def _store_visit(ip: str, path: str, user_id: int | None):
    geo = geolocate(ip) # Résolution GeoIP mutualisée
    visit = Visit(
        ip_hash=hash_ip(ip), # SHA-256 avec sel de projet
        country_code=geo["country_code"],
        city=geo["city"],
        path=path,
        user_id=user_id,
    )
    # ... persistance
```

13.3 Le Service GeoIP Partagé

Plutôt que de maintenir une base MaxMind par projet (150 Mo par instance), la flotte GitSky partage **un unique service GeoIP** exposé sur le réseau interne du VPS. Chaque projet interroge ce service via `app/shared/clients/geoip_client.py` (voir Chap 18).

Cette mutualisation apporte trois avantages :

- Une seule base MaxMind à mettre à jour mensuellement.
- Une empreinte disque projet minimale.
- Une mise à jour du service GeoIP sans redéploiement des projets.

13.4 Visualisations Admin

Le module apporte deux composants au dashboard admin.

13.4.1 1. Carte Mondiale (World Map)

Basée sur `react-simple-maps` et les données GeoIP agrégées, elle affiche la provenance géographique des visiteurs. Les zones sont colorées par densité.

13.4.2 2. Timeline d'Activité

Basée sur `recharts`, elle génère des graphiques temporels du nombre de visites par jour. Les données sont filtrables par rôle (`anonymous`, `user`, `premium`, `admin`) pour distinguer trafic prospect et engagement client.

```
// Exemple simplifié de l'appel Analytics
const res = await fetch(`/api/admin/analytics/world?days=30`);
const data = await res.json();
// data = { countries: [{ code: "FR", visits: 1234 }, ...] }
```

13.5 Endpoints Analytics

Endpoint	Rôle
GET <code>/api/admin/analytics/world?days=N</code>	Agrégation par pays sur N jours
GET <code>/api/admin/analytics/timeline?days=N</code>	Série temporelle des visites
GET <code>/api/admin/analytics/paths?days=N</code>	Top des URLs consultées

Tous les endpoints sont protégés par la dépendance `require_admin`.

13.6 Conformité RGPD

Le module respecte trois principes qui l'exemptent de la nécessité d'un consentement cookies :

1. **Aucune donnée personnelle** stockée (pas d'IP en clair, pas d'identifiant persistant côté client).
2. **Pas de cookie de tracking** — le `ip_hash` est calculé côté serveur.
3. **Finalité limitée** — analytics purement agrégés, pas d'usage marketing ni de recoupement avec des tiers.

Cette conformité est un choix architectural : GitSky **ne peut pas** stocker d'IP en clair, même si un projet le voulait, sans réécrire le module.

13.7 Purge et Rétention

Les entrées `Visit` sont purgées automatiquement au-delà d'un délai configurable (`ANALYTICS_RETENTION_DAYS` , 90 jours par défaut). Un cron quotidien exécute :

```
DELETE FROM visits WHERE created_at < now() - interval '90 days';
```

pour maintenir la table à taille raisonnable.

Le tracking analytique étant traité, nous voyons dans le chapitre suivant le module `security` qui journalise les tentatives d'intrusion via un middleware complémentaire.

14 Module Security : Détection d’Intrusion

14.1 Introduction

Le module `security` de GitSky ajoute un `SecurityMiddleware` qui inspecte chaque requête entrante à la recherche de patterns d’attaque connus, et journalise les événements suspects dans une table `SecurityEvent`. Il est **activé par défaut à partir du tier T1** (`MODULE_SECURITY_MIDDLEWARE=true`) — en To, la protection est déportée sur le proxy Traefik partagé de la flotte.

14.2 Philosophie : Journaliser, ne pas Bloquer

Le middleware **ne bloque jamais les requêtes** — il journalise uniquement. Cette approche est intentionnelle :

- **Éviter les faux positifs** qui bloqueraient des utilisateurs légitimes.
- **Donner une visibilité complète** sur les menaces sans risquer de casser des flux valides.
- **Déléguer le blocage** à des couches spécialisées (Traefik + `fail2ban`), plus adaptées à la décision temps réel.

Un attaquant qui envoie une charge SQL injection reçoit donc une réponse normale (`200` ou `404` selon la route), mais son comportement est enregistré et visible dans le dashboard admin.

14.3 Types d’Événements Détectés

Le middleware classe les requêtes suspectes en trois catégories :

Type	Exemples	Sévérité
<code>path_scan</code>	<code>/.git/</code> , <code>/.env</code> , <code>/wp-config.php</code> , <code>/admin.php</code>	medium → critical
<code>scanner_detected</code>	User-Agent de sqlmap, nikto, nmap, nuclei, gobuster	high
<code>injection_attempt</code>	SQL injection (<code>' OR 1=1--</code>), XSS (<code><script></code>), template injection (<code>{{7*7}}</code>)	critical

Chaque catégorie a ses propres règles de détection dans `app/modules/security/detectors.py`.

14.4 Le Modèle SecurityEvent

Chaque détection produit une entrée SQLAlchemy (voir Chap 4 pour la définition) :

```
class SecurityEvent(Base):
    __tablename__ = "security_events"
    event_type = Column(String, index=True)
    severity = Column(String, index=True)
    ip_address = Column(String, index=True) # IP en clair – attaquants, pas RGPD
    path = Column(String)
    user_agent = Column(String)
    details = Column(JSON)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

Note importante : contrairement au module `analytics` où l'IP est hashée, le module `security` stocke l'IP en clair. La justification RGPD est que les attaquants ne sont pas des utilisateurs légitimes et que la traçabilité prime pour la défense de la plateforme.

14.5 Interface Admin

L'onglet **Sécurité** du dashboard admin (Chap 9) expose :

- **Cartes de synthèse** : total des événements, dernières 24h, critiques, élevés.
- **Top 10 des IPs agressives** : identifier les attaquants récurrents.
- **Journal filtrable** par sévérité, type, IP, période.

```
// Appels API utilisés par l'onglet Sécurité
GET    /api/admin/security/summary?days=7
GET    /api/admin/security/events?severity=critical&page=1&per_page=50
DELETE /api/admin/security/events/old?days=30 // purge périodique
```

14.6 Complémentarité avec Traefik

Le middleware GitSky détecte les attaques applicatives. Le blocage effectif — refuser des connexions au niveau réseau — est délégué à **Traefik + fail2ban** au niveau du VPS partagé :

- Un job périodique lit les événements `SecurityEvent` de sévérité `high` et `critical` de tous les projets.
- Il agrège par IP source sur les 24 dernières heures.
- Les IPs dépassant un seuil (10 événements `critical` ou 50 événements `high`) sont poussées dans une allowlist négative Traefik.
- Traefik refuse ensuite ces IPs sans jamais atteindre les backends GitSky.

Cette architecture à deux niveaux — détection applicative par projet + blocage réseau mutualisé — est décrite en détail au Chap 22 (configuration serveur).

14.7 Purge et Rétention

Les événements de sévérité `low` et `medium` sont purgés au bout de 30 jours. Les événements `high` et `critical` sont conservés 180 jours pour analyse rétrospective. La purge est déclenchée par un cron quotidien.

14.8 Ce que le Middleware ne Fait Pas

Trois choses volontairement absentes :

1. **Pas de blocage** — comme expliqué, c'est le rôle de Traefik.
2. **Pas de notification temps réel** par événement — les alertes agrégées vont dans le fleet dashboard (Chap 19), un email par événement noierait le signal.
3. **Pas de rate limiting** — Traefik gère cela via son propre middleware `RateLimit`. Cette doctrine n'est pas qu'une intention : le générateur émet dans le `docker-compose.yml` de production un routeur Traefik dédié à `/api/auth/login` (5 req/min, burst 10) dès que le module auth est actif — voir Chap 21.
4. **Pas d'inspection des endpoints d'infrastructure** — `/health` (pollé toutes les 60 s par le fleet poller), `/robots.txt` et `/sitemap.xml` sont exclus du middleware, comme du tracking analytics (Chap 13) : journaliser chaque poll noierait les vraies détections et coûterait un commit DB par requête pour rien.

Ces choix maintiennent le module léger et prévisible.

La journalisation défensive étant en place, nous passons dans les chapitres suivants aux modules qui apportent de la valeur métier au projet — framework agentic (Chap 15), monétisation Stripe (Chap 16).

15 Framework Agentic IA et Services Intelligents

15.1 Introduction au Framework Multi-Agents

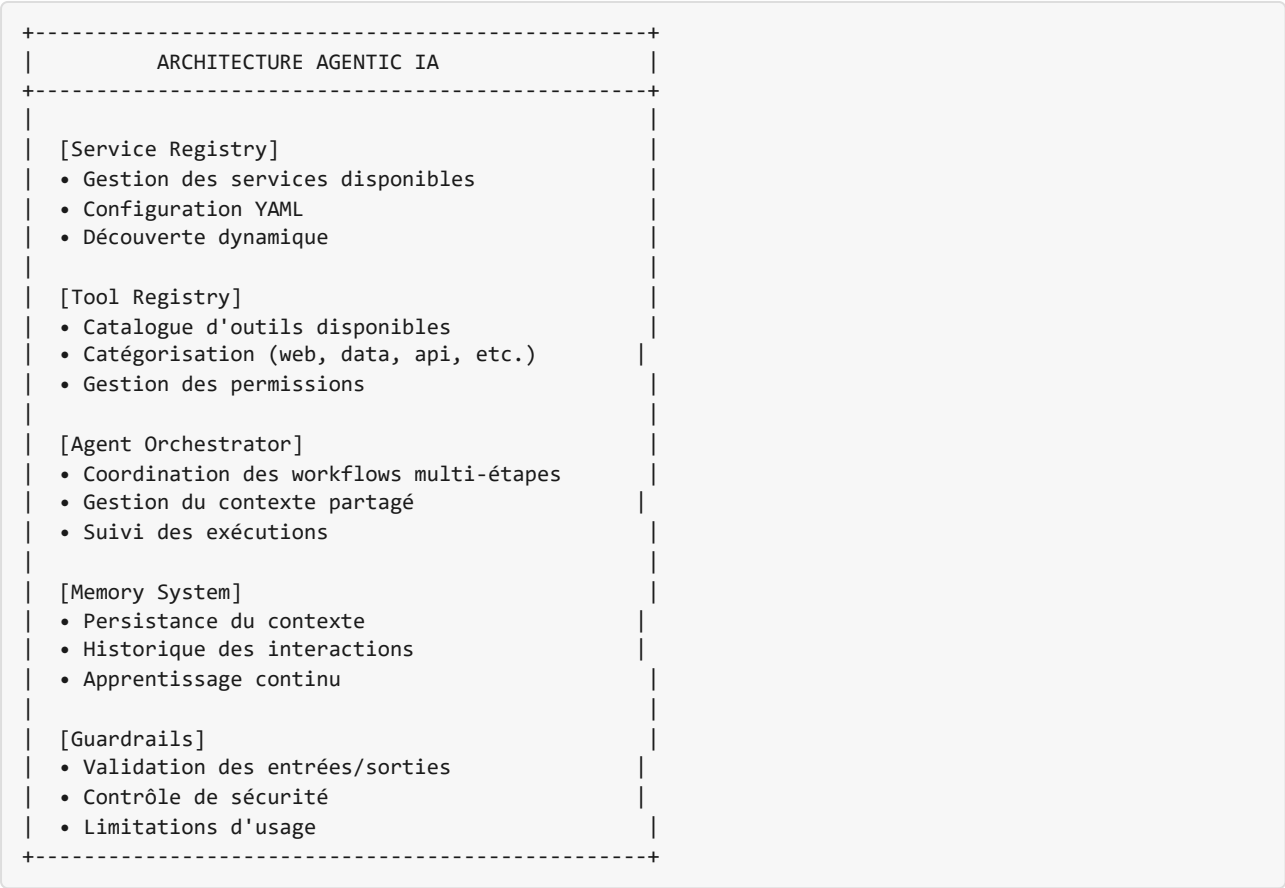
Le projet **GitSky** intègre un framework complet d’agents IA permettant de créer des services intelligents automatisés. Ce système multi-agents transforme la plateforme d’apprentissage en un écosystème d’automatisation où différents agents spécialisés collaborent pour exécuter des tâches complexes.

15.1.1 Philosophie du “GitSky”

Le nom du projet prend tout son sens avec ce framework : l’objectif est de minimiser l’intervention humaine dans les processus répétitifs tout en maximisant la valeur créée par l’intelligence artificielle.

15.2 Architecture du Système Multi-Agents

L’architecture repose sur cinq composants principaux orchestrant la collaboration entre agents :



15.3 Le LLM Proxy Partagé

Un projet GitSky avec `MODULE_AGENTIC=true` n'appelle **jamais directement** les APIs Anthropic ou OpenAI. Tous les appels transitent par un **LLM proxy partagé** hébergé sur le VPS de la flotte, qui apporte quatre bénéfices essentiels :

Bénéfice	Détail
Clé API centralisée	Une seule clé Anthropic (ou OpenAI) sur le VPS, jamais dupliquée dans les projets
Quotas par projet	Chaque projet a un budget journalier et mensuel configurable
Logs centralisés	Toutes les requêtes LLM sont journalisées avec projet, modèle, tokens, coût
Fallback et rotation	Si un modèle est indisponible, le proxy bascule automatiquement vers un modèle équivalent

L'implémentation recommandée est **LiteLLM** — un proxy open source qui expose une API compatible OpenAI et route vers Anthropic, OpenAI, Groq ou Ollama selon la configuration. L'installation et la configuration côté flotte sont décrites au Chap 18.

Côté projet, l'agent appelle simplement le client :

```
# app/shared/clients/llm_proxy_client.py
from openai import OpenAI
from app.core.config import get_settings

settings = get_settings()
client = OpenAI(
    base_url=settings.llm_proxy_url,      # http://llm-proxy:4000
    api_key=settings.llm_proxy_token,    # Token de projet, décodé par LiteLLM
)

response = client.chat.completions.create(
    model="claude-sonnet-4-6",
    messages=[{"role": "user", "content": "..."}],
)
```

Le proxy authentifie le projet via son token, décrémente son quota, journalise l'appel, et transmet la requête au fournisseur. En cas de dépassement de quota, il renvoie une erreur `429` que l'agent traite comme un signal de backoff.

15.4 Configuration des Services via YAML

Le framework utilise une configuration déclarative en YAML pour définir les services disponibles. Cette approche permet de modifier le comportement des agents sans toucher au code Python.

15.4.1 Schéma d'un service

Un service déclare deux choses : des **steps** et des **workflows**.

- Un **step** est une unité de travail nommée, de deux natures possibles :
 - `type: agent` — un appel LLM (modèle + prompt système + température) ;
 - `type: tool` — un appel à un **tool** enregistré (callable Python du registre `tools/`), qui encapsule une API externe (génération d'image, d'audio...).
- Un **workflow** est une liste ordonnée de noms de steps. Un même service peut en exposer plusieurs (un aperçu court, une génération complète...).
- `async_workflows` liste les workflows **longs**, exécutés en job de fond (voir plus bas) ; les autres s'exécutent de façon synchrone.
- `cost_credits` est débité lorsqu'un workflow payant (asynchrone) démarre.

```
# app/modules/agentic/agent_services.yaml
services:
  song_generator:
    enabled: true
    name: "Générateur de chanson"
    description: "Écrit les paroles puis génère l'audio via une API externe"
    category: "music"
    cost_credits: 3
    steps:
      analyze:
        type: agent
        model: "claude-sonnet-5"
        system_prompt: "Résume en 3 lignes l'intention artistique..."
        temperature: 0.5
      lyrics:
        type: agent
        model: "claude-opus-4-8"
        system_prompt: "Écris les paroles fidèles au thème et à la structure..."
        temperature: 0.8
      render:
        type: tool                # appel d'API externe (registre tools/)
        tool: suno_generate
    workflows:
      concept: [analyze, lyrics]    # synchrone : aperçu peu coûteux
      song: [analyze, lyrics, render] # complet
    async_workflows: [song]        # « song » tourne en job de fond
```

IDs de modèles. Utilisez toujours des identifiants Claude à jour (`claude-opus-4-8`, `claude-sonnet-5`, `claude-sonnet-4-6` ...). Un ID obsolète recopié depuis un exemple casse silencieusement le service en production.

15.4.2 Services de référence

Le châssis embarque `template_service` (démonstration minimale d'un step agent). L'exemple complet — un générateur de chanson à pipeline `analyze → lyrics → style → Suno` — est livré dans `examples/mezouedai/` (voir la démonstration $T_0 \rightarrow T_1 \rightarrow T_2$).

15.5 Modèles de Données pour le Tracking des Exécutions

Pour la traçabilité et l'audit, le module utilise trois tables :

```
# app/modules/agentik/models.py – extraits
class ServiceExecution(Base):
    """Exécution complète d'un workflow."""
    __tablename__ = "service_executions"
    # user_id, service_slug, workflow_name, status, input_params, result,
    # cost_credits, created_at
    # status : pending -> running -> completed | failed
    # cost_credits : le coût réellement débité, PERSISTÉ – condition du
    # remboursement après un redémarrage qui a tué le job en vol (voir $Jobs).

class ExecutionStep(Base):
    """Une étape du workflow – checkpoint pour l'audit et la reprise."""
    __tablename__ = "execution_steps"
    # execution_id, idx, name, kind (agent|tool), status, output, created_at

class CreditAccount(Base):
    """Portefeuille de crédits : une génération payante débite ici."""
    __tablename__ = "credit_accounts"
    # user_id (unique), balance
```

`ServiceExecution` porte aussi `cost_credits` : le coût réellement débité, **persisté** (et pas seulement passé en paramètre au job). C'est la condition du remboursement après coup — voir « Reprise après redémarrage » plus bas.

`ExecutionStep` est la « table steps » : chaque étape exécutée y est persistée, ce qui rend l'exécution auditable et **reprenable** (un job interrompu peut repartir de sa dernière étape). Des raffinements ultérieurs (résultats médias détaillés, préférences utilisateur par service) viendront s'ajouter au besoin.

15.6 Le Moteur d'Orchestration

Le moteur (`app/modules/agentik/engine.py`) exécute un workflow : il enchaîne les steps déclarés, **passé un contexte accumulé** d'une étape à la suivante (la sortie de `analyze` nourrit `lyrics`, etc.), trace chaque étape dans `execution_steps`, et remplit le résultat. C'est le pipeline du GitSky Studio (Chap 24) généralisé et rendu piloté par YAML.

Deux principes hérités du Studio :

- **Stub par défaut, fail-closed en production.** Sans LLM proxy configuré, `call_llm` renvoie une réponse simulée déterministe, et un tool comme `suno_generate` renvoie une URL d'exemple. On développe et teste **sans aucune clé** ; le réel s'active en fournissant les variables d'environnement (proxy LLM, `SUNO_API_KEY`). Garde-fou impératif : si `ENVIRONMENT=production` et que la clé manque, le client **lève** au lieu de servir le stub — facturer des crédits pour une réponse simulée n'est jamais un fallback acceptable. Ce contrat vaut pour toute intégration externe du châssis (Stripe compris, Chap 16) et est verrouillé par un test paramétré (`test_failclosed_contract.py`).

- **Sortie structurée et tracée.** Le résultat expose la sortie de chaque step, plus une clé `output` pratique (le dernier texte d'agent — les paroles, pour un aperçu).

15.7 Tâches Longues : Job Asynchrone, sans Broker

La génération média délègue le gros du travail à une **API externe** (Suno) : le backend ne calcule rien, il **attend**. Inutile donc d'introduire un worker Celery + broker (qui casserait la densité de la flotte, Chap 2/21). Le pattern retenu, pour un workflow listé dans `async_workflows` :

1. `execute` débite les crédits, crée l'exécution `pending`, lance le pipeline en **tâche de fond in-process** (`asyncio`) et **rend la main immédiatement** (submit-and-return) — la connexion HTTP n'est jamais tenue pendant des minutes.
2. Le client **suit** l'avancement via `GET /executions/{id}` (polling) jusqu'à `completed`.
3. Si une étape échoue, l'exécution passe `failed` et les crédits sont remboursés.

Les étapes étant **I/O-bound** (appels LLM et API), une simple tâche async suffit : la durabilité vient de la ligne d'exécution persistée + des checkpoints `ExecutionStep`. Pour une API réellement asynchrone (soumission puis webhook de fin), l'exécution passerait par un statut `awaiting_callback` repris par un endpoint webhook — extension documentée, non nécessaire tant que le tool répond en ligne.

15.7.1 Reprise après redémarrage

Le revers du « job in-process » : une tâche `asyncio` ne survit pas à un redémarrage (deploy, crash, OOM). Sans filet, une exécution resterait `running` pour toujours et ses crédits, débités, ne seraient jamais rendus. Au démarrage de l'app (`lifespan`), avant que la moindre tâche ne tourne, toute exécution encore `pending` / `running` est donc **par définition orpheline** : `recover_orphan_executions` la passe `failed` et rembourse son `cost_credits` persisté. C'est précisément pourquoi le coût est stocké sur la ligne d'exécution et pas seulement dans la variable du job.

15.7.2 Propriété des exécutions

`GET /executions/{id}` ne renvoie l'exécution qu'à **son propriétaire** (un admin gardant l'accès pour le support). Pour tout autre utilisateur, la réponse est `404`, pas `403` : renvoyer `403` confirmerait l'existence de l'id (fuite par énumération). Le débit de crédits, lui, est un `UPDATE` conditionnel atomique (`balance >= coût`) et non un lire-puis-écrire — sans quoi deux requêtes concurrentes pourraient dépenser deux fois le même solde.

15.8 API Agent-Services

Le backend expose une API REST complète pour interagir avec le framework :

15.8.1 Endpoints Principaux

```
# app/modules/agentic/router.py

# Catalogue des services (public)
GET /api/agent-services/services
GET /api/agent-services/services/{service_slug}

# Solde de crédits de l'utilisateur (auth)
GET /api/agent-services/credits

# Exécution d'un workflow (auth)
POST /api/agent-services/services/{service_slug}/execute
Body: { "workflow_name": "song", "parameters": {...} }
# - workflow court -> synchrone : renvoie `completed` + résultat ;
# - workflow dans async_workflows -> submit-and-return : renvoie `pending` + id.

# Suivi / polling d'une exécution (auth)
GET /api/agent-services/executions/{execution_id}
```

Il n'y a pas d'endpoint « liste des outils » : un tool n'est pas exposé seul, il est un **type de step** référencé par nom depuis le YAML (`type: tool`).

15.8.2 Exemple d'Exécution

```
import requests

# Lancer la génération complète (workflow asynchrone)
response = requests.post(
    "https://api.votre-domaine.com/api/agent-services/services/song_generator/execute",
    headers={"Authorization": "Bearer <token>"},
    json={
        "workflow_name": "song",
        "parameters": {"singer": "Slah", "theme": "Exil", "rhythm": "Saltana"},
    },
)
job_id = response.json()["id"] # statut initial : "pending"

# Puis suivre le job jusqu'à "completed" (polling)
requests.get(
    f"https://api.votre-domaine.com/api/agent-services/executions/{job_id}",
    headers={"Authorization": "Bearer <token>"},
)
```

15.9 Dashboard Agentic Frontend

Le frontend inclut une interface dédiée pour interagir avec les services agents :

15.9.1 Structure des Composants

```
frontend/src/  
├── agent-dashboard/  
│   ├── DashboardLayout.tsx    # Layout principal  
│   └── ServiceCard.tsx        # Carte de service  
├── agent-services/  
│   └── (composants spécifiques par service)  
└── agent-commons/  
    └── (utilitaires partagés)
```

15.9.2 Fonctionnalités du Dashboard

1. **Catalogue de Services** : Vue grid des services disponibles avec filtrage par catégorie
2. **Exécution en 1-Click** : Lancement des workflows prédéfinis
3. **Historique des Exécutions** : Suivi en temps réel des tâches en cours
4. **Gestion des Préférences** : Personnalisation des paramètres par défaut
5. **Documentation des Outils** : Explorer les capacités disponibles

15.10 Création d'un Nouveau Service Agentic

15.10.1 Étapes de Développement

1. Définir le service dans le YAML

```
services:  
  mon_nouveau_service:  
    name: "Mon Nouveau Service"  
    description: "Description du service"  
    category: "custom"  
    steps: { ... }      # agents et/ou tools  
    workflows: { ... }  # listes ordonnées de noms de steps
```

2. Implémenter la logique métier

```
# backend/app/agents/services/mon_service/  
# └── __init__.py  
# └── implémentation des agents
```

3. Créer l'interface frontend

```
// frontend/src/agent-services/mon-service/  
// └── MonServiceInterface.tsx  
// └── MonServiceResults.tsx
```

4. Tester l'intégration

- Vérifier l'apparition dans le catalogue
- Tester l'exécution du workflow

- Valider la persistance des résultats

15.10.2 Bonnes Pratiques

- **Isolation** : Chaque service doit être indépendant
- **Idempotence** : Les exécutions doivent être reproductibles
- **Journalisation** : Logs détaillés pour le debugging
- **Gestion d'erreurs** : Retours d'erreur clairs pour les utilisateurs
- **Performance** : Timeouts appropriés et traitement asynchrone

15.11 Intégration avec les Autres Composants de GitSky

15.11.1 Authentification et Autorisation

- Les services respectent le système de rôles existant (user, premium, admin)
- Certains services peuvent être réservés aux utilisateurs premium
- Audit via les logs d'activité existants

15.11.2 Analytics et Monitoring

- Les exécutions sont trackées dans les statistiques admin
- Performance mesurée (temps d'exécution, succès/échec)
- Intégration avec le système de notification

15.11.3 Gestion des Données

- Persistance des résultats dans PostgreSQL
- Support des fichiers multimédias via le système d'upload existant
- Nettoyage automatique des données temporaires

15.12 Scénarios d'Utilisation Avancés

15.12.1 Workflows Croisés

Combiner plusieurs services pour des processus complexes :

```

workflow_complexe:
  steps:
    - service: research_agent
      workflow: literature_review
      params: {topic: "AI ethics"}
    - service: news_scraper
      workflow: daily_news_digest
      params: {topics: ["AI ethics news"]}
    - service: custom_service
      workflow: generate_report
      params: {format: "pdf"}

```

15.12.2 Planification Automatique

Exécution périodique de services via le scheduler intégré :

```

# Configuration dans le scheduler
schedule.every().day.at("09:00").do(
    execute_service,
    service_slug="news_scraper",
    workflow_name="daily_news_digest"
)

```

15.12.3 Personnalisation Dynamique

Adaptation des services basée sur le profil utilisateur :

```

# Utilisation des préférences utilisateur
prefs = get_user_service_preferences(user_id, "news_scraper")
params = merge_params(default_params, prefs.get("custom_params", {}))

```

15.13 Dépannage et Maintenance

15.13.1 Diagnostic des Problèmes

1. Vérifier l'état des services

```

# Logs des exécutions
docker logs gitsky_backend | grep -i "agent\|service"

# État de la base de données
psql -U hitl_user -d hitl_db -c "SELECT * FROM service_executions ORDER BY created_at DESC LIMIT 5;"

```

2. Tester manuellement un service

```

curl -X POST http://localhost:8000/api/agent-services/services/template_service/execute \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"workflow_name": "example_workflow"}'

```

3. Vérifier la configuration YAML

```
python -c "import yaml; data = yaml.safe_load(open('backend/app/config/agent_services.yaml'));  
print(data.keys())"
```

15.13.2 Maintenance Courante

- **Nettoyage des anciennes exécutions** : Script de rétention configurable
- **Mise à jour des modèles IA** : Rotation des versions d'API
- **Optimisation des performances** : Monitoring des temps d'exécution
- **Sécurité** : Revue régulière des permissions et accès

Ce framework transforme GitSky d'une plateforme statique en un véritable écosystème d'automatisation intelligente. Le prochain chapitre présente le dernier module standard : la monétisation Stripe, qui permet à un projet en tier T2 de générer du revenu récurrent ou de vendre des produits numériques.

16 Monétisation avec Stripe : Boutique et Abonnements

16.1 Introduction

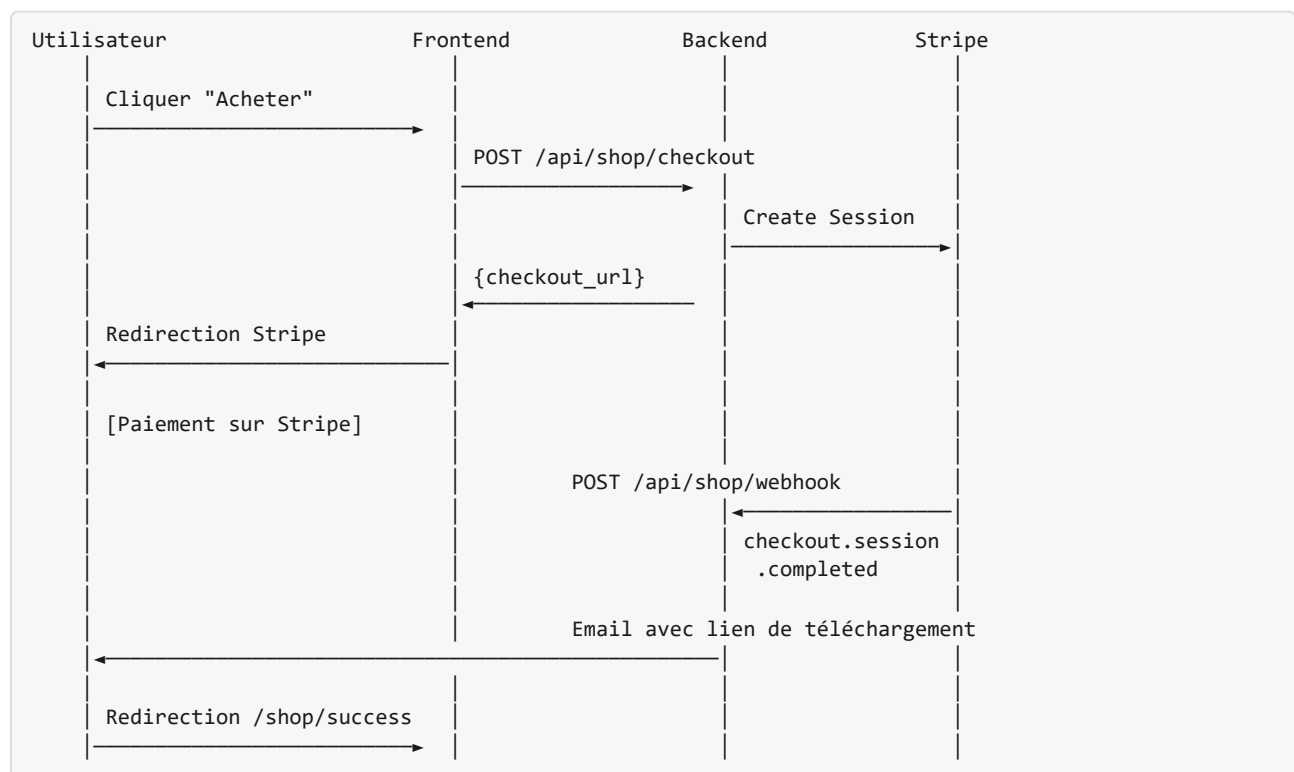
Une plateforme SaaS ne peut survivre sans modèle économique. Le template GitSky intègre deux stratégies de monétisation complémentaires, toutes deux activables indépendamment via des variables d'environnement. Cette approche **feature-flag** permet de démarrer sans paiement et d'activer la monétisation uniquement quand vous êtes prêt.

```
MONETIZATION_SHOP=true           # Vente de produits numériques (achat unique)
MONETIZATION_SUBSCRIPTION=true   # Abonnements récurrents → rôle premium
```

Les deux modes utilisent **Stripe**, la référence mondiale du paiement en ligne, via son **Hosted Checkout** — vous ne gérez jamais de numéros de carte en direct sur votre serveur.

16.2 Architecture Webhook-First

La clé de fiabilité de ce système est son architecture **webhook-first**. Plutôt que de se fier à la redirection de l'utilisateur après paiement (qui peut échouer si le navigateur se ferme), le fulfillment s'appuie sur les événements envoyés directement par Stripe à votre serveur.



16.3 Stripe Multi-Projets : un Compte, N Produits

Stripe n'autorise pas la création de nombreux comptes pour un même opérateur — c'est explicitement interdit par leurs conditions d'utilisation, sanctionné par le gel des paiements. La flotte GitSky utilise donc **un unique compte Stripe**, partagé entre tous les projets qui activent la monétisation.

L'isolation entre projets se fait au niveau **des produits et des métadonnées** :

- Chaque projet a son propre préfixe pour les slugs de produits (`pain-scraper_guide-fastapi`).
- Chaque `Product` et `Subscription` Stripe porte une **métadonnée `project_name`** indispensable au routage des webhooks.
- Un unique endpoint webhook au niveau de la flotte parse la métadonnée et route l'événement vers la base du bon projet.

```
# services/stripe_webhook_router.py (fleet-level, voir Chap 18)
@app.post("/api/shop/webhook")
async def route_webhook(request: Request):
    payload = await request.body()
    event = stripe.Webhook.construct_event(
        payload, request.headers["stripe-signature"], WEBHOOK_SECRET
    )
    project = event.data.object.metadata.get("project_name")
    if not project:
        raise HTTPException(400, "Missing project_name metadata")
    await forward_to_project(project, event)
```

Cette architecture n'ajoute qu'un saut réseau interne (quelques millisecondes) et évite d'avoir à gérer N clés webhook différentes chez Stripe. Le routage est décrit en détail au Chap 18 (Services partagés).

Point d'attention légal : l'unique compte Stripe est rattaché à une seule entité juridique (SAS, LLC...) qui porte donc tous les projets de la flotte. Un projet qui atteindrait une taille justifiant sa propre entité devrait migrer vers son propre compte Stripe — cette migration est prévue au Chap 20 (cycle de vie).

16.4 Sécurité du Webhook Router

Le fait qu'un unique endpoint webhook reçoive les événements Stripe de tous les projets ouvre trois questions de sécurité qu'il faut trancher explicitement.

16.4.1 Le Secret Webhook : Partagé ou par Projet ?

Stripe permet de configurer plusieurs webhooks avec chacun leur propre secret. Deux options :

Option	Description	Avantages	Inconvénients
A — Secret unique	Un webhook Stripe, un secret partagé, un endpoint qui route	Configuration simple, un seul secret à faire tourner	Si le secret fuit, tous les projets sont exposés
B — Secret par projet	Un webhook Stripe par projet (10-30 webhooks), un endpoint valide le bon secret par projet	Isolation forte, un secret compromis n'affecte qu'un projet	Multiplication des configurations Stripe, ré-armement plus complexe

Recommandation : Option A pour les projets To-T1, Option B dès qu'un projet dépasse ~1 000 € de MRR.

Le secret partagé est un compromis acceptable tant que tous les projets sont sous le même contrôle opérationnel. Dès qu'un projet atteint un revenu où la fuite de données de paiement serait coûteuse, il mérite son propre webhook Stripe avec son propre secret.

16.4.2 La Frontière de Confiance Router → Projet

Le webhook router s'exécute au niveau flotte et doit forwarder l'événement au bon projet. Modèle recommandé (push) :

```
Stripe → [Router flotte] → POST http://<projet>-backend:8000/api/shop/webhook-relay
Authorization: Bearer $FLEET_WEBHOOK_RELAY_TOKEN
Body: événement Stripe déjà validé + project_name
```

Le projet fait confiance au router — il **ne re-vérifie pas** la signature Stripe. Il vérifie uniquement le token de relais interne.

Le `FLEET_WEBHOOK_RELAY_TOKEN` est **différent du secret Stripe** : - Il vit uniquement sur le réseau interne du VPS (`shared-services-net`). - Il n'est jamais exposé à l'extérieur. - Sa rotation est décorrélée de celle du secret Stripe (rotation trimestrielle).

16.4.3 Ce qui Reste Isolé Même avec Secret Partagé

- Les clés API projet (`STRIPE_SECRET_KEY` par projet) sont propres à chaque projet — un secret webhook compromis ne donne pas accès aux fonds.
- Les métadonnées `project_name` sont vérifiées côté router avant forward — un projet malveillant ne peut pas déclencher des webhooks pour un autre.
- Le forward push est authentifié par le token de relais interne — un service compromis sur le réseau externe ne peut pas injecter directement dans un projet.

16.4.4 Contrainte Stripe Legal

Stripe interdit explicitement les comptes partagés entre entités juridiques distinctes. Toute la flotte doit donc être détenue par la **même entité** (SAS unique, LLC unique). Un projet vendu à un tiers doit être migré vers un nouveau compte Stripe avant la cession — procédure décrite au Chap 20

(émancipation).

16.5 Boutique de Produits Numériques

16.5.1 Modèle de données

```
class Product(Base):
    name          = Column(String)           # Affiché au client
    slug          = Column(String, unique=True)
    price_cents   = Column(Integer)         # 2900 = 29,00 €
    stripe_price_id = Column(String)         # price_... (depuis Stripe Dashboard)
    file_path     = Column(String)         # Chemin fichier sur le serveur
    is_active     = Column(Boolean)

class Purchase(Base):
    user_id       = Column(Integer, nullable=True) # Nullable = achat invité
    product_id    = Column(Integer)
    email         = Column(String)
    stripe_session_id = Column(String, unique=True)
    download_token = Column(String, unique=True)   # Token URL aléatoire
    download_count = Column(Integer, default=0)
    max_downloads = Column(Integer, default=5)
    token_expires_at = Column(DateTime)
    fulfilled_at   = Column(DateTime)             # Défini par le webhook
```

16.5.2 Configurer un produit

Étape 1 — Créer le produit dans Stripe Dashboard

1. Stripe Dashboard → Products → Add Product
2. Définir le nom, la description, le prix en mode “One time”
3. Copier le `Price ID` (format `price_xxx`)

Étape 2 — Créer le produit en base de données

```
POST /api/admin/shop/products
Authorization: Bearer <admin_token>

{
  "name":          "Guide Complet FastAPI",
  "slug":          "guide-fastapi",
  "description":   "200 pages de référence pratique",
  "price_cents":   2900,
  "currency":      "eur",
  "stripe_price_id": "price_1ABC...",
  "file_path":     "/data/products/guide-fastapi.pdf",
  "cover_image":   "https://cdn.exemple.com/guide-cover.jpg"
}
```

Étape 3 — Placer le fichier sur le serveur

Le `file_path` doit pointer vers un fichier accessible par le conteneur backend. En production, montez un volume Docker :

```
# docker-compose.prod.yml
services:
  backend:
    volumes:
      - /opt/my-app/products:/data/products:ro # read-only pour le backend
```

16.5.3 Sécurité des téléchargements

Le mécanisme de téléchargement est intentionnellement restrictif :

```
@router.get("/download/{token}")
def download_file(token: str, db: Session = Depends(get_db)):
    purchase = db.query(Purchase).filter(
        Purchase.download_token == token
    ).first()

    # Vérifications en cascade
    if not purchase or not purchase.fulfilled_at:
        raise HTTPException(404) # Token inconnu ou non payé
    if now > purchase.token_expires_at:
        raise HTTPException(410, "Lien expiré") # Durée dépassée
    if purchase.download_count >= purchase.max_downloads:
        raise HTTPException(410, "Limite atteinte") # Trop de téléchargements

    purchase.download_count += 1
    db.commit()
    return FileResponse(purchase.product.file_path) # Servi par FastAPI
```

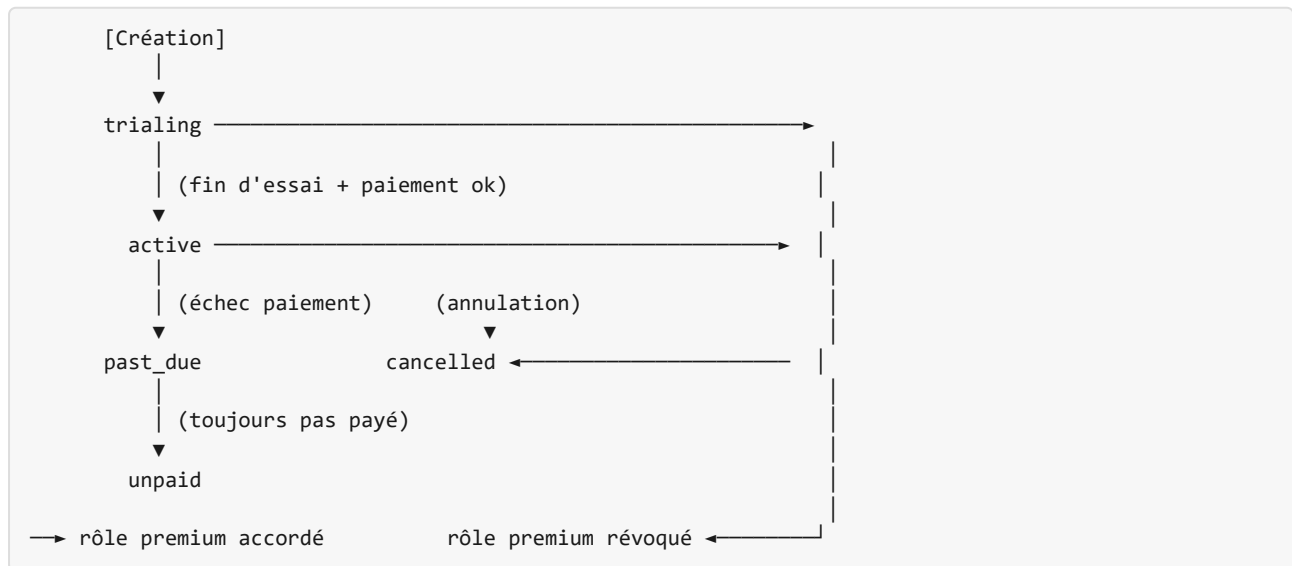
Les tokens sont des chaînes aléatoires de 48 octets (`secrets.token_urlsafe(48)`), impossibles à deviner. La durée de validité et le nombre maximum de téléchargements sont configurables dans `.env`.

16.6 Abonnements Stripe

16.6.1 Modèle de données

```
class Subscription(Base):
    user_id = Column(Integer, unique=True) # 1 abonnement par user
    stripe_subscription_id = Column(String, unique=True) # sub_...
    stripe_customer_id = Column(String) # cus_...
    stripe_price_id = Column(String) # plan souscrit
    status = Column(Enum(SubscriptionStatus))
    current_period_end = Column(DateTime)
    trial_end = Column(DateTime)
    cancelled_at = Column(DateTime)
```


16.6.2 Statuts et transitions



La règle est simple : `status in (active, trialing) → rôle premium`. Tout autre statut → rôle `user`. Cette logique est appliquée automatiquement dans les handlers webhook.

Relier l'événement à un compte — via `metadata`. Un point qui piège : les événements `customer.subscription.*` de Stripe ne portent **aucun de nos identifiants internes** (Stripe ne connaît que ses `cus_...` / `sub_...`). Le lien vers notre `user_id` est établi au moment du checkout, où on l'écrit dans `metadata` ; Stripe renvoie ce `metadata` tel quel dans chaque événement, et le handler y lit `metadata.user_id`.

❌ `user_id = event.data.object.user_id` — toujours `None` sur un vrai événement Stripe ; le compte ne passe jamais premium. ✅ `user_id = event.data.object.metadata.get("user_id")` — posé au checkout, présent partout ensuite.

Un événement sans `metadata.user_id` (abonnement créé hors de notre checkout, webhook mal configuré) est **ignoré** plutôt que d'appliquer un rôle au mauvais compte.

16.6.3 Configurer un plan d'abonnement

Dans Stripe Dashboard : 1. Products → Add Product (choisir “Recurring”) 2. Définir l'intervalle : mensuel, annuel, etc. 3. Copier le `Price ID` du plan

L'API liste les plans disponibles automatiquement depuis Stripe :

```
GET /api/subscription/plans
# Retourne tous les prices actifs de type recurring
```

16.6.4 Essai gratuit

```
MONETIZATION_TRIAL_DAYS=14 # 14 jours d'essai, puis facturation
```

Avec cette configuration, la session Stripe Checkout affiche automatiquement la période d'essai. Le rôle `premium` est accordé dès le début de l'essai (statut `trialing`).

16.6.5 Customer Portal Stripe

Plutôt que de recoder une interface de gestion d'abonnement (changer de plan, mettre à jour la carte, annuler), nous déléguons entièrement à Stripe :

```
POST /api/subscription/portal
Authorization: Bearer <token>
{ "return_url": "https://votre-site.com/profile" }

# Réponse
{ "portal_url": "https://billing.stripe.com/session/..." }
```

L'utilisateur est redirigé vers un portail Stripe hébergé, puis renvoyé vers votre site via `return_url`.

16.7 Webhook — Le Cœur du Système

L'endpoint `/api/shop/webhook` est le seul point d'entrée pour tous les événements Stripe. La signature est vérifiée à chaque appel.

```
@router.post("/webhook", include_in_schema=False)
async def stripe_webhook(request: Request, db: Session = Depends(get_db)):
    payload = await request.body()
    sig = request.headers.get("stripe-signature")

    # Vérification HMAC — rejette tout appel non signé par Stripe
    event = stripe.Webhook.construct_event(
        payload, sig, settings.stripe_webhook_secret
    )

    match event["type"]:
        case "checkout.session.completed":
            if event["data"]["object"]["mode"] == "payment":
                _fulfill_shop_purchase(db, ...) # boutique
            else:
                _fulfill_subscription(db, ...) # abonnement

        case "customer.subscription.updated":
            _update_subscription(db, ...) # sync statut + rôle

        case "customer.subscription.deleted":
            _cancel_subscription(db, ...) # révoque premium

        case "invoice.payment_succeeded":
            # Renouvellement ok → s'assurer que le statut est active
            ...

        case "invoice.payment_failed":
            # Mettre en past_due, notifier l'utilisateur
            ...
```

Fail-closed en production. En développement, sans `STRIPE_WEBHOOK_SECRET`, le client Stripe est stubbé et le corps est parsé directement (relais interne déjà validé). Ce raccourci est **interdit en production** : si `ENVIRONMENT=production` et que le secret manque, `verify_webhook` lève et l'endpoint répond `503` — Stripe réessaiera une fois le webhook configuré, aucun événement n'est

perdu. Sans ce garde-fou, un secret oublié laisserait accepter des événements **forgés** : fulfillment gratuit et passage premium arbitraire. Une signature invalide (secret présent) donne un `400`. Ce contrat est verrouillé par `test_failclosed_contract.py`.

16.7.1 Configurer le webhook en production

1. Stripe Dashboard → Developers → Webhooks → Add Endpoint
2. URL : `https://api.votre-domaine.com/api/shop/webhook`
3. Événements à sélectionner :
 - `checkout.session.completed`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
 - `invoice.payment_succeeded`
 - `invoice.payment_failed`
4. Copier le **Signing Secret** (`whsec_...`) → `STRIPE_WEBHOOK_SECRET` dans `.env`

16.7.2 Tester en local

Stripe CLI permet de rediriger les webhooks vers votre serveur local :

```
# Installer Stripe CLI
brew install stripe/stripe-cli/stripe

# Écouter et rediriger vers le backend local
stripe listen --forward-to localhost:8000/api/shop/webhook

# Dans un autre terminal – simuler un paiement
stripe trigger checkout.session.completed
```

16.8 Pages Frontend

16.8.1 /shop — Catalogue produits

```
// Chargement des produits publics (pas d'auth requise)
useEffect(() => {
  fetch(`${API}/api/shop/products`)
    .then(r => r.json())
    .then(setProducts);
}, []);

// Checkout – redirige vers Stripe
async function handleBuy(product: Product) {
  const res = await fetch(`${API}/api/shop/checkout`, {
    method: "POST",
    headers: { Authorization: `Bearer ${accessToken}` },
    body: JSON.stringify({ product_slug: product.slug }),
  });
  const { checkout_url } = await res.json();
  window.location.href = checkout_url; // Redirection vers Stripe
}
```

16.8.2 /premium — Plans d'abonnement

La page charge les plans depuis Stripe en temps réel, affiche le statut de l'abonnement courant, et propose l'accès au Customer Portal Stripe pour gérer/annuler.

16.8.3 Profil — Section “Mes achats”

La section **Mes achats** dans `/profile` liste tous les achats fulfilled de l'utilisateur avec : - Nom du produit + date + montant payé - Lien de téléchargement actif (si token valide et téléchargements restants) - Indication “Lien expiré” sinon

16.9 Checklist de Mise en Production

- ☐ Créer un compte Stripe et activer le mode Production
- ☐ Créer les produits/plans dans Stripe Dashboard
- ☐ Renseigner STRIPE_SECRET_KEY, STRIPE_PUBLIC_KEY (clés live_)
- ☐ Configurer le webhook Stripe et copier STRIPE_WEBHOOK_SECRET
- ☐ Activer MONETIZATION_SHOP=true et/ou MONETIZATION_SUBSCRIPTION=true
- ☐ Créer les produits en base via POST /api/admin/shop/products
- ☐ Placer les fichiers dans le volume /data/products
- ☐ Lancer la migration : alembic upgrade head
- ☐ Tester un paiement avec une carte test Stripe (4242 4242 4242 4242)
- ☐ Vérifier la réception de l'email de confirmation
- ☐ Vérifier le lien de téléchargement

La monétisation clôt la partie des modules optionnels — un projet en T2 peut désormais générer du revenu. La partie suivante détaille l'industrialisation : générateur, services partagés, fleet dashboard et cycle de vie complet d'un projet.

17 Le Générateur `create-gitsky-project`

17.1 Introduction

Le générateur est le composant qui transforme GitSky d'un template documenté en un template **opérationnel**. Sans lui, chaque nouveau projet exigerait un fork du repository, un renommage manuel de tous les identifiants, une réécriture de `.env`, une configuration Traefik ad hoc — soit plusieurs heures d'erreurs potentielles.

Avec le générateur, un `config.yaml` de 30 lignes suffit pour produire un projet démarrable en une commande.

Ce chapitre décrit le générateur, son fichier de configuration, son fonctionnement, et sa procédure de mise à jour des projets existants.

17.2 Choix Technique : Copier

Le générateur est bâti sur **Copier** — un moteur de scaffolding Jinja2 avec deux propriétés critiques pour GitSky :

1. **Génération initiale** à partir d'un template et d'un fichier de réponses.
2. **Mise à jour** d'un projet déjà généré lorsque le template évolue — indispensable pour propager un correctif de sécurité à 20 projets en production.

Cookiecutter, l'alternative populaire, ne fait que la génération initiale. Elle est inadaptée dès qu'on maintient une flotte.

Installation :

```
pip install copier
```

17.3 Le Fichier `config.yaml`

Chaque projet est décrit par un fichier YAML unique, versionné dans un repo central `startup-factory-configs/` :

```
# projects/pain-scraper.yaml
project:
  name: pain-scraper
  domain: pain-scraper.mystudio.com
  tier: t1 # t0 | t1 | t2

modules:
  agentic: true # override du profil t1
  monetization_subscription: true

data_models:
  - name: Company
    fields:
      name: str
      url: str
      pain_signal: text
      priority: int

domain_routes:
  - prefix: /api/pains
    handlers: pains.py

branding:
  primary_color: "#4F46E5"
  primary_foreground: "#FFFFFF"
  font_family: "Inter"
  logo: assets/pain-scraper-logo.svg

fleet:
  register: true # inscription au fleet dashboard
  stripe_account: mystudio_main # compte Stripe partagé
```

Les propriétés se répartissent en six catégories :

Catégorie	Rôle
project	Identité et tier
modules	Overrides des flags module par rapport au profil de tier
data_models	Modèles SQLAlchemy à scaffold dans app/domain/
domain_routes	Routeurs FastAPI à scaffold dans app/domain/
branding	Variables CSS et actifs à injecter dans le frontend
fleet	Enregistrement dans les services partagés de la flotte

17.4 Génération d'un Nouveau Projet

Une seule commande produit le projet :

```
copier copy \
  --data-file projects/pain-scraper.yaml \
  https://github.com/mystudio/gitsky-template \
  ~/projects/pain-scraper
```

Ce que le générateur fait, en 30 secondes :

1. **Clone le template** localement.
2. **Applique les substitutions Jinja2** dans tous les fichiers (nom du projet, tier, flags module, branding).
3. **Scaffold** `app/domain/` avec les modèles et routes déclarés dans `config.yaml`.
4. **Génère la migration Alembic initiale** pour le domaine et pour chaque module activé.
5. **Applique le branding** en réécrivant les variables CSS Tailwind et le logo.
6. **Génère les labels Traefik** pointant vers le domaine du projet dans le `docker-compose.yml`.
7. **Prépare la base PostgreSQL** du projet : génère des credentials aléatoires dans le `.env` ; le conteneur PostgreSQL du projet crée la base au premier démarrage (voir Chap 18 §2).
8. **Enregistre le projet** auprès du fleet dashboard via son API (voir Chap 19).
9. **Crée un commit git initial** dans le nouveau repo.

Le projet est ensuite démarrable en une commande :

```
cd ~/projects/pain-scraper
docker compose up -d
```

17.5 Logique Complexe : Context Hooks, Tasks et Migrations

Contrairement à Cookiecutter, Copier n'a pas de scripts `pre_gen` / `post_gen`. Il expose **trois mécanismes distincts**, que GitSky combine :

Mécanisme	Déclaration	Moment	Rôle dans GitSky
Context hook	<code>_jinja_extensions</code> + classe <code>ContextHook</code> (paquet <code>copier-template-extensions</code>)	Avant le rendu Jinja	Résout le profil de tier en flags <code>MODULE_*</code> , calcule les valeurs dérivées — sans multiplier les questions posées
<code>_tasks</code>	Liste de commandes dans <code>copier.yaml</code>	Après la génération des fichiers	Provisionne la DB, génère les migrations, enregistre au fleet dashboard, initialise le repo git
<code>_migrations</code>	Liste versionnée dans <code>copier.yaml</code>	Lors d'un <code>copier</code> <code>update</code>	Applique les nouvelles migrations sans casser les données existantes

Le **context hook** est l'équivalent d'un « pré-traitement » : il enrichit le contexte Jinja avant que les fichiers ne soient rendus. C'est là que le tier est résolu, avec **exactement la même logique que le runtime** (source unique de vérité) :

```
# extensions/context.py
from copier_template_extensions import ContextHook

class TierResolver(ContextHook):
    def hook(self, context: dict) -> None:
        tier = context.get("gitsky_tier", "t0")
        context["resolved_modules"] = resolve_profile(tier)
```

Déclaration dans `copier.yml` :

```
_jinja_extensions:
- copier_template_extensions.TemplateExtensionLoader
- extensions/context.py:TierResolver

_tasks:
- "python .copier/tasks/provision_db.py"
- "python .copier/tasks/register_fleet.py"
- "git init && git add -A && git commit -m 'Initial commit'"
```

Comme les context hooks et les `_tasks` exécutent du code, `copier copy` et `copier update` exigent le drapeau `--trust` (`unsafe=True` via l’API Python). Ces trois mécanismes sont ce qui distingue un scaffolder statique d’un vrai générateur de projet.

17.6 Mise à Jour d’un Projet Existant

Quand le template GitSky évolue (nouvelle version, correctif de sécurité, nouveau module), les projets existants peuvent recevoir la mise à jour :

```
cd ~/projects/pain-scraper
copier update
```

Copier applique un **diff à trois voies** :

- État actuel du projet.
- État du template au moment de la génération initiale.
- Nouvel état du template.

Les modifications spécifiques au projet (code métier, contenu) sont préservées. Les modifications côté template (nouveaux fichiers, correctifs) sont appliquées. Les conflits sont marqués pour résolution manuelle — Copier ne casse rien silencieusement.

Recommandation opérationnelle : exécuter `copier update` sur tous les projets de la flotte lors d’un correctif de sécurité critique. Le fleet dashboard (Chap 19) fournit une vue “template version” par projet pour identifier ceux qui sont à jour.

17.7 Bootstrapping d’une Flotte : Zero-to-N Projets

Le générateur permet une procédure de démarrage d’une flotte en trois commandes :


```
# 1. Prépare les services partagés du VPS (une seule fois)
./scripts/bootstrap-fleet.sh

# 2. Pour chaque idée à tester, crée un YAML et génère le projet
copier copy --data-file projects/idea-1.yaml <template> ~/projects/idea-1
copier copy --data-file projects/idea-2.yaml <template> ~/projects/idea-2
# ... 30 projets en 30 minutes

# 3. Démarre tous les projets
for dir in ~/projects/*/; do
  (cd "$dir" && docker compose up -d)
done
```

Le passage manuel à 30 projets serait impraticable. Le générateur rend cette échelle atteignable.

17.8 Anti-Patterns à Éviter

- **Éditer les fichiers générés à la main sans remonter la modification dans le template.** Toute correction utile à plusieurs projets doit remonter au template pour propagation via `copier update`.
- **Sauter le fleet register.** Un projet non enregistré n'apparaît pas dans le dashboard, ne bénéficie ni du kill mechanism ni des alertes.
- **Cloner un projet existant plutôt que le générer.** Le clone hérite des dérives et empêche la mise à jour propre du template.

Le générateur produit les projets ; les services partagés (Chap 18) leur donnent leurs points de convergence.

18 Services Partagés sur un Seul VPS

18.1 Introduction

Une flotte GitSky repose sur plusieurs **services partagés** qui vivent au niveau du VPS et sont consommés par tous les projets. Sans cette mutualisation, chaque projet dupliquerait ces composants — chaque instance porterait sa propre base MaxMind, sa propre clé API LLM, son propre relais SMTP — ce qui rendrait la flotte économiquement et opérationnellement ingérable.

Ce chapitre décrit chacun de ces services : rôle, implémentation recommandée, contrat d'interface, configuration.

18.2 Vue d'Ensemble

VPS mutualisé – services partagés	
[Traefik]	HTTPS wildcard + routing
[PostgreSQL]	données des services partagés
[Landing Collector]	Collecte des leads T0 partagée
[LLM Proxy]	Clés API + quotas
[GeoIP]	Résolution IP → pays/ville
[SMTP Relay]	Emails transactionnels
[Stripe Webhook Router]	Routage des events vers le bon projet
↓ Consommés par tous les projets ↓	
Projet A (T0)	Projet B (T1) Projet C (T2) ...

Tous les services partagés vivent sur le réseau interne `shared-services-net` du VPS, jamais exposé à Internet. Seul Traefik est joignable depuis l'extérieur.

18.3 1. Traefik : Routage HTTPS Mutualisé

Traefik est déjà décrit au Chap 1 comme le reverse proxy du template. Au niveau flotte, son rôle s'étend :

- Un unique certificat wildcard `*.mystudio.com` via Let's Encrypt DNS-01.
- Auto-découverte des projets qui rejoignent le réseau `proxy-net`.
- Middleware `RateLimit` par défaut (100 req/s par IP) pour tous les projets.
- Middleware `IPWhitelist` négatif alimenté par le job d'analyse des `SecurityEvent` (voir Chap 14).

```
# shared-services/traefik/docker-compose.yml
services:
  traefik:
    image: traefik:v3.1
    command:
      - --providers.docker=true
      - --providers.docker.network=proxy-net
      - --entrypoints.web.address=:80
      - --entrypoints.websecure.address=:443
      - --certificatesresolvers.letsencrypt.acme.dnsChallenge=true
      - --certificatesresolvers.letsencrypt.acme.dnsChallenge.provider=cloudflare
    ports:
      - "80:80"
      - "443:443"
    networks:
      - proxy-net
```

18.4 2. PostgreSQL : un conteneur par projet

Chaque projet doté d'une base embarque **son propre conteneur PostgreSQL**, défini dans son `docker-compose.yml` (Chap 1), plutôt qu'une base parmi d'autres dans une instance partagée. Cette isolation par conteneur permet :

- La restauration ou le kill d'un projet sans impacter les autres — on détruit un conteneur et son volume, rien d'autre.
- Des sauvegardes indépendantes, un dump par conteneur (Chap 23).
- Une révocation nette : supprimer le conteneur coupe tout accès, plus radicalement qu'un `REVOKE`.
- **Une isolation CPU/I-O réelle** : une requête lourde ou un autovacuum emballé sur un projet ne dégrade pas les autres. On ne peut pas plafonner le CPU d'une *base* au sein d'une instance ; on le peut pour un *conteneur*.
- **Un plafond de connexions par projet** : chaque conteneur dispose de ses ~100 connexions, sans se disputer un pool commun.

Le prix est de ~52 Mo de RAM par conteneur PostgreSQL au repos (mesuré, `postgres:16.3-alpine`). Sur une flotte réaliste — une dizaine de T1/T2 dotés d'une base — cela reste sous 1 Go, largement dans le budget du VPS (voir le tableau des coûts). **Les To n'ont pas de base propre** (§3) et ne coûtent rien de ce côté.

Note d'architecture. Une première version du template mutualisait une seule instance PostgreSQL avec une base par projet. Elle économisait ~500 Mo de RAM sur le VPS de 8 Go, mais au prix d'un couplage fort : une requête pathologique ou un autovacuum en retard sur un projet T1 non validé pouvait dégrader les T2 qui, eux, génèrent du revenu. Sur une *startup factory* où le code To/T1 est expérimental par construction, concentrer des codebases non éprouvées dans la même instance que les projets rentables était le mauvais compromis. Le conteneur par projet a donc été retenu — les trois bénéfices ci-dessus (restauration, sauvegarde, révocation) y sont d'ailleurs *plus* vrais qu'avec une base partagée.

La création de la base est prise en charge par Compose au premier démarrage (variables `POSTGRES_DB` / `POSTGRES_USER` / `POSTGRES_PASSWORD` du `.env`). Le générateur produit ces credentials — un mot de passe aléatoire par projet — et les injecte dans le `.env` :

```
# À la génération (create-gitsky-project) : credentials uniques par projet.  
POSTGRES_PASSWORD=$(python -c "import secrets; print(secrets.token_hex(24))")
```

Une instance PostgreSQL partagée subsiste néanmoins dans les services partagés, mais **uniquement pour les données des services eux-mêmes** : la table centrale du landing collector (§3) et les logs du LLM proxy. Elle n'héberge aucune base de projet.

18.5 3. Landing Collector

Pour le tier To, chaque projet est une simple landing sans base de données propre. Les captures d'emails doivent quand même être collectées quelque part. Le **landing collector** est un service partagé unique qui reçoit les formulaires via une API commune et les persiste dans une table centrale.

```
# shared-services/landing-collector/main.py - extrait  
@app.post("/leads")  
async def collect_lead(lead: LeadIn):  
    async with get_pool().acquire() as conn:  
        await conn.execute(  
            "INSERT INTO leads (project, email, source, utm_campaign, created_at) "  
            "VALUES ($1, $2, $3, $4, now())",  
            lead.project, lead.email, lead.source, lead.utm_campaign,  
        )  
    return {"ok": True}
```

Une To poste sur cet endpoint :

```
await fetch("https://landing-collector.mystudio.internal/leads", {  
    method: "POST",  
    body: JSON.stringify({ project: "pain-scraper", email }),  
});
```

Le fleet dashboard (Chap 19) lit cette table pour afficher le funnel de chaque projet To. La **capture** (`POST /leads`) reste publique — les landings postent sans secret — mais la **lecture** des stats (`GET /leads/{project}/stats`) est réservée au dashboard : elle exige un en-tête `X-Collector-Token` comparé à `COLLECTOR_STATS_TOKEN`. Sinon, le funnel (nombre d'inscrits) de n'importe quel projet fuiterait à quiconque joint le collecteur. Comme partout dans le châssis : ouvert en développement sans token, 503 fail-closed en production non configurée.

18.6 4. LLM Proxy Partagé

Décrit au Chap 15 (agentic) et au Chap 5 (clients partagés), le LLM proxy est le point de convergence de tous les appels IA de la flotte. Implémentation recommandée : **LiteLLM**.

```
# shared-services/llm-proxy/config.yaml
model_list:
  - model_name: claude-sonnet-4-6
    litellm_params:
      model: anthropic/claude-sonnet-4-6
      api_key: os.environ/ANTHROPIC_API_KEY
  - model_name: claude-opus-4-7
    litellm_params:
      model: anthropic/claude-opus-4-7
      api_key: os.environ/ANTHROPIC_API_KEY

general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: postgresql://litellm:pwd@postgres/litellm_logs

litellm_settings:
  set_verbose: false
```

Chaque projet reçoit son propre token d'authentification, avec quota configurable :

```
# Créer un token de projet avec quota
curl -X POST https://llm-proxy.internal/key/generate \
  -H "Authorization: Bearer $LITELLM_MASTER_KEY" \
  -d '{"key_alias": "pain-scraper", "max_budget": 20.00, "budget_duration": "30d"}'
```

Les logs consolidés servent au fleet dashboard pour afficher le coût LLM par projet.

18.7 5. Relais SMTP

Un unique compte SMTP (par exemple **Resend**, **Postmark**, ou un serveur Postfix maison) sert tous les projets. Les emails transactionnels — activation de compte, invitation waitlist, notifications de kill, reçus Stripe — passent tous par ce relais.

Chaque projet est identifié par un expéditeur (`no-reply@pain-scraper.mystudio.com`), et le contenu est rendu depuis les templates Jinja du projet lui-même.

18.8 6. Service GeoIP

Un unique service qui expose la résolution IP → géolocalisation via une API interne :

```
# shared-services/geoip/docker-compose.yml
services:
  geoip:
    image: mystudio/geoip-service:latest
    volumes:
      - geoip_data:/data/GeoLite2 # Base MaxMind, mise à jour mensuelle par cron
    networks:
      - shared-services-net
```

Les modules `analytics` (Chap 13) et `security` (Chap 14) des projets consomment ce service via `geoip_client.py`.

18.9 7. Router de Webhooks Stripe

Décrit au Chap 16, un unique endpoint public reçoit tous les webhooks Stripe de la flotte et les route vers le projet cible via la métadonnée `project_name`.

```
# shared-services/stripe-webhook-router/main.py - extrait
@app.post("/api/shop/webhook")
async def route_webhook(request: Request):
    payload = await request.body()
    event = stripe.Webhook.construct_event(
        payload, request.headers["stripe-signature"], WEBHOOK_SECRET
    )
    project = event.data.object.metadata.get("project_name")
    if not project:
        raise HTTPException(400, "Missing project_name metadata")
    await forward_to_project(project, event)
```

18.10 Sécurité des Services Partagés

Trois règles à respecter absolument :

1. **Aucun service partagé n'est directement exposé sur Internet** — seul Traefik voit le trafic externe.
2. **Les credentials sont stockés hors du VCS** (fichier `.secrets/` ignoré par git, ou vault type Bitwarden Secrets).
3. **Un compromis d'un projet ne compromet jamais les autres** — chaque projet a ses propres credentials LLM, SMTP, DB, et ne peut pas emprunter ceux d'un autre.

18.11 Coût Total des Services Partagés

Sur le VPS 8 Go à 20 €/mois, les services partagés consomment :

Service	RAM	Disque
Traefik	~50 Mo	~100 Mo
PostgreSQL (données des services partagés)	~200 Mo	~2 Go (logs LLM + leads)
Landing Collector	~50 Mo	négligeable
LLM Proxy (LiteLLM)	~150 Mo	~200 Mo (logs)
SMTP Relay (Postfix local)	~80 Mo	~100 Mo
GeoIP Service	~100 Mo	~200 Mo (base MaxMind)
Stripe Webhook Router	~40 Mo	négligeable
Total	~670 Mo	~2,6 Go

Il reste donc ~7,3 Go de RAM pour les projets applicatifs. À cela s'ajoutent les conteneurs PostgreSQL **par projet** (§2) : ~52 Mo au repos chacun (mesuré, `postgres:16.3-alpine`), uniquement pour les T1/T2 — les To n'ont pas de base. Une flotte de 10 T1 + 5 T2 ajoute ainsi ~780 Mo, laissant de quoi porter la centaine de To mentionnée au Chap 2 (les To ne consomment que leur backend).

Les services partagés donnent aux projets leurs points de convergence. Le fleet dashboard (Chap 19) donne à l'opérateur la vue unifiée qui rend l'ensemble pilotable.

19 Fleet Dashboard : la Vue Unifiée de la Flotte

19.1 Introduction

Le fleet dashboard est **l'app séparée** qui donne à l'opérateur la vue unifiée de tous les projets de sa flotte : leur tier, leur stage funnel, leurs métriques de traction, leur coût cumulé, leur candidat au kill mechanism.

Sans ce dashboard, opérer une flotte de 30 projets exige de se souvenir de chaque projet individuellement, de consulter des logs éparpillés, et de rater les signaux faibles qui doivent déclencher une promotion ou un kill. Avec le dashboard, tout tient sur une page.

Le fleet dashboard est déployé sur le domaine premier `mystudio.com` (contrairement aux projets qui vivent sur `*.mystudio.com`) et n'est accessible qu'à l'opérateur de la flotte.

19.2 Architecture

Le fleet dashboard est lui-même un projet GitSky T2 avec des modules spéciaux :

- `MODULE_AUTH=true` — un unique compte admin (l'opérateur).
- `MODULE_ADMIN=true` — l'ensemble des surfaces est admin.
- `MODULE_FLEET=true` — module dédié qui agrège les données de tous les projets.

Le module `fleet` interroge :

- La base du **landing collector** pour les métriques TO.
- Les **APIs de santé** de chaque projet pour connaître leur état.
- Les **logs du LLM proxy** pour le coût IA par projet.
- Le **compte Stripe partagé** pour le revenu par projet.
- Le **service Docker** local pour l'état des conteneurs.

Aucune donnée n'est dupliquée dans la base du fleet dashboard — il consulte les sources en temps réel et agrège en mémoire (cache court côté serveur).

19.3 Vue Principale : la Grille de Projets

L'écran d'accueil est une grille où chaque ligne représente un projet :

Projet	Tier	Sous-domaine	Signups (14j)	Conversion	RAM	Coût 30j	Signal
pain-scraper	To	pain-scraper.mystudio.com	47	4,2 %	55 Mo	12 €	✔ promotable
code-reviewer-pro	T2	code-reviewer-pro.com	—	—	890 Mo	340 €	—
gitsky-app	T2	gitsky.mystudio.com	—	—	720 Mo	180 €	—
dead-idea-42	To	dead-idea-42.mystudio.com	3	0,4 %	45 Mo	89 €	! pending kill
... 26 autres projets							

Les colonnes sont triables. Un filtre en tête de tableau permet d'isoler par tier, par statut, ou par candidat au kill.

19.4 Onglets Détaillés par Projet

Un clic sur un projet ouvre une vue détaillée à onglets.

19.4.1 Funnel

Reprend les critères de promotion du Chap 2 sous forme de jauges :

- Visites cumulées, avec objectif ≥ 500 pour la mesure de conversion.
- Signups, avec seuil de promotion $T_0 \rightarrow T_1$ à ≥ 30 .
- Retours qualitatifs (extraits de la table `feedbacks` du landing collector).
- Barre visuelle “J+21 restants” avec compte à rebours avant kill.

19.4.2 Métriques

Timelines `recharts` :

- Signups par jour.
- Visites par jour.
- Coût cumulé (traffic + infra + LLM).
- Rétention D7 (à partir du T_1).

19.4.3 Logs et Événements

Agrégation des dernières entrées `SecurityEvent` , appels LLM significatifs, webhooks Stripe reçus, alertes du cron `kill_check` .

19.4.4 Actions

Actions manuelles disponibles à l'opérateur :

- **Promouvoir de tier** (avec confirmation) : redéploie le projet avec le nouveau `.env` .
- **Kill maintenant** : lance la procédure du Chap 2, section Kill Mechanism.
- **Sauvegarder maintenant** : force un dump manuel de la base.
- **Voir les logs live** : ouvre un flux `docker logs` streamé.

19.5 Sources de Données du Dashboard

Aucune métrique du fleet dashboard n'est calculée sur des données stockées localement — tout provient d'agrégations en direct des sources autoritatives. Voici où chaque métrique est puisée :

Métrique	Source	Méthode d'accès	Latence typique
Signups To (14 j)	Table <code>leads</code> du landing collector	SELECT count avec filtre <code>project</code> + <code>created_at</code>	< 100 ms
Conversion To	Table <code>leads</code> + count visites landing	Ratio calculé à la demande	< 200 ms
Visites 30 j	Table <code>visits</code> (module analytics) OU landing collector	Routage selon <code>MODULE_ANALYTICS</code> actif	< 200 ms
Rétention D7	Table <code>visits</code> avec <code>ip_hash</code> + <code>user_id</code>	Cohortes calculées par projet	1-3 s
RAM par projet	<code>docker stats --no-stream --format</code>	Exec Docker CLI	< 500 ms
Coût 30 j — trafic	Aucune source automatique	Saisie manuelle par l'opérateur	—
Coût 30 j — infra	Amortissement VPS × part RAM du projet	Calcul (<code>VPS_MONTHLY</code> / <code>total_ram</code> × <code>project_ram</code>)	< 50 ms
Coût 30 j — LLM	Table <code>spend_logs</code> du LLM proxy (LiteLLM)	SELECT sum avec filtre <code>key_alias</code>	< 200 ms
Coût 30 j — Stripe fees	Table <code>purchases</code> + taux Stripe standard	Calcul dérivé	< 200 ms
MRR	Table <code>subscriptions</code> + <code>Product.price_cents</code>	SUM des abonnements actifs	< 300 ms
Signal kill	Composite des 3 signaux du Chap 2 + delta J+21	Calcul dérivé, cache 1 h	< 500 ms
Dernière sauvegarde OK	Fichier <code>/backups/last-success.log</code> par projet	Lecture disque	< 100 ms
État Docker	<code>docker ps --format</code>	Exec Docker CLI	< 300 ms
Événements sécurité	Tables <code>security_events</code> de tous les projets	Requête cross-DB	1-2 s

Le principe : pas de duplication. Le fleet dashboard n'a pas sa propre table de métriques. Il consulte les sources en temps réel avec un cache Redis court (30 s à 5 min selon la volatilité). Cela garantit qu'un projet supprimé disparaît immédiatement du dashboard sans opération de nettoyage manuelle.

Deux exceptions à cette règle :

- La table `fleet_lifecycle_events` (voir Chap 20) est propriété du dashboard — c'est le seul journal transversal persistant.

- La table `fleet_manual_costs` accueille les saisies opérateur pour trafic payant (Google Ads, Meta Ads...) qu’aucun système n’expose de manière programmable par projet.

19.5.1 Pourquoi le Coût Trafic est Manuel

Aucun réseau publicitaire (Google Ads, Meta Ads, Reddit Ads...) n’expose de facturation aisément programmable par projet. L’opérateur saisit les dépenses hebdomadaires par projet dans un formulaire léger. Automatiser cela demanderait des intégrations fragiles pour un gain marginal — c’est un choix d’architecture délibéré.

19.6 Les Alertes Automatiques

Le module `fleet` exécute plusieurs crons qui alimentent une file d’alertes :

Alerte	Déclencheur	Sévérité
<code>pending_kill</code>	J-2 avant expiration To sans signal	medium
<code>budget_exceeded</code>	Coût cumulé au-delà du plafond du tier	high
<code>security_high</code>	≥ 10 événements <code>critical</code> en 24h sur un projet	high
<code>deployment_failed</code>	Un projet ne répond plus à <code>/health</code> depuis 5 min	critical
<code>llm_quota_hit</code>	Un projet atteint 90 % de son quota LLM mensuel	medium
<code>template_outdated</code>	Un correctif de template n’a pas été propagé après 30 j	low

Ces alertes sont visibles dans une barre latérale permanente. Elles ne génèrent **pas d’emails individuels** — l’opérateur consulte le dashboard à un rythme quotidien fixe (recommandation : matin, 10 min).

19.7 Métriques Agrégées de Flotte

Un onglet “Overview” affiche les KPI de la flotte entière :

- **Taux de survie global** : pourcentage de projets qui montent d’un tier.
- **Coût mensuel total** (infra + LLM + Stripe fees).
- **Revenu mensuel total** (MRR + achats one-off).
- **Marge nette** de la flotte.
- **Empreinte RAM cumulée** vs capacité du VPS.

Ces KPI permettent de décider si la flotte doit être élargie (racheter des idées à harvester), consolidée (kill des projets zombies), ou faire l'objet d'un scale-up (VPS plus gros).

19.8 Le Fleet Dashboard comme Contrat

Un projet n'est pas "vivant" dans la flotte tant qu'il n'est pas enregistré dans le fleet dashboard. Le générateur `create-gitsky-project` (Chap 17) inscrit automatiquement chaque nouveau projet lors de sa génération via un `POST /api/fleet/projects/register`.

Cette convention garantit qu'aucun projet ne peut exister « en dehors » du dashboard, échappant au kill mechanism ou à la comptabilité budgétaire.

Cet endpoint est protégé. `register` n'est pas une route utilisateur (le générateur n'a ni compte ni JWT) : elle est gardée par un **token partagé machine-à-machine**, exigé dans l'en-tête `X-Fleet-Token` et comparé à `FLEET_REGISTER_TOKEN` (comparaison à temps constant). Le générateur l'envoie depuis son environnement. Même sémantique que les autres stubs du châssis : ouvert en développement sans token, mais **fail-closed** en production — un token non configuré fait répondre `503`, jamais un register public. Sans ce garde-fou, n'importe qui pourrait créer ou **écraser** (tier, domaine) les projets de la flotte. La lecture des stats du landing collector (Chap 18) suit exactement le même modèle (`X-Collector-Token`).

19.9 Un Kill est un Événement du Dashboard, pas un Script Manuel

L'exécution du kill mechanism passe **toujours** par le dashboard, jamais par un `docker compose down` ad hoc — sinon l'archivage, la libération des ressources et la journalisation ne se font pas. Cette discipline est ce qui permet aux calculs de ROI de flotte de rester justes à long terme.

L'opérateur voit sa flotte, décide sur chaque projet. Le prochain chapitre formalise ces décisions dans le cycle de vie complet d'un projet, du harvest initial à l'archivage ou au succès.

20 Cycle de Vie d'un Projet dans la Flotte

20.1 Introduction

Ce chapitre formalise le cycle de vie complet d'un projet GitSky, depuis l'idée jusqu'à son archivage (kill) ou son émancipation (succès). Il rassemble sous une même chronologie les concepts distribués dans les chapitres précédents — tiers (Chap 2), création de module métier (Chap 11), kill mechanism (Chap 2), promotion de tier (Chap 2), fleet dashboard (Chap 19).

L'objectif est double : donner à l'opérateur un playbook clair, et documenter le contrat de la flotte pour tout collaborateur qui interviendrait sur un projet.

20.2 Vue d'Ensemble : Cinq Stages

Harvest (idée)	T0 Landing	T1 MVP	T2 SaaS	Émancip. ou Kill
0€ ~1 jour	~20€ 14-21j	~200€ 30-45j	~1000€ variable	variable final

Chaque stage a ses **entrées** (ce qu'on collecte), ses **livrables** (ce qu'on produit), ses **critères de sortie** (ce qui déclenche le passage au suivant), et son **coût plafonné** (au-delà, kill automatique).

20.3 Stage 0 — Harvest

Précède l'existence du projet dans GitSky.

Entrées : sources de signal (Reddit, G2 reviews, HackerNews threads, job boards).

Activités :

- Extraction de patterns de douleur sur une source (via LLM proxy, script custom).
- Clustering des signaux similaires en "idée".
- Scoring initial (volume de signal, prix des solutions adjacentes, gap concurrentiel).

Livrables : un `config.yaml` prêt pour le générateur, avec le nom du projet, le tier To, et le copy de landing rédigé dans la langue verbatim de la source.

Critère de sortie : l'idée passe le score minimum. Coût de ce stage : ~0 € hors temps opérateur.

20.4 Stage 1 — Tier To (Landing)

Le projet naît. Le générateur produit un To en < 5 minutes.

Entrées : `config.yaml` , actifs de branding.

Activités :

- Déploiement automatique via `copier copy && docker compose up -d` .
- Trafic dirigé vers la landing depuis la source d'origine du signal (post Reddit, ads ciblées, SEO sur les queries identifiées).
- Collecte des emails via le landing collector partagé.

Livrables :

- Une landing live à `<nom>.mystudio.com` .
- Une entrée dans le fleet dashboard.
- Des métriques de conversion suivies quotidiennement.

Critère de sortie (To → T1) : l'un des trois signaux du Chap 2 — conversion ≥ 3 % sur 500 visites, ou ≥ 30 signups, ou ≥ 3 retours qualitatifs — sur 21 jours.

Critère de kill To : aucun signal atteint à J+21 OU coût cumulé > 100 € en trafic et infra. Kill automatique.

20.5 Stage 2 — Tier T1 (MVP Lite)

Le projet a un signal de demande. Il devient un produit utilisable.

Activités opérateur :

- Éditer le `config.yaml` : `tier: t1` , activer les modules nécessaires (auth, éventuellement onboarding).
- Ajouter les `data_models` du domaine métier.
- Lancer `copier update` .
- Redéploier.
- Migrer les leads To vers `users` (rôle waitlist), envoyer les invitations.

Activités développeur :

- Implémenter la feature métier core dans `app/domain/` .
- Ajouter les composants React nécessaires.
- Rédiger le contenu d'accueil et la fonctionnalité minimale.

Livrables : MVP fonctionnel utilisable par les leads To devenus users.

Critère de sortie (T1 → T2) : ≥ 10 actifs (usage ≥ 3 fois par semaine), rétention D7 ≥ 30 % , ≥ 1 paiement réel OU ≥ 3 déclarations WTP explicites — voir Chap 2.

Critère de kill T1 : rétention D7 < 15 % à J+30 OU aucun signal WTP à J+45 OU coût cumulé > 500 €. Kill automatique.

20.6 Stage 3 — Tier T2 (SaaS Complet)

Le projet a de la traction. Il devient un produit commercialisable.

Activités opérateur :

- Éditer le `config.yaml` : `tier: t2`, activer `admin`, `i18n`, `monetization_*`.
- Configurer Stripe (produits ou plans d'abonnement) dans le compte partagé, avec la métadonnée `project_name` (voir Chap 16).
- Passer sur un domaine dédié (`<nom>.com`) si le projet mérite l'investissement DNS.
- Rédiger les traductions EN si le marché cible est international.

Activités développeur :

- Compléter les surfaces admin.
- Ajouter les modules métier avancés.
- Optimiser le SEO et la conversion.

Livrables : SaaS opérationnel avec revenus mesurés.

Critère de kill T2 : churn > 20 %/mois sur 3 mois consécutifs OU MRR < 100 € à J+90 — évaluation manuelle par l'opérateur, alerte au fleet dashboard.

20.7 Migration Sous-Domaine → Domaine Premier

Quand un projet monte en T2 et mérite son propre domaine (`pain-scraper.com` au lieu de `pain-scraper.mystudio.com`), la migration doit préserver le SEO déjà acquis et ne pas casser les liens partagés par des utilisateurs. Voici la procédure en quatre étapes.

20.7.1 Étape 1 — Provisionnement du Nouveau Domaine (J-7)

- Achat du domaine.
- Configuration DNS chez le registrar :
 - Enregistrement `A` pointant vers l'IP du VPS.
 - Enregistrements `MX` vers le SMTP relay partagé si le projet reçoit des emails.
 - Enregistrement `CAA` autorisant Let's Encrypt.
- Ajout des labels Traefik du projet pour accepter les **deux domaines simultanément**.
- Génération du certificat pour le nouveau domaine (Let's Encrypt HTTP-01 standard — pas DNS-01 puisque ce n'est pas un wildcard).

À ce stade, les deux URLs répondent — l'ancienne reste primaire, la nouvelle est en accueil.

20.7.2 Étape 2 — Préparation SEO (J-4)

- Email aux utilisateurs annonçant le nouveau domaine (motif : identité de marque forte).
- Mise à jour du `canonical` de toutes les pages pour pointer vers le **nouveau** domaine (via composant SEO du Chap 10).
- Mise à jour du `sitemap.xml` pour lister les URLs sous le nouveau domaine.
- Soumission du sitemap à Google Search Console pour le nouveau domaine, revendication de propriété.

Le `canonical` pointant vers le nouveau signale à Google la préférence sans encore casser l'ancien.

20.7.3 Étape 3 — Bascule Primaire (J-0)

- Inversion du `PROJECT_DOMAIN` dans le `.env` : le nouveau domaine devient primaire.
- L'ancien sous-domaine renvoie désormais une **redirection 301** vers le nouveau, gérée par un middleware Traefik :

```
labels:  
- "traefik.http.middlewares.redirect-old.redirectregex.regex=https://pain-  
  scraper.mystudio.com/(.*)"   
- "traefik.http.middlewares.redirect-old.redirectregex.replacement=https://pain-scraper.com/${1}"  
- "traefik.http.middlewares.redirect-old.redirectregex.permanent=true"
```

- Envoi d'un email de confirmation aux utilisateurs.

20.7.4 Étape 4 — Suivi (J+30)

- Vérification que Google indexe le nouveau domaine et déprécie l'ancien.
- Vérification du trafic organique : la baisse initiale (~30-40 %) doit se résorber en 6-8 semaines.
- L'ancien sous-domaine reste actif **en redirection au moins 12 mois** — jamais supprimé tant que des liens externes peuvent exister.

20.7.5 Anti-Patterns à Éviter

- **Suppression brutale du sous-domaine** — casse tous les liens externes accumulés, perte SEO définitive.
- **Absence de redirection 301** — Google traite le nouveau domaine comme un site vierge, perte de tout le PageRank.
- **Migration pendant une campagne active** — la baisse SEO temporaire tue le rendement des campagnes en cours.

20.7.6 Timing Recommandé

Idéal : juste après validation T1 → T2, **avant** le lancement d'une grosse acquisition (Product Hunt, article invité, campagne payante) qui ferait converger du trafic vers l'ancien domaine.

20.8 Stage 4 — Émancipation ou Consolidation

Un projet qui atteint une taille significative peut suivre plusieurs voies.

20.8.1 Option A — Rester dans la flotte

Le projet continue à bénéficier des services partagés. C'est le cas par défaut jusqu'à ce que le projet dépasse ~1 000 € de MRR.

20.8.2 Option B — Émancipation

Au-delà de ce seuil, l'opérateur peut décider de sortir le projet de la flotte partagée :

- Migration vers un compte Stripe propre (rattachement à une entité juridique dédiée).
- Migration vers un VPS dédié (ou cluster).
- Extraction de la base PostgreSQL du service partagé vers une base dédiée.
- Retrait des labels Traefik du proxy partagé.

Cette procédure est décrite dans le playbook `docs/emancipation.md` du repo `startup-factory-configs/`. Elle est réversible en cas de sur-anticipation.

20.8.3 Option C — Vente ou acqui-hire

Un projet à traction significative peut être vendu. La flotte étant conçue avec isolation stricte (une DB, un domaine, un compte Stripe metadata-namespacé), cette extraction est mécanique.

20.9 Logique d'Évaluation du `kill_check`

Le cron `kill_check` est le mécanisme le plus critique de la flotte — il exécute automatiquement les kills sur des projets sans surveillance humaine. Sa logique doit être auditable et son évaluation reproductible.

20.9.1 Fréquence et Fenêtre d'Exécution

- Exécution quotidienne à **03:00 UTC** (après backup à 02:00, pour préserver la dernière sauvegarde d'un projet sur le point d'être tué).
- Fenêtre d'évaluation glissante : 21 jours pour T0, 30 jours pour T1, 90 jours pour T2.
- Ancre temporelle : `first_deployed_at` du projet (colonne dans `fleet_lifecycle_events`).

20.9.2 Données Collectées par Tier

Pour chaque projet actif, le cron collecte :

To (Landing) :

Variable	Source	Description
signup_count	leads (landing collector)	Nombre de leads collectés depuis first_deployed_at
visit_count	leads + tracking landing	Visites totales
qualitative_feedback_count	leads.feedback	Réponses à l'email de suivi
total_cost	fleet_manual_costs + amortissement infra	Trafic + infra + LLM
days_since_deploy	now() - first_deployed_at	Jours écoulés

T1 (MVP Lite) :

Variable	Source	Description
active_users_last_7d	table sessions du projet	Users avec ≥ 3 sessions sur 7 j
retention_d7	cohortes users $\times$ visits	Cohorte à J+7 / cohorte initiale
paid_users_count	subscriptions + purchases	Abonnements actifs ou achats réels
wtp_declarations	feedbacks.wtp	Réponses positives à un email « would you pay for X »
total_cost	idem To étendu	Trafic + infra + LLM cumulés

T2 (SaaS Complet) :

Variable	Source	Description
mrr	subscriptions.status IN (active, trialing)	SUM Subscription.plan_price
churn_rate_3m	Delta subscriptions.cancelled / initial	Taux de churn cumulé 90 jours
days_below_mrr_threshold	Historique MRR quotidien	Jours consécutifs avec MRR < 100 €

20.9.3 Règles d'Évaluation

Chaque tier évalue un verdict parmi healthy | pending_kill | kill_now | manual_review.

To :

```
def evaluate_t0(m):
    if m.days_since_deploy < 19:
        return "healthy"
    if (m.signup_count >= 30
        or (m.visit_count >= 500 and m.conversion_rate >= 0.03)
        or m.qualitative_feedback_count >= 3):
        return "healthy" # promotion T1 recommandée à l'opérateur
    if m.days_since_deploy >= 21 or m.total_cost >= 100:
        return "kill_now"
    return "pending_kill" # zone J+19 à J+21 sans signal
```

T1 :

```
def evaluate_t1(m):
    if m.days_since_deploy < 30:
        return "healthy"
    if (m.retention_d7 >= 0.30
        and (m.paid_users_count >= 1 or m.wtp_declarations >= 3)
        and m.active_users_last_7d >= 10):
        return "healthy" # promotion T2 recommandée
    if m.retention_d7 < 0.15 and m.days_since_deploy >= 30:
        return "kill_now"
    if m.days_since_deploy >= 45 and m.wtp_declarations == 0:
        return "kill_now"
    if m.total_cost >= 500:
        return "kill_now"
    return "pending_kill"
```

T2 :

```
def evaluate_t2(m):
    if (m.churn_rate_3m > 0.20 and m.days_below_mrr_threshold >= 90):
        return "manual_review"
    if m.days_below_mrr_threshold >= 90 and m.mrr < 100:
        return "manual_review"
    return "healthy"
```

T2 ne se kill jamais automatiquement. Les projets à ce tier ont des utilisateurs payants et méritent une évaluation humaine avant tout retrait.

20.9.4 Pipeline Post-Verdict

Verdict	Action
healthy	Rien
pending_kill	Email opérateur + entrée dashboard, ré-évaluation à J+2
kill_now	Procédure de shutdown déclenchée (voir section suivante)
manual_review	Alerte permanente au dashboard, aucune action automatique

20.9.5 Idempotence et Sécurité

- Le cron est idempotent : ré-exécuter deux fois de suite ne double pas les kills (verrou pris sur `fleet_lifecycle_events`).
- Un kill est réversible pendant 90 jours (sauvegarde froide conservée).
- L'opérateur peut annuler un `pending_kill` depuis le dashboard sans avoir à modifier le code.
- Chaque évaluation est journalisée : reproductibilité des verdicts sur demande d'audit.

20.10 Kill Mechanism : le Détail Opérationnel

Le kill mechanism a été défini au Chap 2. Voici son implémentation opérationnelle par le cron `kill_check`.

```
# services/kill-check.sh - extrait
#!/bin/bash
# Lancé quotidiennement à 03:00 UTC par le fleet dashboard.

for project in $(curl -s $FLEET/api/projects?status=active | jq -r '[][.name]'); do
    metrics=$(curl -s $FLEET/api/projects/$project/metrics)
    verdict=$(echo "$metrics" | python3 evaluate_kill.py)

    case "$verdict" in
        "kill_now")
            curl -X POST $FLEET/api/projects/$project/kill
            ;;
        "pending_kill")
            curl -X POST $FLEET/api/projects/$project/mark-pending-kill
            curl -X POST $FLEET/api/notifications/alert \
                -d "project=$project&type=pending_kill"
            ;;
        "healthy")
            ;;
    esac
done
```

La procédure de kill elle-même, orchestrée par le fleet dashboard :

1. `docker compose down` dans le dossier du projet.
2. Retrait des labels Traefik (rechargement Traefik).
3. Suppression des enregistrements DNS pointant vers le sous-domaine.
4. Dump PostgreSQL compressé archivé sur stockage cold (S3 Glacier ou Backblaze B2 archive).
5. Suppression de la base du service PostgreSQL partagé.
6. Suppression du token LLM proxy du projet.
7. Journalisation dans le fleet dashboard : statut `killed`, date, tier atteint, coût total, raison.

Le sous-domaine est libéré 30 jours après le kill (garde contre le squatting immédiat par un autre projet).

20.11 Reconstitution d'un Projet Killed

Un projet tué peut être ressuscité si un signal tardif émerge — rare mais possible (par exemple un article viral cite la landing archivée) :

1. Extraire le dump PostgreSQL du stockage cold.
2. Régénérer le projet via `copier copy` avec le même `config.yaml` (versionné).
3. Restaurer la base.
4. Redéployer.

Cette procédure prend ~30 minutes. La conservation des dumps pendant 90 jours minimum rend cette option disponible.

20.12 Le Journal de la Flotte

Chaque événement du cycle de vie (naissance, promotion, kill, émancipation) est journalisé dans une table `fleet_lifecycle_events` interrogeable depuis le fleet dashboard. À échéance annuelle, ce journal permet de calculer :

- Taux de survie $T_0 \rightarrow T_1 \rightarrow T_2$.
- Coût moyen par projet à chaque stage.
- ROI global de la flotte.

Ces indicateurs sont la boucle de feedback qui améliore la phase harvest — si le taux de survie T_0 est nul, c'est le scoring initial qui est cassé, pas les projets.

Ce chapitre clôt la partie industrialisation. La dernière partie du livre couvre la production et la maintenance : configuration du serveur Ubuntu, sauvegardes de flotte et bonnes pratiques opérationnelles.

21 Dockerisation Avancée pour la Production

21.1 Introduction

Le passage du développement à la production nécessite une approche différente de Docker. En local nous privilégions le confort (rechargement à chaud, ports exposés) ; en production nous visons performance, sécurité et légèreté. Ce chapitre décortique les Dockerfiles optimisés du template GitSky, valides pour les trois tiers T0, T1 et T2.

21.2 Le Pattern Multi-Stage

Le “Multi-Stage build” est la technique de référence pour créer des images de production. Elle consiste à utiliser une image pour compiler ou construire l’application, puis à copier uniquement le résultat final dans une image de base très légère.

21.2.1 Backend (FastAPI)

L’image finale de GitSky ne contient pas les outils de compilation Python, seulement les dépendances installées et le code source.

- **Stage 1 (Builder)** : installation des dépendances via `pip install --user .`
- **Stage 2 (Production)** : récupération uniquement du dossier `.local` et du code de l’application.

21.2.2 Frontend (React/Vite)

Pour React, le gain est encore plus impressionnant :

- **Stages 1 & 2** : installation des modules npm et exécution de `npm run build`.
- **Stage 3 (Production)** : copie uniquement du dossier `dist` (quelques Mo de JS/CSS statique) et utilisation d’un serveur léger comme `serve` pour le mettre à disposition.

21.3 Sécurité des Conteneurs

Une règle d’or en production : **ne jamais lancer de conteneur en tant qu’utilisateur root.**

Dans nos Dockerfiles, nous créons systématiquement un utilisateur système (`appuser`) aux droits restreints. Si un attaquant parvient à exploiter une faille dans l’application, il est confiné dans le conteneur sans pouvoir impacter l’hôte.

```
# Exemple dans le Dockerfile
RUN groupadd -r appuser && useradd -r -g appuser appuser
USER appuser
```

Le code reste en lecture seule pour `appuser` . Le dossier `/app` appartient à `root` : `appuser` peut l'exécuter mais pas le réécrire — une faille applicative ne peut donc pas modifier le code en place. Attention au piège : `WORKDIR /app` crée le dossier en `root` , et `COPY --chown=appuser` ne change que les *fichiers copiés*, pas le dossier lui-même. Les écritures runtime légitimes (SQLite d'un To, uploads) vont donc dans un dossier `/data` dédié, seul emplacement rendu inscriptible pour `appuser` :

```
RUN mkdir -p /data && chown appuser:appuser /data
VOLUME /data
```

21.4 Serveur de Production : Gunicorn + Uvicorn

En développement, nous utilisons `uvicorn --reload` . En production, **Gunicorn** orchestre plusieurs “workers” Uvicorn. Cela permet de traiter plusieurs requêtes simultanément et de redémarrer automatiquement un worker s'il échoue.

```
gunicorn app.core.main:app -k uvicorn_worker.UvicornWorker --bind 0.0.0.0:8000
```

⚠ **Classe de worker.** On utilise le paquet `uvicorn-worker` (`uvicorn_worker.UvicornWorker`), et non l'ancien `uvicorn.workers.UvicornWorker` intégré à Uvicorn : ce dernier est **déprécié depuis Uvicorn 0.30** et émet un avertissement au démarrage.

Le nombre de workers ne figure pas dans le `CMD` . Le calibrage est par tier — 1 pour un To, 2 pour un T1, 4 pour un T2 sous charge normale — mais le figer au build produirait *une image par tier*, en contradiction avec la règle « un seul Dockerfile » ci-dessous. On passe donc le nombre par la variable `WEB_CONCURRENCY` , que Gunicorn lit nativement, injectée depuis le `.env` du projet. Promouvoir un T1 en T2 se résume alors à changer cette valeur et redéployer — sans rebuild.

21.5 Surveillance de l'État (Healthchecks)

Pour que Traefik sache si un conteneur est réellement prêt à recevoir du trafic, une instruction `HEALTHCHECK` interroge l'endpoint `/health` toutes les 30 secondes. On sonde avec **Python** (`urllib.request` , déjà présent dans l'image) plutôt qu'avec `curl` : cela évite une couche `apt-get` supplémentaire, allège l'image, et supprime une dépendance réseau au build. `urlopen` lève sur un `503` — le conteneur est alors marqué non sain, ce qui est exactement le comportement voulu quand la base est injoignable (Chap 23 §4.1).

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD ["python", "-c", "import urllib.request,sys; sys.exit(0 if
    urllib.request.urlopen('http://localhost:8000/health', timeout=4).status == 200 else 1)"]
```


21.6 Un Seul Dockerfile pour Trois Tiers

Le template GitSky n’a pas de Dockerfile par tier — **un seul Dockerfile production** est utilisé quel que soit le tier du projet. Ce sont les flags `MODULE_*` du `.env` qui décident, à l’exécution, quels routers, modèles et migrations sont chargés.

Trois avantages :

- **Un seul artefact à builder** et à stocker dans le registre — pas de multiplication d’images.
- **Passer de T1 à T2** se fait via un simple redéploiement avec un `.env` mis à jour, sans rebuild.
- **La revue de sécurité** se fait sur une seule image, pas trois.

L’empreinte disque de l’image reste identique quel que soit le tier — **~320 Mo pour le backend, ~260 Mo pour le frontend** (tailles mesurées). Le backend part de `python:3.12-slim` (~180 Mo) plus les dépendances installées ; le frontend de `node:24-alpine` (~230 Mo) plus `serve` et le `dist/` (le bundle statique lui-même ne pèse que quelques centaines de Ko, mais l’image qui le sert porte la base Node). L’empreinte **mémoire** au runtime, en revanche, varie fortement selon les modules activés — c’est cette variation qui permet de porter 100 To sur le même VPS.

21.7 Empreinte Mémoire par Tier (Mesurée)

Tier	Modules activés	RAM initiale	RAM sous charge légère
To	landing-collector uniquement	~50 Mo	~60 Mo
T1	Auth + security + analytics	~180 Mo	~250 Mo
T2	Tous les modules + agentic	~700 Mo	~900 Mo à 1 Go

Un T2 avec agentic actif consomme davantage que la somme des modules — le framework agentic charge des modèles et des tool registries qui ont leur propre empreinte.

Le pattern Docker prod est posé pour tous les tiers. Le prochain chapitre décrit la configuration du serveur Ubuntu 24.04 qui accueille l’ensemble de la flotte, du hardening SSH au bootstrap des services partagés.

22 Configuration du Serveur de Production

22.1 Ubuntu Server 24.04 — Sécurisation, Docker et Accès SSH

22.2 Prérequis et Contexte

Ce chapitre couvre la mise en place complète d'un serveur Ubuntu 24.04 fraîchement installé, depuis la première connexion jusqu'au déploiement de l'infrastructure Docker. Les opérations sont à effectuer dans l'ordre indiqué.

Ce dont tu as besoin avant de commencer :

- Un serveur Ubuntu 24.04 avec une IP publique
- PuTTY et PuTTYgen installés sur ton poste Windows
- Un accès root initial par mot de passe (fourni par ton hébergeur)

22.3 Première Connexion avec PuTTY

22.3.1 Installer PuTTY

Télécharge la suite complète depuis le site officiel :

<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>

Installe le package **MSI 64-bit** — il inclut PuTTY, PuTTYgen et Pageant.

22.3.2 Configurer et sauvegarder une session PuTTY

Ouvre PuTTY et configure ta session avant la première connexion — tu n'auras plus à ressaisir ces paramètres ensuite.

```

PuTTY Configuration
|
+-- Session
|   Host Name : 217.76.61.192      (l'IP de ton serveur)
|   Port      : 22
|   Connection type : SSH
|   Saved Sessions : "mon-serveur-prod"  <- donne un nom
|
+-- Connection
|   +-- Data
|       Auto-login username : root
|   +-- SSH
|       +-- Auth
|           +-- Credentials
|               Private key file : (laisser vide pour l'instant)
|
+-- Session
    -> bouton [Save]

```

Double-clique sur le nom de la session sauvegardée pour te connecter. PuTTY te demande le mot de passe root fourni par ton hébergeur.

22.4 Sécurisation SSH par Clé Publique

L'objectif est de remplacer l'authentification par mot de passe — vulnérable au bruteforce — par une authentification par clé cryptographique.

Règle absolue : ne désactiver le mot de passe SSH qu'après avoir vérifié que la connexion par clé fonctionne. Ne jamais fermer la session courante avant ce test.

22.4.1 Générer une Paire de Clés avec PuTTYgen

Ouvre **PuTTYgen** (installé avec PuTTY).

```

+-----+
|                PuTTYgen                |
| 1. Key type    : [ED25519]  <- sélectionner |
| 2. Cliquer [Generate] |
|    Bouger la souris dans la zone vide |
| 3. Key comment : monnom@monpc |
| 4. Key passphrase : [mot de passe fort] |
|    Confirm passphrase: [meme mot de passe] |
| 5. [Save private key] -> serveur_prod.ppk |
| NE PAS FERMER PUTTY GEN |
+-----+

```

ED25519 est l'algorithme recommandé en 2024 — plus court, plus rapide et plus sûr que RSA 2048.

La **passphrase** protège ta clé privée sur ton poste. Si quelqu'un vole ton fichier `.ppk`, il ne peut rien faire sans elle.

Sauvegarde le fichier `.ppk` dans `C:\Users\toi\.ssh\serveur_prod.ppk`. Fais immédiatement une copie de sauvegarde sur un support externe ou dans un gestionnaire de mots de passe (Bitwarden, 1Password).

22.4.2 Déposer la Clé Publique sur le Serveur

Dans ta session PuTTY ouverte (connecté par mot de passe), exécute :

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
nano ~/.ssh/authorized_keys
```

Dans PuTTYgen, copie le contenu de la zone “**Public key for pasting into OpenSSH authorized_keys**”. C’est la grande zone de texte en haut de PuTTYgen. Elle ressemble à :

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBx... monnom@monpc
```

Colle cette ligne dans nano, puis :

- Sauvegarder : `Ctrl+O` puis `Entree`
- Quitter : `Ctrl+X`

```
chmod 600 ~/.ssh/authorized_keys
```

22.4.3 Configurer PuTTY pour Utiliser la Clé

Dans PuTTY, charge ta session sauvegardée et modifie-la :

```
PuTTY Configuration
|
+-- Connection -> SSH -> Auth -> Credentials
|   Private key file for authentication :
|   [Browse...] -> selectionner serveur_prod.ppk
|
+-- Session
    -> [Save]    <- sauvegarder les modifications
```

22.4.4 Tester la Connexion par Clé

Sans fermer ta session actuelle, ouvre une nouvelle fenêtre PuTTY et double-clique sur ta session sauvegardée.

PuTTY doit te demander la **passphrase de ta clé** (pas le mot de passe du serveur). Entre-la — tu es connecté.

Si la connexion échoue, consulte la section FAQ en fin de chapitre avant de continuer.

22.4.5 Désactiver l'Authentification par Mot de Passe

Une fois la connexion par clé vérifiée et fonctionnelle :

```
nano /etc/ssh/sshd_config
```

Localise et modifie ces lignes (utilise `Ctrl+W` pour chercher dans nano) :

```
PasswordAuthentication no  
PermitRootLogin prohibit-password
```

`prohibit-password` autorise root à se connecter uniquement par clé — le mot de passe root devient inutilisable à distance, ce qui est plus sûr que de bloquer root complètement (tu pourrais avoir besoin d'accès en cas de problème avec un utilisateur secondaire).

```
systemctl restart ssh
```

Teste une dernière fois depuis une nouvelle fenêtre PuTTY. Les bots que fail2ban surveille ne peuvent désormais plus rien faire — il n'y a plus de mot de passe à bruteforcer.

22.5 Mise à Jour du Système

```
apt update && apt upgrade -y  
reboot
```

Le redémarrage applique les éventuels nouveaux noyaux installés lors de la mise à jour.
Reconnecte-toi après le redémarrage.

22.6 Installation des Paquets de Base

```
apt install -y \  
  curl \  
  git \  
  unzip \  
  ufw \  
  fail2ban \  
  htop
```

Paquet	Rôle
<code>curl</code>	Téléchargements HTTP depuis le terminal
<code>git</code>	Cloner les dépôts de tes projets
<code>unzip</code>	Décompression d'archives
<code>ufw</code>	Pare-feu simplifié
<code>fail2ban</code>	Protection contre le bruteforce
<code>htop</code>	Monitoring CPU/mémoire interactif

22.7 Configuration du Pare-feu UFW

```
ufw default deny incoming    # Bloquer tout le trafic entrant par défaut
ufw default allow outgoing   # Autoriser tout le trafic sortant

ufw allow ssh                # Port 22 — NE PAS OUBLIER
ufw allow 80/tcp              # HTTP (Traefik)
ufw allow 443/tcp             # HTTPS (Traefik)

ufw enable
```

Vérifie l'état :

```
ufw status verbose
```

```
+-----+
|                UFW STATUS                |
|-----+-----+-----+
| To          Action    From              |
| --          - - - - - - - - - - - - - - |
| 22/tcp      ALLOW     Anywhere            |
| 80/tcp      ALLOW     Anywhere            |
| 443/tcp     ALLOW     Anywhere            |
|-----+-----+-----+
| Ports NON ouverts (restes internes) :    |
| 5432 (PostgreSQL)                        |
| 6379 (Redis)                             |
| 8080 (Dashboard Traefik)                 |
|-----+-----+-----+
+-----+
```

Ne jamais ouvrir les ports 5432 et 6379. PostgreSQL et Redis sont accessibles uniquement via les réseaux internes Docker — ils ne doivent jamais être joignables depuis l'extérieur.

22.8 Configuration de Fail2ban

Fail2ban surveille les logs du serveur et banne automatiquement les IPs qui échouent trop souvent à s'authentifier.

```
nano /etc/fail2ban/jail.local
```

```
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 5
```

```
[sshd]
enabled = true
```

```
systemctl enable fail2ban
systemctl restart fail2ban
```

Vérifie que la jail SSH est active :

```
fail2ban-client status sshd
```

Tu verras les IPs déjà bannies — les bots scannent en permanence les serveurs exposés sur internet. C'est normal d'en voir plusieurs dès les premières minutes.

22.9 Installation de Docker

Ne pas utiliser `apt install docker.io` — c'est une version ancienne maintenue par Ubuntu. Utiliser le dépôt officiel Docker :

```
# Clé GPG officielle Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Ajout du dépôt officiel
echo \
"deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| tee /etc/apt/sources.list.d/docker.list > /dev/null

# Installation
apt update
apt install -y \
docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
```

Vérifie l'installation :

```
docker --version
docker compose version
```

Active Docker au démarrage :

```
systemctl enable docker
systemctl start docker
```

22.10 Structure des Répertoires

```
mkdir -p /opt/traefik/config
mkdir -p /backups
```

```
+-----+
| /opt/                                     |
| +-- traefik/                             |
| |   +-- docker-compose.yml               |
| |   +-- config/                         |
| |       +-- middlewares.yml              |
| |                                       |
| +-- projet-a/    <- git clone            |
| +-- projet-b/    <- git clone            |
|                                       |
| /backups/                                             |
| +-- projet-a/                                         |
| +-- projet-b/                                         |
+-----+
```

22.11 Déploiement du Traefik Partagé

Crée le fichier de configuration :

```
nano /opt/traefik/docker-compose.yml
```



```

services:
  traefik:
    image: traefik:v3.0.4
    container_name: traefik_shared
    restart: always
    command:
      - "--providers.docker=true"
      - "--providers.docker.exposedbydefault=false"
      - "--providers.docker.network=proxy-net"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      # Redirection HTTP -> HTTPS globale pour tous les projets
      - "--entrypoints.web.http.redirections.entrypoint.to=websecure"
      - "--entrypoints.web.http.redirections.entrypoint.scheme=https"
      - "--entrypoints.web.http.redirections.entrypoint.permanent=true"
      # Let's Encrypt
      - "--certificatesresolvers.letsencrypt.acme.httpchallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.httpchallenge.entrypoint=web"
      - "--certificatesresolvers.letsencrypt.acme.email=${ACME_EMAIL}"
      - "--certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json"
      - "--providers.file.directory=/etc/traefik/config"
      - "--providers.file.watch=true"
      - "--accesslog=true"
      - "--log.level=WARN"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - traefik_certs:/letsencrypt
      - ./config:/etc/traefik/config:ro
    networks:
      - proxy-net
    security_opt:
      - no-new-privileges:true

volumes:
  traefik_certs:

networks:
  proxy-net:
    name: proxy-net
    driver: bridge

```

Crée le fichier `.env` de Traefik :

```
nano /opt/traefik/.env
```

```
ACME_EMAIL=ton@email.com
```

Crée les middlewares partagés :

```
nano /opt/traefik/config/middlewares.yml
```

```

http:
  middlewares:
    security-headers:
      headers:
        frameDeny: true
        contentTypeNosniff: true
        browserXssFilter: true
        referrerPolicy: "strict-origin-when-cross-origin"
        forceSTSHeader: true
        stsSeconds: 31536000
        stsIncludeSubdomains: true

  rate-limit:
    ratelimit:
      average: 100
      burst: 50

```

Lance Traefik :

```

cd /opt/traefik
docker compose up -d
docker compose logs -f # Verifier qu'il démarre sans erreur

```

22.12 Récapitulatif de l'État du Serveur

Une fois toutes les étapes effectuées, voici ce que tu dois avoir :

ETAT DU SERVEUR	
Composant	Etat attendu
SSH par cle	Actif
Mot de passe SSH	Desactive
UFW	Actif (22,80,443)
Fail2ban	Actif, jail sshd OK
Docker Engine	Actif
Docker Compose	Disponible
Traefik shared	Conteneur running
proxy-net	Reseau cree

Commandes de vérification globale :

```

ufw status # Pare-feu
fail2ban-client status # Jails actives
docker ps # Conteneurs running
docker network ls # Reseaux Docker

```

22.13 FAQ

Q : PuTTY me demande toujours le mot de passe après avoir configuré la clé.

La clé n'est pas chargée dans la session. Dans PuTTY, charge ta session sauvegardée, va dans **Connection** → **SSH** → **Auth** → **Credentials**, clique sur **Browse** et sélectionne ton fichier `.ppk`. Puis remonte dans **Session** et clique sur **Save** avant de te connecter.

Q : PuTTY affiche "Access denied" en boucle.

Deux causes possibles :

- Le mot de passe SSH a déjà été désactivé sur le serveur et PuTTY n'utilise pas encore la clé. Configurer la clé dans PuTTY comme décrit ci-dessus.
 - La clé publique n'a pas été correctement copiée dans `authorized_keys`. Si tu as encore accès au serveur via la console de ton hébergeur (KVM, VNC, console web), vérifie le contenu du fichier :
`cat ~/.ssh/authorized_keys`
-

Q : J'ai perdu ma clé `.ppk`. Je n'arrive plus à me connecter.

La plupart des hébergeurs proposent un accès console d'urgence (KVM, VNC, ou console web dans le panel) qui ne passe pas par SSH. Connecte-toi via cette console, puis :

```
# Remettre temporairement l'auth par mot de passe
nano /etc/ssh/sshd_config
# Passer PasswordAuthentication a yes
systemctl restart ssh
```

Génère une nouvelle paire de clés, redépote la clé publique, puis redésactive le mot de passe.

Q : Fail2ban a banni ma propre IP.

```
fail2ban-client set sshd unbanip TON_IP
```

Pour éviter que ça se reproduise, ajoute ton IP dans la whitelist dans `/etc/fail2ban/jail.local` :

```
[DEFAULT]
ignoreip = 127.0.0.1/8 ::1 TON_IP_FIXE
```

Cela n'est utile que si ton FAI te fournit une IP fixe.

Q : Comment connaître ma propre IP publique pour la whitelist ?

```
curl ifconfig.me
```

Q : Traefik ne génère pas le certificat Let's Encrypt.

Trois causes fréquentes :

- Le DNS du domaine ne pointe pas encore vers l'IP du serveur. Let's Encrypt ne peut pas valider un domaine qui n'est pas résolu. Vérifie avec : `nslookup mondomaine.com`
- Le port 80 est bloqué par UFW. Let's Encrypt utilise le challenge HTTP sur le port 80 pour valider le domaine. Vérifier : `ufw status`
- Le fichier `acme.json` a des permissions incorrectes. Traefik le crée lui-même dans le volume — ne pas le toucher manuellement.

Q : Comment voir les domaines et certificats gérés par Traefik ?

```
# Inspecter les logs Traefik
docker logs traefik_shared 2>&1 | grep -i "acme\\|certificate\\|error"

# Lister les routes actives
docker exec traefik_shared traefik version

# Inspecter le fichier acme.json (certificats stockés)
docker exec traefik_shared cat /letsencrypt/acme.json | python3 -m json.tool
```

Q : Comment ajouter un nouveau projet sur le serveur ?

```
cd /opt
git clone https://github.com/toi/mon-projet.git
cd mon-projet
cp .env.example .env.prod
nano .env.prod    # Remplir les secrets
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Traefik détecte automatiquement les nouveaux conteneurs et configure le routing — aucune intervention sur Traefik lui-même.

Q : Un conteneur redémarre en boucle. Comment diagnostiquer ?

```
# Voir l'état de tous les conteneurs
docker ps -a

# Logs du conteneur en erreur
docker logs NOM_DU_CONTENEUR --tail=50

# Suivre les logs en temps réel
docker logs NOM_DU_CONTENEUR -f

# Inspecter la configuration du conteneur
docker inspect NOM_DU_CONTENEUR
```

Q : Le serveur est lent ou saturé. Comment identifier la cause ?

```
# Vue globale CPU/RAM/swap en temps réel
htop

# Consommation par conteneur Docker
docker stats

# Espace disque
df -h

# Les processus qui consomment le plus d'I/O disque
iotop -o
```

Q : Comment mettre à jour Docker ?

```
apt update
apt install --only-upgrade docker-ce docker-ce-cli containerd.io
```

Docker est rétrocompatible — la mise à jour du moteur n’affecte pas les conteneurs en cours d’exécution.

Q : Comment sauvegarder les certificats Traefik ?

Le volume `traefik_certs` contient le fichier `acme.json` avec tous les certificats. L’inclure dans la stratégie de sauvegarde :

```
docker run --rm \
  -v traefik_certs:/data \
  -v /backups:/backup \
  alpine tar czf /backup/traefik_certs_$(date +%Y%m%d).tar.gz -C /data .
```

En pratique, la perte des certificats n’est pas catastrophique — Traefik les renouvelle automatiquement via Let’s Encrypt. Le seul impact est quelques minutes sans HTTPS lors du renouvellement.

22.14 Bootstrap des Services Partagés pour une Flotte

Sur ce serveur Ubuntu durci, l'installation de la flotte GitSky consiste à déployer une fois pour toutes les services partagés décrits au Chap 18, puis à laisser le générateur (Chap 17) déployer les projets individuellement.

22.14.1 Le Script `bootstrap-fleet.sh`

Un unique script d'orchestration installe l'ensemble des services partagés :

```
# scripts/bootstrap-fleet.sh
#!/usr/bin/env bash
set -euo pipefail

echo "1/6 Création des réseaux Docker partagés..."
docker network create proxy-net || true
docker network create shared-services-net || true

echo "2/6 Démarrage de Traefik (wildcard SSL)..."
(cd shared-services/traefik && docker compose up -d)

echo "3/6 Démarrage de PostgreSQL partagé..."
(cd shared-services/postgres && docker compose up -d)

echo "4/6 Démarrage du landing-collector, GeoIP, SMTP relay..."
(cd shared-services/landing-collector && docker compose up -d)
(cd shared-services/geoip && docker compose up -d)
(cd shared-services/smtp-relay && docker compose up -d)

echo "5/6 Démarrage du LLM proxy..."
(cd shared-services/llm-proxy && docker compose up -d)

echo "6/6 Démarrage du fleet dashboard..."
(cd shared-services/fleet-dashboard && docker compose up -d)

echo "Flotte prête. Les projets peuvent être générés."
```

Ce script est idempotent — il peut être ré-exécuté pour redémarrer les services après un reboot.

22.14.2 Répertoire de Travail sur le Serveur

L'organisation recommandée du système de fichiers :

```
/opt/mystudio/
├── shared-services/           # Services partagés du VPS (versionné)
│   ├── traefik/
│   ├── postgres/
│   ├── llm-proxy/
│   └── ...
├── projects/                 # Un dossier par projet généré
│   ├── pain-scraper/
│   ├── code-reviewer-pro/
│   └── ...
├── configs/                  # Fichiers config.yaml (repo startup-factory-configs)
│   └── projects/
└── backups/                  # Voir Chap 23
```

Chaque dossier de projet est un clone du template GitSky avec les substitutions du générateur appliquées. Les projets **ne partagent pas de code** entre eux — seuls les services externes sont mutualisés.

22.14.3 fail2ban et Traefik

L'analyse des `SecurityEvent` de tous les projets (Chap 14) alimente une règle fail2ban qui pousse les IPs agressives dans une allowlist négative Traefik, transformant la détection applicative par projet en blocage réseau mutualisé — sans avoir à configurer fail2ban par projet.

22.14.4 Isolation par Projet : Ce qui est Vraiment Isolé

Trois niveaux d'isolation stricts :

Niveau	Isolation	Partage
Conteneurs applicatifs	1 backend + 1 frontend par projet	Le réseau <code>proxy-net</code> (Traefik uniquement)
Base de données	1 DB PostgreSQL par projet	Le processus PostgreSQL
Secrets	1 fichier <code>.env</code> par projet	Aucun

Un attaquant qui compromet un projet ne peut atteindre ni les autres projets ni les services partagés. Le seul chemin latéral serait via Traefik lui-même — d'où l'importance de le tenir à jour.

Le serveur est prêt à porter la flotte. Le dernier chapitre couvre la maintenance quotidienne et hebdomadaire nécessaire pour la garder en bonne santé sur la durée.

23 Maintenance en Production : Sécurité, Sauvegardes et Surveillance

23.1 Pourquoi la Maintenance est Non-Négociable

Déployer une application en production n'est pas la fin du travail — c'est le début d'une responsabilité. Une plateforme sans maintenance est comme une voiture sans vidange : elle fonctionne parfaitement au départ, puis se dégrade silencieusement jusqu'à la panne catastrophique.

Pour le projet GitSky, quatre menaces pèsent sur vos données et votre disponibilité :

MENACES EN PRODUCTION		
MENACE	EXEMPLE CONCRET	PROTECTION
Perte de données	Crash disque, DELETE accidentel	Sauvegardes automatiques
Corruption	Migration ratée, panne matérielle	Sauvegardes + tests de restauration
Exposition	Fuite de credentials, injection SQL	Rotation des secrets + durcissement DB
Dégradation	Disque plein, requêtes lentes	Monitoring + maintenance planifiée

Ce chapitre vous donne les outils, les scripts, et le calendrier pour maintenir votre plateforme en bonne santé durablement.

23.2 Partie 1 : Sauvegardes PostgreSQL

23.2.1 La Règle du 3-2-1

Toute stratégie de sauvegarde professionnelle respecte cette règle fondamentale :

- **3** copies de vos données
- **2** sur des supports ou emplacements différents
- **1** copie hors site (S3, Backblaze, autre serveur)

Pour GitSky, cela se traduit par : 1. La base de données active sur votre serveur (copie 1) 2. Les dumps compressés sur le même serveur dans `/backups/` (copie 2) 3. Les dumps synchronisés vers un stockage cloud (copie 3, hors site)

23.2.2 Types de Sauvegardes

Dump logique (`pg_dump`) : Exporte les données sous forme de SQL. Simple, portable, restaurable sur n'importe quelle instance PostgreSQL. Idéal pour les petites et moyennes bases (< 50 Go).

Sauvegarde physique (`pg_basebackup`) : Copie binaire des fichiers de données. Plus rapide pour les très grandes bases, nécessite la même version de PostgreSQL pour la restauration.

WAL Archiving (avancé) : Archive les journaux de transactions en continu, permettant une restauration à n'importe quel instant précis (Point-In-Time Recovery). Recommandé quand votre base contient des données financières critiques.

Pour GitSky, le **dump logique quotidien** est le point de départ adapté.

23.2.3 Script de Sauvegarde Automatisé

Créez le fichier `scripts/backup_db.sh` à la racine de votre projet :

```
#!/usr/bin/env bash
# =====
# scripts/backup_db.sh
# Sauvegarde automatisée de la base PostgreSQL avec rotation et alerte.
#
# Usage : ./scripts/backup_db.sh
# Variables d'environnement requises (lues depuis .env.backup) :
#   POSTGRES_CONTAINER - nom du conteneur Docker PostgreSQL
#   POSTGRES_DB         - nom de la base de données
#   POSTGRES_USER        - utilisateur PostgreSQL
#   BACKUP_DIR           - répertoire local de stockage
#   BACKUP_RETENTION     - nombre de jours de rétention (défaut: 14)
#   S3_BUCKET            - bucket S3 (optionnel)
#   ALERT_EMAIL          - email pour les alertes d'échec (optionnel)
# =====

set -euo pipefail

# — Configuration —————
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ENV_FILE="${SCRIPT_DIR}/../.env.backup"

if [[ -f "$ENV_FILE" ]]; then
    # shellcheck source=/dev/null
    source "$ENV_FILE"
fi

POSTGRES_CONTAINER="${POSTGRES_CONTAINER:-gitsky_db}"
POSTGRES_DB="${POSTGRES_DB:-gitsky}"
POSTGRES_USER="${POSTGRES_USER:-postgres}"
BACKUP_DIR="${BACKUP_DIR:-/backups/postgres}"
BACKUP_RETENTION="${BACKUP_RETENTION:-14}"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/backup_${POSTGRES_DB}_${TIMESTAMP}.sql.gz"
LOG_FILE="${BACKUP_DIR}/backup.log"

# — Fonctions utilitaires —————
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a "$LOG_FILE"
}

alert() {
    local message="$1"
    log "ERREUR: $message"
    if [[ -n "${ALERT_EMAIL:-}" ]]; then
        echo "$message" | mail -s "[GitSky] ALERTE: Échec sauvegarde DB" "$ALERT_EMAIL" 2>/dev/null || true
    fi
}

# — Vérifications préalables —————
# Créer le répertoire AVANT le premier `log` : la fonction log() écrit dans
# $BACKUP_DIR/backup.log via tee, et avec `set -o pipefail` un tee vers un
# dossier inexistant fait échouer la toute première sauvegarde.
mkdir -p "$BACKUP_DIR"

log "=== Démarrage de la sauvegarde ==="

# Vérifier que le conteneur tourne
if ! docker inspect "$POSTGRES_CONTAINER" --format='{{.State.Status}}' 2>/dev/null | grep -q
    "running"; then
    alert "Le conteneur $POSTGRES_CONTAINER n'est pas en cours d'exécution."
    exit 1
fi

# — Sauvegarde —————
```

```

log "Dump de la base '$POSTGRES_DB' vers $BACKUP_FILE..."

if docker exec "$POSTGRES_CONTAINER" \
  pg_dump -U "$POSTGRES_USER" --format=plain --no-owner --no-acl "$POSTGRES_DB" \
  | gzip -9 > "$BACKUP_FILE"; then

  SIZE=$(du -sh "$BACKUP_FILE" | cut -f1)
  log "Sauvegarde réussie : $BACKUP_FILE ($SIZE)"
else
  alert "pg_dump a échoué pour la base '$POSTGRES_DB'."
  rm -f "$BACKUP_FILE"
  exit 1
fi

# — Vérification d'intégrité —————
log "Vérification de l'intégrité du fichier..."
if ! gzip -t "$BACKUP_FILE"; then
  alert "Le fichier de sauvegarde est corrompu : $BACKUP_FILE"
  rm -f "$BACKUP_FILE"
  exit 1
fi
log "Intégrité OK."

# — Upload S3 (optionnel) —————
if [[ -n "${S3_BUCKET:-}" ]]; then
  log "Upload vers S3 : s3://${S3_BUCKET}/postgres-backups/..."
  if aws s3 cp "$BACKUP_FILE" "s3://${S3_BUCKET}/postgres-backups/" \
    --storage-class STANDARD_IA; then
    log "Upload S3 réussi."
  else
    alert "L'upload S3 a échoué. La sauvegarde locale est conservée."
    # On ne quitte pas en erreur : la sauvegarde locale est valide
  fi
fi

# — Rotation : suppression des anciennes sauvegardes —————
log "Suppression des sauvegardes de plus de ${BACKUP_RETENTION} jours..."
DELETED=$(find "$BACKUP_DIR" -name "backup_*.sql.gz" -mtime "+${BACKUP_RETENTION}" -print -delete |
  wc -l)
log "$DELETED ancien(s) fichier(s) supprimé(s)."

# — Résumé —————
TOTAL_BACKUPS=$(find "$BACKUP_DIR" -name "backup_*.sql.gz" | wc -l)
TOTAL_SIZE=$(du -sh "$BACKUP_DIR" 2>/dev/null | cut -f1)
log "=== Sauvegarde terminée. Fichiers conservés : $TOTAL_BACKUPS ($TOTAL_SIZE) ==="

```

Rendez-le exécutable :

```
chmod +x scripts/backup_db.sh
```

Créez le fichier de configuration `.env.backup` (ne jamais committer ce fichier). Sur une flotte GitSky, chaque projet a **son propre conteneur** `{projet}_db` et sa base au nom du projet (Chap 18 §2) : ces valeurs viennent donc de `.env.backup`, pré-rempli à la génération, jamais codées en dur. Le générateur produit d'ailleurs un `.env.backup.example` déjà renseigné pour le projet.

```
# .env.backup – configuration du script de sauvegarde (projet « pain-scrapers »)
POSTGRES_CONTAINER=pain-scrapers_db
POSTGRES_DB=pain_scrapers
POSTGRES_USER=pain_scrapers
BACKUP_DIR=/backups/postgres
BACKUP_RETENTION=14
# S3_BUCKET=mon-bucket-backups      # décommenter si S3 disponible
# ALERT_EMAIL=admin@mondomaine.com  # décommenter pour les alertes email
```

23.2.4 Automatisation avec Cron

Sur votre serveur Linux, éditez la crontab :

```
crontab -e
```

Ajoutez ces lignes :

```
# Sauvegarde quotidienne à 2h00 du matin
0 2 * * * /opt/gitisky/scripts/backup_db.sh >> /var/log/gitisky_backup.log 2>&1

# Vérification hebdomadaire que les sauvegardes existent bien
0 9 * * 1 /opt/gitisky/scripts/check_backups.sh >> /var/log/gitisky_backup.log 2>&1
```

23.2.5 Alternative : Service Docker dédié aux Sauvegardes

Si vous préférez tout gérer dans Docker Compose, ajoutez ce service dans `docker-compose.prod.yml` :

```
# À ajouter dans docker-compose.prod.yml

backup:
  image: postgres:16-alpine
  restart: "no" # Ne démarre pas automatiquement, lancé par cron
  environment:
    PGPASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - /backups/postgres:/backups
  networks:
    - internal
  command: >
    sh -c "pg_dump -h postgres -U ${POSTGRES_USER} ${POSTGRES_DB}
    | gzip > /backups/backup_$(date +%Y%m%d_%H%M%S).sql.gz
    && find /backups -name '*.sql.gz' -mtime +14 -delete
    && echo 'Backup done.'"
  depends_on:
    - postgres
  profiles:
    - backup # Lancé uniquement avec --profile backup
```

Cron pour lancer ce service :

```
0 2 * * * cd /opt/gitisky && docker compose -f docker-compose.prod.yml --profile backup run --rm backup
```

23.2.6 Tester la Restauration — Obligatoire

Une sauvegarde non testée n'est pas une sauvegarde. Chaque mois, vérifiez que vous pouvez restaurer :

```
#!/usr/bin/env bash
# scripts/test_restore.sh
# Restaure la dernière sauvegarde dans un conteneur de test et vérifie le contenu.

BACKUP_DIR="/backups/postgres"
LATEST=$(ls -t "$BACKUP_DIR"/backup_*.sql.gz | head -1)

if [[ -z "$LATEST" ]]; then
    echo "ERREUR : Aucune sauvegarde trouvée dans $BACKUP_DIR"
    exit 1
fi

echo "Test de restauration depuis : $LATEST"

# Démarrer un PostgreSQL de test
docker run -d --name pg_restore_test \
    -e POSTGRES_PASSWORD=test \
    -e POSTGRES_DB=gitsky_test \
    postgres:16-alpine

sleep 3

# Restaurer
gunzip -c "$LATEST" | docker exec -i pg_restore_test \
    psql -U postgres gitsky_test

# Vérifier que les tables principales existent
RESULT=$(docker exec pg_restore_test \
    psql -U postgres gitsky_test -t -c "
        SELECT COUNT(*) FROM information_schema.tables
        WHERE table_schema = 'public';
    ")

# Nettoyage
docker rm -f pg_restore_test

echo "Tables restaurées : $RESULT"
if [[ "$RESULT" -gt 0 ]]; then
    echo "✓ Test de restauration réussi."
else
    echo "✗ ERREUR : Aucune table trouvée après restauration !"
    exit 1
fi
```

23.3 Partie 2 : Durcissement de la Sécurité

23.3.1 2.1 Principe du Moindre Privilège pour PostgreSQL

Par défaut, votre application se connecte probablement en tant que superutilisateur `postgres`. C'est l'équivalent de faire tourner votre serveur web en tant que `root` : si quelqu'un exploite une faille dans votre API, il obtient un accès total à votre base de données.

La solution est de créer un utilisateur applicatif avec uniquement les droits nécessaires :

```
-- Connectez-vous à PostgreSQL en tant que superutilisateur
-- docker exec -it gitsky_db psql -U postgres

-- 1. Créer l'utilisateur applicatif
CREATE USER gitsky_app WITH PASSWORD 'mot_de_passe_fort_ici';

-- 2. Donner accès à la base uniquement
GRANT CONNECT ON DATABASE gitsky TO gitsky_app;
GRANT USAGE ON SCHEMA public TO gitsky_app;

-- 3. Donner les permissions CRUD (pas DROP, pas CREATE TABLE)
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO gitsky_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO gitsky_app;

-- 4. Appliquer ces droits aux futures tables (créées par les migrations)
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO gitsky_app;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO gitsky_app;

-- 5. L'utilisateur de migration (Alembic) doit garder des droits de DDL
-- Créer un utilisateur séparé pour les migrations
CREATE USER gitsky_migrate WITH PASSWORD 'autre_mot_de_passe_fort';
GRANT ALL PRIVILEGES ON DATABASE gitsky TO gitsky_migrate;
```

Mettez ensuite à jour votre `.env` :

```
# Connexion applicative (lecture/écriture uniquement) – driver async
DATABASE_URL=postgresql+asyncpg://gitsky_app:mot_de_passe_fort@postgres/gitsky

# Connexion migrations (droits DDL complets) – driver async (env.py Alembic async)
DATABASE_URL_MIGRATE=postgresql+asyncpg://gitsky_migrate:autre_mdp@postgres/gitsky
```

23.3.2 2.2 Calendrier de Rotation des Secrets

La rotation régulière des secrets limite la fenêtre d'exposition en cas de fuite silencieuse (credential stuffing, ancien développeur, log accidentel).

SECRET	FRÉQUENCE	COMMENT FAIRE
SECRET_KEY (JWT)	6 mois	Tous les tokens existants sont invalidés → re-login
POSTGRES_PASSWORD	6 mois	Changer dans Postgres + .env + redéployer
SMTP_PASSWORD	1 an	Via interface du fournisseur email
STRIPE_SECRET_KEY STRIPE_WEBHOOK_SECRET	Si compromis uniquement	Via Stripe Dashboard Révoquer l'ancien
Clés SSH serveur	1 an	Générer nouvelle paire + supprimer l'ancienne

Générer une nouvelle `SECRET_KEY` :

```
python -c "import secrets; print(secrets.token_hex(64))"
```

Procédure de rotation sans interruption pour `SECRET_KEY` :

```
# Dans config.py, vous pouvez temporairement accepter deux clés
# pendant une période de transition (ex: 24h) :

SECRET_KEY = "nouvelle_clé_principale"
SECRET_KEY_OLD = "ancienne_clé" # Retirer après 24h

# Dans le middleware JWT :
# Essayer de vérifier avec la nouvelle clé,
# si ça échoue, essayer avec l'ancienne.
# Ainsi les utilisateurs connectés ne sont pas déconnectés brutalement.
```

23.3.3 2.3 Vérification du Réseau : PostgreSQL ne doit Jamais être Exposé

PostgreSQL doit être strictement interne. Vérifiez avec ce script :

```
#!/usr/bin/env bash
# scripts/check_security.sh
# Audit rapide de la surface d'exposition réseau.

set -euo pipefail

echo "=== Audit de sécurité réseau ==="
ERRORS=0

# 1. PostgreSQL ne doit pas écouter sur 0.0.0.0
PG_BINDING=$(docker inspect gitsky_db \
  --format='{{range $p, $b := .NetworkSettings.Ports}}{{ $p}} -> {{ $b }}{{ end}}' 2>/dev/null || echo
  "")

if echo "$PG_BINDING" | grep -q "0.0.0.0"; then
  echo "x CRITIQUE : PostgreSQL est exposé publiquement ! ($PG_BINDING)"
  ERRORS=$((ERRORS + 1))
else
  echo "✓ PostgreSQL non exposé publiquement."
fi

# 2. Port 5432 ne doit pas être accessible depuis l'extérieur
if timeout 3 bash -c "echo > /dev/tcp/localhost/5432" 2>/dev/null; then
  echo "x AVERTISSEMENT : Port 5432 accessible en local. Vérifiez les règles firewall."
else
  echo "✓ Port 5432 non accessible depuis l'hôte."
fi

# 3. Vérifier que les fichiers .env ne sont pas dans git
if git -C "$(dirname "$0")/.." ls-files --error-unmatch .env 2>/dev/null; then
  echo "x CRITIQUE : Le fichier .env est tracké par git !"
  ERRORS=$((ERRORS + 1))
else
  echo "✓ .env non tracké par git."
fi

# 4. Vérifier les permissions des fichiers de secrets
for file in .env .env.backup .env.prod; do
  if [[ -f "$file" ]]; then
    PERMS=$(stat -c "%a" "$file")
    if [[ "$PERMS" != "600" ]] && [[ "$PERMS" != "400" ]]; then
      echo "x AVERTISSEMENT : $file a des permissions trop larges ($PERMS). Recommandé: 600"
      echo "  Corriger avec : chmod 600 $file"
    else
      echo "✓ Permissions de $file : OK ($PERMS)"
    fi
  fi
done

echo ""
if [[ $ERRORS -gt 0 ]]; then
  echo "=== $ERRORS problème(s) critique(s) détecté(s). Action immédiate requise. ==="
  exit 1
else
  echo "=== Audit terminé. Aucun problème critique. ==="
fi
```

23.3.4 2.4 Scan des Vulnérabilités dans les Dépendances

Les bibliothèques tierces introduisent régulièrement des CVE (failles connues). Scannez-les mensuellement :


```
# Backend Python
pip install pip-audit
pip-audit -r backend/requirements.txt

# Frontend Node.js
cd frontend && npm audit

# Pour un rapport détaillé avec les corrections suggérées
npm audit fix --dry-run
```

Ajoutez ce contrôle dans votre CI/CD (GitHub Actions) :

```
# .github/workflows/security.yml
name: Security Audit

on:
  schedule:
    - cron: '0 8 * * 1' # Chaque lundi matin
  push:
    paths:
      - 'backend/requirements.txt'
      - 'frontend/package.json'

jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Python dependency audit
        run: |
          pip install pip-audit
          pip-audit -r backend/requirements.txt

      - name: Node.js dependency audit
        working-directory: frontend
        run: npm audit --audit-level=high
```

23.4 Partie 3 : Santé de la Base de Données PostgreSQL

23.4.1 3.1 Comprendre l'Autovacuum

PostgreSQL ne supprime pas physiquement les lignes quand vous faites un `DELETE` ou `UPDATE`. Il les marque comme “mortes” (dead tuples) et laisse un processus d’arrière-plan, **autovacuum**, les nettoyer périodiquement. Si l’autovacuum ne suit pas la cadence d’écriture, la table gonfle et les performances se dégradent.

Ce script de diagnostic vous donne une vue claire de l’état de vos tables :

```
-- scripts/db_health.sql
-- Exécuter avec : docker exec -i gitsky_db psql -U postgres gitsky < scripts/db_health.sql

\echo '=== État des tables (lignes mortes vs vivantes) ==='
SELECT
    schemaname,
    relname                                AS table_name,
    n_live_tup                             AS lignes_vivantes,
    n_dead_tup                             AS lignes_mortes,
    CASE WHEN n_live_tup > 0
        THEN round(100.0 * n_dead_tup / n_live_tup, 1)
        ELSE 0
    END                                    AS pct_mortes,
    last_autovacuum::date                  AS dernier_vacuum,
    last_autoanalyze::date                 AS dernier_analyze,
    pg_size_pretty(pg_total_relation_size(relid)) AS taille_totale
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 15;

\echo ''
\echo '=== Taille de la base ==='
SELECT pg_size_pretty(pg_database_size(current_database())) AS taille_base;

\echo ''
\echo '=== 10 plus grosses tables ==='
SELECT
    relname                                AS table_name,
    pg_size_pretty(pg_total_relation_size(relid)) AS taille_totale,
    pg_size_pretty(pg_relation_size(relid))      AS taille_donnees,
    pg_size_pretty(pg_total_relation_size(relid)
        - pg_relation_size(relid))              AS taille_index
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 10;

\echo ''
\echo '=== Requêtes les plus lentes (si pg_stat_statements activé) ==='
SELECT
    round(total_exec_time::numeric, 2) AS temps_total_ms,
    calls,
    round((total_exec_time / calls)::numeric, 2) AS temps_moyen_ms,
    left(query, 100) AS requête
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Exécution :

```
docker exec -i gitsky_db psql -U postgres gitsky < scripts/db_health.sql
```

Interprétation : - pct_mortes > 20% avec dernier_vacuum vieux de plusieurs jours →
 VACUUM manuel recommandé - Une table dont la taille_index dépasse la taille_données →
 indices potentiellement redondants

VACUUM manuel si nécessaire :

```
docker exec gitsky_db psql -U postgres gitsky -c "VACUUM ANALYZE;"
```

23.4.2 3.2 Vérification de la Taille du Disque

Le disque plein est la cause numéro un de corruption silencieuse en production. PostgreSQL ne peut plus écrire dans son WAL, les transactions échouent, et la base peut nécessiter une réparation manuelle.

```
#!/usr/bin/env bash
# scripts/check_disk.sh
# Vérifie l'espace disque et alerte si le seuil est dépassé.

THRESHOLD_WARN=70 # Alerte à 70%
THRESHOLD_CRIT=85 # Critique à 85%
ALERT_EMAIL="${ALERT_EMAIL:-}"

check_path() {
    local path="$1"
    local label="$2"
    local usage

    usage=$(df "$path" 2>/dev/null | awk 'NR==2 {sub(/%/, "", $5); print $5}')
    if [[ -z "$usage" ]]; then return; fi

    if [[ "$usage" -ge "$THRESHOLD_CRIT" ]]; then
        echo "x CRITIQUE : $label à ${usage}% (seuil: ${THRESHOLD_CRIT}%)"
        [[ -n "$ALERT_EMAIL" ]] && \
            echo "Disque $label à ${usage}% sur $(hostname)" | \
            mail -s "[GitSky] ALERTE CRITIQUE: Disque presque plein" "$ALERT_EMAIL"
        return 1
    elif [[ "$usage" -ge "$THRESHOLD_WARN" ]]; then
        echo "Δ ATTENTION : $label à ${usage}% (seuil d'alerte: ${THRESHOLD_WARN}%)"
        [[ -n "$ALERT_EMAIL" ]] && \
            echo "Disque $label à ${usage}% sur $(hostname)" | \
            mail -s "[GitSky] ATTENTION: Disque $label à ${usage}%" "$ALERT_EMAIL"
    else
        echo "✓ $label : ${usage}% utilisé"
    fi
}

echo "=== Vérification de l'espace disque - $(date) ==="
check_path "/" "Disque principal"
check_path "/backups" "Disque sauvegardes" 2>/dev/null || true

# Taille des logs Docker (peut devenir très volumineux)
DOCKER_LOG_SIZE=$(find /var/lib/docker/containers -name "*.log" \
    -exec du -ch {} + 2>/dev/null | tail -1 | cut -f1)
echo "    Logs Docker : $DOCKER_LOG_SIZE"
```

23.4.3 3.3 Rotation des Logs Docker

Par défaut, Docker accumule les logs de vos conteneurs indéfiniment. Sur un serveur avec peu de RAM et de disque, cela peut consommer des Go en quelques mois.

Configurez une limite globale dans `/etc/docker/daemon.json` :

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "50m",
    "max-file": "3"
  }
}
```

Appliquez la configuration :

```
sudo systemctl restart docker
```

Ou par service dans `docker-compose.prod.yml` pour un contrôle plus fin :

```
services:
  backend:
    logging:
      driver: "json-file"
      options:
        max-size: "50m"
        max-file: "5"

  postgres:
    logging:
      driver: "json-file"
      options:
        max-size: "100m"
        max-file: "3"
```

23.5 Partie 4 : Monitoring et Surveillance

23.5.1 4.1 Monitoring de Disponibilité

Vous devez être informé avant vos utilisateurs en cas de panne. Configurez un monitoring externe dès le premier jour.

UptimeRobot (gratuit) — recommandé pour démarrer : 1. Créez un compte sur uptimerobot.com 2. Ajoutez un monitor HTTP(S) vers `https://votre-domaine.com/api/health` 3. Configurez les alertes email / Telegram / Slack 4. Fréquence de vérification : toutes les 5 minutes (plan gratuit)

L'endpoint `/health` est déjà présent dans le template GitSky (dans `app/core/main.py`, servi sous `/api/health` derrière Traefik). Le stack étant **asynchrone** (SQLAlchemy async), la vérification base l'est aussi — et l'endpoint conserve sa charge utile existante (tier, modules activés) en plus du statut base :

```
# app/core/main.py
@app.get("/health")
async def health(db: AsyncSession = Depends(get_db)) -> dict:
    try:
        await db.execute(text("SELECT 1"))
    except Exception:
        raise HTTPException(status_code=503, detail="Database unavailable")
    return {"status": "ok", "database": "ok", "tier": settings.gitisky_tier, ...}
```

Sans ce `SELECT 1`, `/health` répondrait `200` avec une base morte — l'incident passerait inaperçu. Le `503` marque le conteneur non sain côté Traefik et fleet dashboard.

Alertes Traefik — vérifiez vos logs d'accès pour détecter des anomalies :

```
#!/usr/bin/env bash
# scripts/check_errors.sh
# Analyse les logs Traefik pour détecter des taux d'erreurs anormaux.

CONTAINER="${TRAEFIK_CONTAINER:-traefik}"
WINDOW_MINUTES=60
THRESHOLD_5XX=10 # Alerte si plus de 10 erreurs 5xx en 1 heure

ERROR_COUNT=$(docker logs "$CONTAINER" --since "${WINDOW_MINUTES}m" 2>&1 \
    | grep -c '"status":5' || true)

if [[ "$ERROR_COUNT" -ge "$THRESHOLD_5XX" ]]; then
    echo "x ALERTE : $ERROR_COUNT erreurs 5xx détectées dans la dernière heure."
    [[ -n "${ALERT_EMAIL:-}" ]] && \
        echo "$ERROR_COUNT erreurs 5xx en ${WINDOW_MINUTES} minutes sur $(hostname)" | \
        mail -s "[GitSky] Taux d'erreurs anormal" "$ALERT_EMAIL"
else
    echo "✓ Taux d'erreurs normal : $ERROR_COUNT erreur(s) 5xx (seuil: $THRESHOLD_5XX)"
fi
```

23.5.2 4.2 Monitoring Applicatif avec les SecurityEvents

Votre template intègre déjà un `SecurityMiddleware` qui enregistre les comportements suspects dans la table `security_events`. Consultez-la régulièrement :

```
-- Événements de sécurité des dernières 24h
SELECT
    event_type,
    severity,
    ip_address,
    COUNT(*) AS occurrences,
    MAX(created_at) AS dernier_vu
FROM security_events
WHERE created_at > NOW() - INTERVAL '24 hours'
GROUP BY event_type, severity, ip_address
ORDER BY occurrences DESC
LIMIT 20;

-- IPs suspectes (> 100 événements en 1h)
SELECT ip_address, COUNT(*) AS nb_events
FROM security_events
WHERE created_at > NOW() - INTERVAL '1 hour'
GROUP BY ip_address
HAVING COUNT(*) > 100
ORDER BY nb_events DESC;
```

Si vous détectez une IP récurrente dans les événements critiques, bloquez-la au niveau Traefik ou avec `ufw` :

```
# Bloquer une IP avec le firewall Linux
sudo ufw deny from 1.2.3.4 to any
```

23.6 Partie 5 : Mises à Jour

23.6.1 Stratégie de Mise à Jour

Ne jamais mettre à jour en production sans d'abord tester sur un environnement de staging. Le principe est : **staging d'abord, production ensuite**.

```
Développement → Staging (clone de prod) → Production
```

23.6.2 Images Docker

```
#!/usr/bin/env bash
# scripts/update_images.sh
# Met à jour les images Docker et redémarre les services.

set -euo pipefail

PROJECT_DIR="/opt/gitsky"
COMPOSE_FILE="docker-compose.prod.yml"

cd "$PROJECT_DIR"

echo "=== Mise à jour des images Docker - $(date) ==="

# Récupérer les nouvelles versions des images de base. Épingler une LTS
# SUPPORTÉE : Node 20 est en fin de vie depuis 2026-04-30 – utiliser Node 24
# (LTS active), sans quoi la base ne reçoit plus de correctifs de sécurité.
docker pull postgres:16.3-alpine
docker pull node:24-alpine
docker pull python:3.12-slim
docker pull traefik:v3.1

# Reconstruire les images applicatives (Compose v2 : `docker compose`, pas
# `docker-compose` v1 déprécié).
docker compose -f "$COMPOSE_FILE" build --no-cache

# Redémarrer avec les nouvelles images (zéro downtime si répliqué)
docker compose -f "$COMPOSE_FILE" up -d --remove-orphans

# Supprimer les anciennes images non utilisées
docker image prune -f

echo "=== Mise à jour terminée ==="
```

23.6.3 Système d'Exploitation

```
#!/usr/bin/env bash
# scripts/update_os.sh
# Met à jour le système et redémarre si nécessaire (Debian/Ubuntu).

set -euo pipefail

echo "=== Mise à jour du système - $(date) ==="

apt-get update -qq
apt-get upgrade -y --no-install-recommends
apt-get autoremove -y
apt-get autoclean

# Vérifier si un redémarrage est nécessaire
if [[ -f /var/run/reboot-required ]]; then
    echo "⚠ Un redémarrage est requis pour appliquer les mises à jour kernel."
    echo "  Planifiez un redémarrage en maintenance : sudo reboot"
else
    echo "✓ Aucun redémarrage requis."
fi

echo "=== Mise à jour OS terminée ==="
```

23.6.4 Dépendances Applicatives

```
#!/usr/bin/env bash
# scripts/audit_dependencies.sh
# Scan de sécurité des dépendances Python et Node.js.

set -uo pipefail
ERRORS=0

echo "=== Audit des dépendances - $(date) ==="

# Python
echo "--- Backend Python ---"
if pip-audit -r /opt/gitisky/backend/requirements.txt 2>&1; then
    echo "✓ Aucune CVE Python détectée."
else
    echo "✗ Des vulnérabilités ont été détectées dans les dépendances Python."
    ERRORS=$((ERRORS + 1))
fi

# Node.js
echo "--- Frontend Node.js ---"
if (cd /opt/gitisky/frontend && npm audit --audit-level=high 2>&1); then
    echo "✓ Aucune CVE Node.js critique détectée."
else
    echo "✗ Des vulnérabilités critiques ont été détectées dans les dépendances Node.js."
    ERRORS=$((ERRORS + 1))
fi

exit $ERRORS
```

23.7 Partie 6 : Calendrier de Maintenance

23.7.1 Tableau de Bord Récapitulatif

FRÉQUENCE	TÂCHE	DURÉE
QUOTIDIEN (automatique)	✦ Sauvegarde PostgreSQL (automatique via cron)	0 min
	✦ Vérification de l'espace disque (cron)	0 min
	✦ Monitoring de disponibilité (UptimeRobot)	0 min
HEBDOMADAIRE (manuel)	✦ Révision des logs d'erreurs (Traefik + app)	10 min
	✦ Révision des SecurityEvents suspects	5 min
	✦ Vérifier que la dernière sauvegarde existe	2 min
	✦ Vérifier les alertes UptimeRobot	2 min
MENSUEL (manuel)	✦ Mise à jour OS + images Docker	30 min
	✦ Scan CVE des dépendances (pip-audit, npm audit)	15 min
	✦ Test de restauration d'une sauvegarde	20 min
	✦ Analyse de la santé DB (db_health.sql)	10 min
	✦ Révision des logs Docker (taille)	5 min
SEMESTRIEL (manuel)	✦ Rotation SECRET_KEY (re-login de tous les users)	15 min
	✦ Rotation du mot de passe PostgreSQL	15 min
	✦ Révision des utilisateurs DB (supprimer inactifs)	10 min
	✦ Révision des clés SSH autorisées sur le serveur	10 min
	✦ Mise à jour de la politique de sauvegarde	15 min
SUR INCIDENT	✦ Rotation immédiate de TOUS les secrets	
	✦ Analyse des logs autour de l'heure de l'incident	
	✦ Vérification de l'intégrité des données	
	✦ Notification des utilisateurs si données exposées	

23.7.2 Crontab Complète Recommandée

Voici la crontab complète à installer sur votre serveur de production :


```
# =====
# Crontab GitSky – Maintenance Automatisée
# Installer avec : crontab -e
# =====

# Variables d'environnement
MAILTO=""
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# — Sauvegardes —————
# Sauvegarde quotidienne à 2h00
0 2 * * * /opt/gitisky/scripts/backup_db.sh >> /var/log/gitisky_backup.log 2>&1

# — Surveillance disque —————
# Vérification de l'espace disque toutes les heures
0 * * * * /opt/gitisky/scripts/check_disk.sh >> /var/log/gitisky_monitor.log 2>&1

# — Surveillance des erreurs —————
# Analyse des logs d'erreurs toutes les heures
30 * * * * /opt/gitisky/scripts/check_errors.sh >> /var/log/gitisky_monitor.log 2>&1

# — Audit de sécurité —————
# Audit de sécurité réseau chaque dimanche à 3h
0 3 * * 0 /opt/gitisky/scripts/check_security.sh >> /var/log/gitisky_security.log 2>&1

# Scan CVE des dépendances le 1er de chaque mois
0 4 1 * * /opt/gitisky/scripts/audit_dependencies.sh >> /var/log/gitisky_security.log 2>&1

# — Mises à jour —————
# Mise à jour de sécurité OS chaque lundi à 4h (sans redémarrage auto)
0 4 * * 1 apt-get update -qq && apt-get upgrade -y -o Dpkg::Options::="--force-confdef" >>
/var/log/gitisky_updates.log 2>&1

# — Nettoyage —————
# Nettoyage des images Docker non utilisées chaque dimanche
0 5 * * 0 docker image prune -f >> /var/log/gitisky_cleanup.log 2>&1

# Rotation des logs de maintenance (garder 30 jours)
0 6 1 * * find /var/log/gitisky_*.log -mtime +30 -delete 2>/dev/null || true
```

23.8 Partie 7 : Procédures d’Urgence

23.8.1 Restauration d’Urgence

En cas de corruption ou de suppression accidentelle :

```

#!/usr/bin/env bash
# scripts/emergency_restore.sh
# Restaure la base depuis la dernière sauvegarde.
#
# ATTENTION : Ceci remplace TOUTES les données actuelles.
# Faire une sauvegarde de l'état actuel avant de lancer si possible.
#
# Usage : ./scripts/emergency_restore.sh [chemin/vers/backup.sql.gz]

set -euo pipefail

BACKUP_DIR="/backups/postgres"
POSTGRES_CONTAINER="${POSTGRES_CONTAINER:-gitsky_db}"
POSTGRES_DB="${POSTGRES_DB:-gitsky}"
POSTGRES_USER="${POSTGRES_USER:-postgres}"

# Choisir la sauvegarde
if [[ -n "${1:-}" ]]; then
    BACKUP_FILE="$1"
else
    BACKUP_FILE=$(ls -t "$BACKUP_DIR"/backup_*.sql.gz | head -1)
    echo "Utilisation de la dernière sauvegarde : $BACKUP_FILE"
fi

if [[ ! -f "$BACKUP_FILE" ]]; then
    echo "ERREUR : Fichier de sauvegarde introuvable : $BACKUP_FILE"
    exit 1
fi

echo "=== RESTAURATION D'URGENCE ==="
echo "Source : $BACKUP_FILE"
echo "Cible : base '$POSTGRES_DB' dans le conteneur '$POSTGRES_CONTAINER'"
echo ""
echo "ATTENTION : Cette opération va écraser toutes les données actuelles."
read -p "Confirmez-vous ? (tapez 'OUI' en majuscules) : " CONFIRM

if [[ "$CONFIRM" != "OUI" ]]; then
    echo "Restauration annulée."
    exit 0
fi

# Arrêter l'application pour éviter les écritures pendant la restauration
echo "1/4 Arrêt des services applicatifs..."
cd /opt/gitsky && docker compose -f docker-compose.prod.yml stop backend

# Réinitialiser la base
echo "2/4 Réinitialisation de la base..."
docker exec "$POSTGRES_CONTAINER" \
    psql -U "$POSTGRES_USER" -c "
        SELECT pg_terminate_backend(pid)
        FROM pg_stat_activity
        WHERE datname = '$POSTGRES_DB' AND pid <> pg_backend_pid();
    "

docker exec "$POSTGRES_CONTAINER" \
    psql -U "$POSTGRES_USER" -c "DROP DATABASE IF EXISTS ${POSTGRES_DB}_old;"
docker exec "$POSTGRES_CONTAINER" \
    psql -U "$POSTGRES_USER" -c "ALTER DATABASE $POSTGRES_DB RENAME TO ${POSTGRES_DB}_old;"
docker exec "$POSTGRES_CONTAINER" \
    psql -U "$POSTGRES_USER" -c "CREATE DATABASE $POSTGRES_DB OWNER $POSTGRES_USER;"

# Restaurer
echo "3/4 Restauration des données..."
gunzip -c "$BACKUP_FILE" | docker exec -i "$POSTGRES_CONTAINER" \
    psql -U "$POSTGRES_USER" "$POSTGRES_DB"

```

```
# Redémarrer
echo "4/4 Redémarrage des services..."
docker compose -f docker-compose.prod.yml up -d backend

echo ""
echo "=== Restauration terminée ==="
echo "L'ancienne base est préservée sous le nom '${POSTGRES_DB}_old'."
echo "Supprimez-la après vérification : DROP DATABASE ${POSTGRES_DB}_old;"
```

23.8.2 Checklist Post-Incident

Après tout incident de sécurité (accès non autorisé, fuite de données suspectée) :

- ❑ Isoler le système si nécessaire (couper le trafic entrant dans Traefik)
- ❑ Sauvegarder l'état actuel AVANT toute modification (pour l'analyse)
- ❑ Identifier l'étendue : quelles données, quelle fenêtre temporelle ?
- ❑ Révoquer TOUS les tokens JWT actifs (changer SECRET_KEY)
- ❑ Rotater TOUS les secrets (DB, Stripe, SMTP)
- ❑ Analyser les logs Traefik et SecurityEvents autour de l'heure de l'incident
- ❑ Corriger la vulnérabilité exploitée
- ❑ Restaurer depuis sauvegarde si données corrompues
- ❑ Notifier les utilisateurs affectés (RGPD : obligation sous 72h si données personnelles)
- ❑ Documenter l'incident (timeline, impact, mesures prises)
- ❑ Mettre en place des contrôles pour éviter la récurrence

23.9 Maintenance à l'Échelle de la Flotte

Les procédures précédentes valent projet par projet. Sur une flotte de 20-30 projets, elles doivent être **orchestrées** depuis le fleet dashboard (Chap 19) — sinon l'opérateur passe ses journées à jongler entre `ssh` et `docker compose` .

23.9.1 Sauvegarde Multi-Projets

Un cron unique boucle sur **tous les conteneurs de bases projet** et applique la stratégie 3-2-1 à chacun. Chaque projet ayant son propre conteneur PostgreSQL (Chap 18 §2), on énumère les conteneurs `*_db` — et non les bases d'une instance partagée :

```
# scripts/backup-fleet.sh - extrait
#!/usr/bin/env bash
set -euo pipefail

DATE=$(date +%Y%m%d_%H%M%S)
S3_BUCKET="s3://mystudio-backups"

# Conteneurs de bases projet : convention de nommage {projet}_db.
CONTAINERS=$(docker ps --format '{{.Names}}' --filter 'name=_db$')

for container in $CONTAINERS; do
    project="${container%_db}"
    dbname="${project//-/_}" # identifiant PostgreSQL (pas de tiret)
    echo "Sauvegarde de $project..."
    docker exec "$container" pg_dump -U postgres -Fc "$dbname" \
    | gzip > "/backups/${dbname}_${DATE}.dump.gz"
    aws s3 cp "/backups/${dbname}_${DATE}.dump.gz" "$S3_BUCKET/$project/"
done

# Rotation : garder 14 jours de dumps locaux
find /backups -name "*.dump.gz" -mtime +14 -delete
```

Ce script tourne à 02:00 UTC, avant le `kill_check` de 03:00 — les sauvegardes des projets sur le point d'être tués sont ainsi préservées.

23.9.2 Vérification des Sauvegardes de Flotte

Le fleet dashboard expose une colonne « dernière sauvegarde OK » par projet. Un cron hebdomadaire teste la restauration d'un projet aléatoire dans un environnement de staging pour vérifier que les dumps sont exploitables.

Une sauvegarde jamais restaurée est une sauvegarde inutile.

23.9.3 Rotation des Secrets

À l'échelle d'une flotte, la rotation manuelle des secrets projet par projet est irréaliste. Le fleet dashboard automatise :

- **Rotation trimestrielle des secrets DB** : le fleet dashboard génère un nouveau mot de passe, l'applique dans PostgreSQL, met à jour le `.env` du projet, et redémarre le conteneur.
- **Rotation semestrielle des `SECRET_KEY`** : idem, avec invalidation automatique de toutes les sessions actives.

Les secrets Stripe et LLM proxy sont mutualisés au niveau flotte et donc rotatés une seule fois pour tous les projets.

23.9.4 Monitoring de Disponibilité de Flotte

Un unique cron toutes les 60 secondes interroge `/health` sur tous les projets. Un projet muet depuis plus de 5 minutes déclenche l'alerte `deployment_failed` du fleet dashboard (Chap 19). L'opérateur voit l'incident sur son dashboard matinal et peut consulter les logs live sans SSH.

23.9.5 Alerte Disque Consolidée

Sur un VPS mutualisé, le remplissage disque peut venir d'un unique projet qui log excessivement. Une jauge par projet dans le fleet dashboard, alimentée par `docker system df -v`, identifie immédiatement le responsable — bien plus rapide que de fouiller `du -sh /var/lib/docker/*`.

23.10 Conclusion

La maintenance n'est pas une activité optionnelle : c'est ce qui distingue un projet de démonstration d'une plateforme en production fiable. Les trois piliers à mettre en place impérativement dès le premier jour sont :

1. **Les sauvegardes automatiques** — sans elles, tout le reste est inutile.
2. **Le monitoring de disponibilité** — savoir avant vos utilisateurs.
3. **L'alerte disque** — le disque plein est la panne la plus courante et la plus évitable.

Sur une flotte GitSky, ces trois piliers sont **mutualisés au niveau du fleet dashboard** — un seul cron de sauvegardes, un seul monitoring, une seule alerte disque consolidée. Cette centralisation transforme la maintenance d'une activité chronophage à N projets en une routine matinale de 10 minutes sur un tableau unique.

Une fois ces fondations posées, les autres pratiques de ce chapitre peuvent être intégrées progressivement, en commençant par celles qui correspondent aux risques les plus élevés (données financières Stripe → rotation des secrets en priorité).

24 GitSky Studio : Direction Artistique de Flotte

24.1 Introduction

L’industrialisation d’une flotte crée une tension frontale : un même template qui porte 100 projets pousse vers l’**homogénéité visuelle**. Or 100 landings identiques sont un poison — non seulement la conversion s’effondre, mais surtout la mesure est biaisée : si un design générique plombe *toutes* les landings, on ne teste plus la demande d’une idée, on teste notre skin.

Le levier `branding` du générateur (Chap 17) ne résout pas ce problème : couleur, police et logo restent du *theming*. Deux landings token-personnalisées demeurent visiblement sœurs — même squelette, mêmes sections, même rythme.

Le **GitSky Studio** apporte la réponse : un service partagé d’agents IA qui, à partir du signal d’une idée, **conçoit la direction artistique, rédige le contenu, produit les médias et assemble** une vitrine distincte, aux standards GitSky. Ce n’est pas un module de plus greffé sur le template — c’est le framework agentic (Chap 15) **retourné vers l’intérieur** : GitSky applique à sa propre fabrique la technologie qu’il vend à ses projets.

24.2 Le Châssis et la Vitrine

Le Studio n’a de sens que si l’on scinde d’abord chaque projet en deux couches à cycles de vie opposés :

	Châssis (partagé)	Vitrine (propre au projet)
Contenu	backend, auth, sécurité, deps, build, i18n runtime	landing : layout, sections, copy, médias, esthétique
Propriété	template	projet
<code>copier update</code>	propage les correctifs	n’y touche jamais
Liberté artistique	nulle (c’est le moteur)	totale

C’est cette frontière qui réconcilie industrialisation et différenciation : on industrialise le moteur, on libère la carrosserie. Le diff à trois voies de Copier (Chap 17) respecte cette frontière dès lors qu’elle est **déclarée explicitement** — la vitrine est une zone `project-owned`, exclue de la propagation.

24.3 Le Pipeline d'Agents

Le Studio est un **service agentic pré-configuré**, déclaré dans `agent_services.yaml` (Chap 15) au même titre que les services vendus aux projets. Son métier : concevoir des landings GitSky. Un brief de marque circule entre agents spécialisés, chacun produisant un artefact contractuel :

Agent	Entrée	Sortie
Directeur artistique	signal harvest (thread source, audience, verticale)	brief de marque : palette, duo typographique, ton, registre visuel, choix de skin
Rédacteur	brief + verbatim de la source	copy par section, dans la langue exacte de l'audience
Média	brief + copy	prompts → hero, illustrations, éventuellement vidéo, déclinés du brief
Assembleur	brief + copy + médias + catalogue de blocs	arrangement de la landing → Creative Manifest
QA / Guardrails	l'ensemble	vérifie les standards GitSky (voir plus bas)

Le directeur artistique tient le brief comme **contexte partagé** (l'orchestrator et le memory system du Chap 15). Le modèle sous-jacent est **multimodal** : l'agent *voit* ses propres sorties pour les juger et itérer.

Le principe verbatim du harvest (Chap 20) s'étend ici au **design verbatim** : matcher les codes esthétiques de l'audience-source. Une landing pour développeurs issus de Reddit, pour créateurs issus de TikTok, ou pour acheteurs B2B issus de LinkedIn n'obéissent pas aux mêmes codes visuels.

24.4 Le Creative Manifest : Geler la Sortie de l'IA

Un principe non négociable structure toute l'intégration : **l'IA est non-déterministe, mais GitSky exige la reproductibilité**. Un `config.yaml` doit toujours régénérer le même projet — c'est ce qui rend le cycle kill → résurrection du Chap 20 possible.

La règle est donc :

Le Studio tourne **une seule fois**, à la création. Sa sortie — le **Creative Manifest** — est **persistée et versionnée**, jamais re-générée à la volée. Le générateur reste 100 % déterministe : il consomme le manifest figé.

```
# startup-factory-configs/manifests/pain-scraper.yaml – extrait
manifest_version: 3
brief:
  skin: editorial-serif
  palette: { primary: "#1A1A1A", accent: "#C8452D" }
  type_pairing: { display: "Fraunces", body: "Inter" }
  tone: "direct, technique, sans bullshit"
blocks:
  - hero: { headline: "...", media: assets/pain-scraper/hero-3.webp }
  - pain_points: { items: [...] }
  - email_capture: { cta: "Rejoindre la beta" }
media:
  - id: hero-3
    prompt: "..."
    license: generated-owned
```

Autrement dit : **l'agent enrichit le `config.yaml` , il ne remplace pas la génération.** Le manifest est la section `branding` + `landing` du config, en version augmentée. Les médias vivent dans le media store partagé de la flotte.

24.5 Le Modèle de Publication

La question « GitSky déploie-t-il automatiquement, ou faut-il un humain avant publication ? » appelle une réponse de principe :

On barre l'étape **irréversible** (rendre public, indexer, dépenser en acquisition), jamais l'étape de **génération**. Générer est gratuit et réversible ; publier un domaine dédié avec du SEO acquis et des utilisateurs payants ne l'est pas.

C'est la philosophie des tiers appliquée à la publication : cheap à tester, délibéré à engager. Elle donne trois modes indexés sur le **blast radius**, pas un choix binaire :

Mode	Quand	Rôle de l'humain
Auto-publish guardrailé	To (sous-domaine jetable), si les guardrails passent	<i>on-the-loop</i> : surveille le dashboard, hors du chemin critique
Preview-first (défaut sain)	déploiement auto sur <code>x.preview.mystudio.com</code> (noindex), live sur un clic	<i>on-the-loop</i>
HITL bloquant	promotion T1→T2, domaine dédié, campagne payante	<i>in-the-loop</i> : approbation obligatoire

Trois convictions sous-tendent ce modèle :

Ne pas réintroduire le goulot que la fabrique a supprimé. 100 landings × approbation manuelle, et l'opérateur *redevient* le bottleneck que GitSky existe pour éliminer. Au To, l'humain doit être *on the loop*, pas *in it*.

Remplacer « human-in-the-loop » par « guardrails-in-the-loop ». Le gate par défaut est *machine*. L'humain n'intervient que sur échec de guardrail ou fort blast radius. C'est « cadrer, pas brider » — la philosophie du module security (Chap 14) transposée au design.

L'autonomie se gagne (ratchet de confiance). Tant que le Studio n'a pas fait ses preuves, davantage de gates humains. Quand ses guardrails affichent un track record — conversion mesurée et zéro incident de marque, visibles au fleet dashboard — on desserre. Comme un déploiement canari, l'autonomie progresse par paliers de confiance.

24.6 Itération : des Diffs sur le Manifest

Lorsque l'opérateur veut retoucher — « change cette couleur », « réécris cette section », « remplace ce hero » — ces modifications sont des **diffs sur le Creative Manifest**, pas des régénérations.

- Le manifest est la source de vérité, **versionné et réversible comme du code**.
- Édition humaine directe et instruction en langage naturel à l'IA atterrissent toutes deux comme une **nouvelle version de manifest**.
- Le fleet dashboard affiche l'historique de design d'un projet ; un rollback est un simple retour de version.
- La **régénération complète** reste possible, mais c'est l'option nucléaire : explicite, jamais implicite.

Règle de propriété des pixels, pour éviter que l'IA écrase le travail humain :

Zone	Statut
Piloté par le manifest	régénérable — l'IA peut y repasser
Échappatoire full-custom (code à la main)	gelée — jamais régénérée

Un projet qui gradue en T2 et mérite du sur-mesure bascule sa vitrine dans la zone échappatoire, définitivement protégée de toute régénération.

24.7 Les Guardrails : Cadrer, pas Brider

Le cadre est ce qui permet de lâcher l'IA sans qu'elle casse la qualité. L'agent ne peint pas des pixels libres — il **compose dans un système de design borné** (tokens + catalogue de blocs + skins curés). La contrainte garantit à *la fois* qualité et diversité ; la liberté totale produit de la bouillie moyenne.

L'agent QA valide avant tout déploiement :

Guardrail	Vérifie
Accessibilité	contraste WCAG, tailles, alternatives textuelles des médias
Cohérence de marque	respect du brief (palette, typo, ton)
Claims légaux	absence d'allégations factuelles non étayées sur la landing
Licences médias	droits clairs sur chaque asset (généré-possession ou licencié)

24.7.1 Le Paradoxe de l'Homogénéité IA

Un piège à concevoir *contre* : en résolvant « toutes pareilles à cause du template », on risque « toutes pareilles à cause de l'IA » — le même *AI look* sur 100 landings. L'antidote est double : composer dans des skins curés (variété structurelle garantie) et surveiller la diversité inter-projets comme une métrique de flotte à part entière.

24.8 La Boucle de Feedback : la Vraie Douve

Le fleet dashboard mesure déjà la conversion de chaque landing (Chap 19). Le Studio referme la boucle : **quelles directions artistiques convertissent** est réinjecté dans le scoring du directeur artistique.

Le Studio s'améliore alors *parce qu'il est câblé à un portefeuille de données de conversion réelles* — un avantage qu'aucun outil de design générique isolé ne possède. La génération elle-même n'est pas la douve ; la boucle d'apprentissage sur données de flotte l'est.

24.9 Anti-Patterns à Éviter

- **Régénérer à la volée à chaque déploiement.** Casse la reproductibilité et le cycle kill → résurrection. Le manifest se fige.
- **Exiger une approbation humaine pour chaque To.** Réintroduit le goulot d'étranglement opérateur.
- **Lâcher l'IA sans design system.** Produit une homogénéité IA, pire que l'homogénéité template car plus difficile à diagnostiquer.
- **Éditer les fichiers générés sans passer par le manifest.** La prochaine régénération écrase le travail — sauf dans la zone échappatoire déclarée.

Le Studio donne à la flotte sa direction artistique à l'échelle. Il clôt la boucle industrielle : la même architecture agentic sert les produits et la fabrique qui les engendre. La partie suivante revient à la production et à la maintenance de cet ensemble en conditions réelles.

25 Guide de l'Opérateur : Utiliser, Déployer, Maintenir GitSky

Les chapitres précédents ont construit chaque brique de GitSky. Celui-ci les réunit en un **parcours opérateur** de bout en bout : de la première connexion au VPS jusqu'à la routine matinale sur une flotte de dizaines de projets. C'est le chapitre à garder ouvert le jour où l'on passe à l'action.

Trois temps, dans cet ordre :

1. **Utiliser** — générer un projet à partir d'une idée.
2. **Déployer** — le mettre en ligne sur le VPS partagé.
3. **Maintenir** — le garder (et toute la flotte) en bonne santé.

25.1 1. Utiliser : de l'idée au projet

25.1.1 1.1 Préparer le VPS (une seule fois)

Avant le premier projet, le serveur partagé est provisionné **une fois** (Chap 22) : durcissement SSH par clé, pare-feu UFW, Fail2ban, installation de Docker, puis démarrage des services partagés — Traefik (seul exposé), l'instance PostgreSQL des services, le landing collector, le LLM proxy, le GeoIP (Chap 18).

```
# Sur le VPS, une fois : bootstrap des services partagés (Chap 18/22).
cd /opt/gitisky/shared_services
docker compose up -d
```

25.1.2 1.2 Décrire l'idée dans un `config.yaml`

Chaque projet naît d'un fichier de configuration déclaratif (Chap 17). Le tier de départ est **toujours To** — un projet gagne ses tiers par signal mesurable, il ne démarre jamais au niveau final (Chap 2) :

```
# projects/pain-scraper.yaml
project:
  name: pain-scraper
  tier: t0
  domain: pain-scraper.mystudio.com
branding:
  primary_color: "#4F46E5"
landing:
  skin: clean
  blocks:
    - type: hero
      headline: "Trouvez la douleur avant d'écrire le code"
    - type: email_capture
      cta: "Rejoindre la liste"
```

25.1.3 1.3 Générer le projet

Une commande produit un projet démarrable (Chap 17) :

```
copier copy \  
  --data-file projects/pain-scrafer.yaml \  
  https://github.com/mystudio/gitsky-template \  
  ~/projects/pain-scrafer
```

Le générateur résout le tier en flags de modules, scaffolde `app/domain/`, génère les migrations, applique le branding, produit le `docker-compose.yml` et le `.env` (avec des **secrets aléatoires par projet** — `SECRET_KEY`, mot de passe PostgreSQL), enregistre le projet au fleet dashboard, et crée le commit initial. Le `.env.backup.example` est pré-rempli avec les noms réels du projet pour la maintenance (Chap 23).

Pour tester dix idées, on écrit dix YAML et on boucle — *zero-to-N* projets en quelques minutes (Chap 17 §Bootstrapping d'une Flotte).

25.2 2. Déployer : du projet à la mise en ligne

25.2.1 2.1 Le modèle de déploiement

Un projet généré est **auto-suffisant** : son `docker-compose.yml` de production décrit tout ce dont il a besoin. Selon le tier (Chap 21) :

- **T0** — frontend (landing statique art-dirigée, Chap 24) + backend minimal. Pas de base propre : les leads vont au landing collector partagé (Chap 18 §3).
- **T1 / T2** — frontend + backend + **son propre conteneur PostgreSQL** + service `migrate` éphémère. Le conteneur DB reste sur le réseau interne, jamais exposé (Chap 23 §2.3).

Une seule image de production par service, identique pour les trois tiers : ce sont les flags `MODULE_*` du `.env` qui décident, au démarrage, quels routers et migrations se chargent. Le nombre de workers Gunicorn vient de `WEB_CONCURRENCY` (1/2/4 selon le tier) — promouvoir T1 → T2 = changer cette valeur et redéployer, **sans rebuild** (Chap 21).

25.2.2 2.2 Déployer un projet

```
# Sur le VPS, dans le dossier du projet.  
cd /opt/gitsky/projects/pain-scrafer  
# .env généré à la création : contient tier, secrets, WEB_CONCURRENCY.  
docker compose up -d --build
```

Compose build les images, crée la base du projet (T1/T2), applique les migrations via le service `migrate`, puis démarre backend et frontend. Traefik détecte les labels du projet et route `pain-scrafer.mystudio.com` (frontend) et `api.pain-scrafer.mystudio.com` (backend) avec un

certificat SSL wildcard automatique (Chap 22).

25.2.3 2.3 Vérifier le déploiement

L'endpoint `/health` sonde la base (`SELECT 1`) et renvoie `503` si elle est injoignable — le `HEALTHCHECK` Docker s'en sert, et Traefik n'envoie du trafic qu'à un conteneur sain (Chap 21/23) :

```
curl https://api.pain-scraper.mystudio.com/api/health
# {"status":"ok","database":"ok","tier":"t0",...}
```

Le projet apparaît alors dans le fleet dashboard, où l'opérateur suit son statut sans SSH (Chap 19).

25.3 3. Maintenir : garder la flotte en bonne santé

La maintenance n'est pas optionnelle : c'est ce qui distingue une démo d'une plateforme fiable (Chap 23). Sur une flotte, elle est **mutualisée** — un seul jeu de crons pour tous les projets, piloté depuis le VPS.

25.3.1 3.1 Les trois piliers (dès le premier jour)

1. **Sauvegardes automatiques** — sans elles, tout le reste est inutile.
2. **Monitoring de disponibilité** — savoir avant les utilisateurs.
3. **Alerte disque** — le disque plein est la panne la plus courante et évitable.

25.3.2 3.2 L'automatique : `crontab.fleet`

Un unique crontab (`shared_services/crontab.fleet` , Chap 23 §6) orchestre toute la flotte :

Fréquence	Tâche	Script
60 s	Poll <code>/health</code> de la flotte → alerte <code>deployment_failed</code>	<code>fleet-health.sh</code>
02:00	Sauvegarde 3-2-1 de toutes les bases projet	<code>backup-fleet.sh</code>
03:00	Kill check (déclenché par le dashboard, Chap 20)	—
Horaire	Jauge disque consolidée	<code>fleet-disk.sh</code>

La sauvegarde de 02:00 précède **volontairement** le kill check de 03:00 : un projet sur le point d'être tué garde une sauvegarde fraîche. Chaque conteneur DB étant isolé (Chap 18 §2), `backup-fleet.sh` boucle sur les conteneurs `*_db` et en dump un par un.

25.3.3 3.3 Le manuel : la routine

Les tâches qui demandent un œil humain sont regroupées dans le calendrier de maintenance (`shared_services/MAINTENANCE.md` , Chap 23 §6) :

- **Hebdo** — revue des erreurs 5xx et des `security_events` suspects ; vérifier que la dernière sauvegarde de flotte existe.
- **Mensuel** — **tester une restauration** au hasard (une sauvegarde jamais restaurée est inutile), analyse santé DB (`db_health.sql`), scan CVE des dépendances.
- **Semestriel** — rotation des `SECRET_KEY` et mots de passe DB.

25.3.4 3.4 Mettre à jour toute la flotte

Un correctif du template se propage par `copier update` — chaque projet rejoue ses migrations et reçoit les évolutions du châssis, sans perdre sa vitrine art-dirigée (Chap 17 §Mise à Jour, Chap 24) :

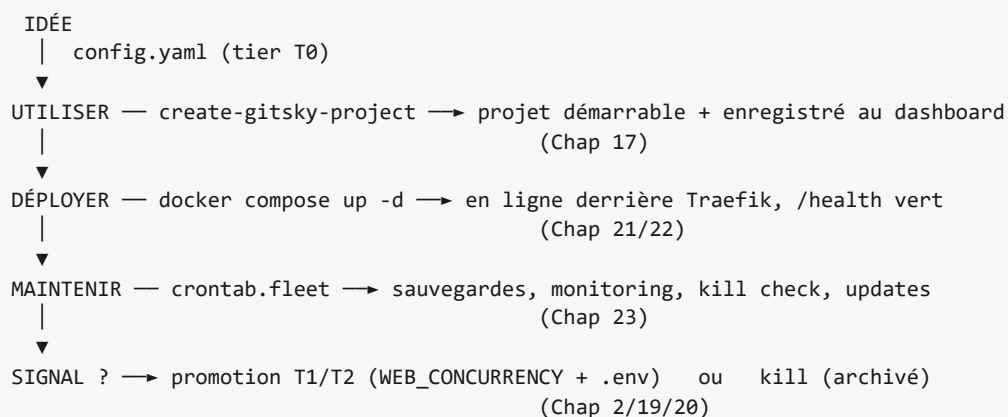
```
cd /opt/gitisky/projects/pain-scraper
copier update          # applique les nouveautés du template
docker compose up -d --build
```

Les images de base se rafraîchissent via `update_images.sh` (Chap 23 §5), qui épingle des versions **supportées** (Node LTS active, etc.).

25.3.5 3.5 En cas d'incident

Un runbook incident (Chap 23 §7) couvre la restauration d'urgence (`emergency_restore.sh` , qui préserve l'ancienne base sous `{db}_old`) et la checklist post-incident (isoler, rotater tous les secrets, analyser les logs, notifier sous 72 h si données personnelles — RGPD).

25.4 Le cycle complet, en une image



C'est cette boucle — répétée sur des dizaines d'idées, avec discipline de portefeuille — qui transforme GitSky d'un template en une **usine à hypothèses de startup**.

26 Conclusion : GitSky, un Template pour Industrialiser le SaaS

26.1 Ce que nous avons construit

En vingt-cinq chapitres, nous sommes partis d'une page blanche pour arriver à un **template industriel** capable de porter un portefeuille de projets à des tiers de maturité variés, du test rapide d'une idée jusqu'au SaaS en production avec revenus récurrents. Le Chap 25 réunit d'ailleurs tout le parcours opérateur — **utiliser, déployer, maintenir** — en un seul guide de bout en bout.

GitSky n'est plus une seule application, mais un **système** :

- Un template paramétrable en trois tiers (To/T1/T2).
- Une architecture core + modules qui rend chaque projet à la fois minimal et extensible.
- Un générateur (`create-gitsky-project`) qui produit un projet démarrable en une commande.
- Des services partagés qui mutualisent l'infrastructure sur un unique VPS.
- Un fleet dashboard qui unifie la vision de la flotte et pilote le cycle de vie.
- Une discipline de portefeuille avec critères numériques de promotion et kill mechanism automatique.

26.1.1 L'Architecture Finale

```
+-----+
| ÉCOSYSTÈME GitSky |
+-----+
|
| Template GitSky (core + modules)
|   core      : FastAPI, Auth, Admin shell, SEO
|   modules   : Onboarding, Tutorials, Analytics,
|               Security, i18n, Agentic, Monetization
|   domain    : Métier propre à chaque projet
|
| Générateur create-gitsky-project (Chap 17)
|   config.yaml → projet prêt à démarrer
|
| Services Partagés VPS (Chap 18)
|   Traefik wildcard SSL
|   PostgreSQL (données des services) + DB par projet
|   Landing collector, LLM proxy, GeoIP, SMTP
|
| Fleet Dashboard (Chap 19)
|   Vue unifiée, alertes, kill mechanism
|
| Cycle de Vie (Chap 20)
|   Harvest → T0 → T1 → T2 → Émancip. ou Kill
|
+-----+
```

26.2 Les Huit Principes Fondamentaux

Tout au long de ce parcours, huit règles d'or ont guidé chaque décision architecturale :

1. **Un template, trois tiers.** Aucun projet ne démarre au niveau final — il gagne ses tiers par signal mesurable.
2. **Core minimal, modules activables.** Un projet ne paie que le coût des fonctionnalités qu'il utilise.
3. **Isolation stricte par projet.** Une DB, un domaine, une allocation Stripe metadata-namespacée — jamais de code partagé entre projets.
4. **Mutualisation des services partagés.** Un seul LLM proxy, un seul Traefik, un seul landing collector — le VPS porte le socle. La base de données, elle, est **isolée par projet** (un conteneur PostgreSQL chacun) : c'est ce qui contient les pannes et la charge CPU/I-O d'un projet non validé sans toucher aux autres (Chap 18 §2).
5. **Un unique Dockerfile pour tous les tiers.** L'empreinte varie en mémoire au runtime, jamais en artefacts de build.
6. **Génération et mise à jour reproductibles.** Copier + hooks Python transforment un `config.yaml` en projet prêt, et propagent les correctifs à toute la flotte.
7. **Kill par défaut, pas par exception.** Un projet qui n'atteint pas ses critères est archivé automatiquement. C'est le succès du protocole, pas un échec.
8. **Le fleet dashboard est le contrat.** Aucun projet n'existe en dehors du dashboard. Cette convention rend la flotte pilotable à long terme.

26.3 Ce que GitSky rend Possible

À l'échelle d'un opérateur solo ou d'une petite équipe, l'objectif est clair : **passer de l'idée à la webapp déployée en heures plutôt qu'en semaines**, et maintenir plusieurs dizaines de projets à des stades variés sans se perdre.

Les gains mesurables :

Métrique	Avant template	Avec GitSky
Temps idée → landing live	1-3 jours	< 5 min
Temps landing → MVP	2-4 semaines	2-3 jours
Coût d'infrastructure pour 30 projets	~500 €/mois (VPS multiples)	~20 €/mois (VPS mutualisé)
Correctif de sécurité propagé à N projets	Manuel, plusieurs heures	<code>copier update</code> × N, quelques minutes
Détection d'un projet zombie	Manuelle, souvent oubliée	Kill automatique à J+21

26.4 Prochaines Étapes

Le template est extensible. Pistes d'évolution prioritaires :

- **Modules supplémentaires** : content moderator, communauté et commentaires, notifications push, feature flags par utilisateur.
- **Automatisation du harvest** : agents dédiés à l'exploration de sources (Reddit API, G2 scraping, HackerNews Algolia) et à la production automatique de `config.yaml`.
- **CI/CD intégré au fleet dashboard** : tests par projet, déploiement multi-projets sur push.
- **Monitoring avancé** : métriques d'usage anonymisées agrégées au niveau flotte pour détecter les patterns émergents.
- **Skills GitSky en mode agent** : un opérateur demande « génère et déploie 10 projets sur ces 10 idées », l'agent orchestre la boucle complète.
- **Marketplace de modules** : ouvrir la contribution externe de modules réutilisables (paiements alternatifs, intégrations tierces).

26.5 Un Mot sur la Philosophie GitSky

Le nom du projet n'est pas seulement un titre technique — c'est un objectif. Construire des systèmes où la machine gère la complexité et l'infrastructure, pour laisser l'humain se concentrer sur ce qui compte : identifier une vraie demande, écouter les utilisateurs, itérer sur le produit.

Un template ne fait pas de bons produits — il retire les frottements qui empêchaient de tester rapidement lesquels le seraient. La discipline de portefeuille (Chap 2 et Chap 19) est ce qui transforme la vitesse d'exécution en apprentissage cumulatif : chaque projet tué apprend quelque chose sur le marché ou sur la méthode ; chaque projet qui monte de tier valide un signal de plus en plus fort.

En maîtrisant cette stack et cette discipline, vous n'avez pas seulement appris à construire une webapp — vous avez appris à opérer une **usine à hypothèses de startup**.

Bravo pour être arrivé au bout de ce Tome 1.